

**BỘ THÔNG TIN VÀ TRUYỀN THÔNG
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**



**TS. VŨ HỮU TIẾN
ThS. ĐỖ THỊ LIÊN**

**BÀI GIẢNG
NGÔN NGỮ LẬP TRÌNH JAVA**

**Mã học phần: INT13108
(03 tín chỉ)**

Hà Nội, 11/2019

MỤC LỤC

MỤC LỤC.....	2
DANH SÁCH HÌNH VẼ	6
CHƯƠNG 1. TỔNG QUAN VỀ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VÀ NGÔN NGỮ LẬP TRÌNH JAVA.....	7
1.1. Các phương pháp tiếp cận trong lập trình ứng dụng	7
1.2. Các khái niệm cơ bản của lập trình hướng đối tượng.....	7
1.2.1. Lớp và đối tượng.....	8
1.2.2. Phương thức và thông điệp	8
1.2.3. Tính đóng gói.....	9
1.2.4. Kế thừa.....	9
1.2.5. Đa hình.....	9
1.3. Giới thiệu về ngôn ngữ lập trình hướng đối tượng Java.....	9
1.3.1. Lịch sử của Java.....	9
1.3.2. Đặc trưng của ngôn ngữ Java.....	10
1.3.3. Các công nghệ Java.....	12
1.3.4. Áp dụng Java trong thực tế	12
1.3.5. Môi trường lập trình Java.....	12
1.3.6. Cấu trúc chương trình Java	13
1.3.7. Chương trình Java đầu tiên :	14
1.4. Các kiểu dữ liệu trong Java	16
1.5. Các phép toán cơ bản trong Java.....	17
1.5.1. Biến	17
1.5.2. Phép toán số học	18
1.5.3. Phép toán trên các bit.....	19
1.5.4. Phép toán chỉ mối quan hệ	19
1.5.5. Phép toán logic.....	20
1.5.6. Độ ưu tiên của các phép toán.....	21
1.6. Cấu trúc lệnh trong Java.....	21
1.6.1. Lệnh in dữ liệu ra màn hình	21
1.6.2. Lệnh nhập dữ liệu từ bàn phím	23
1.6.3. Các lệnh điều khiển trong Java	24
BÀI TẬP CHƯƠNG 1	28

CHƯƠNG 2. LỚP VÀ ĐỐI TƯỢNG TRONG JAVA	29
2.1. Cơ bản về lớp	29
2.1.1. Khai báo lớp	29
2.1.2. Mô tả lớp bằng UML	30
2.1.2.1. Mô tả thuộc tính có cú pháp như sau:	31
2.1.2.2. Mô tả hành vi (operation) có cú pháp như sau:	32
2.1.3. Phương thức của lớp	32
2.1.4. Truyền tham số cho phương thức	34
2.1.5. Phương thức khởi tạo (Constructor)	36
2.1.6. Khai báo và khởi tạo đối tượng	36
2.1.7. Lớp lồng nhau và lớp nội bộ	38
2.2. Phân loại biến trong Java	40
2.3. Một số thư viện cơ bản trong Java	42
2.3.1. Lớp bao ngoài (Wrapper class)	42
2.3.2. ArrayList	43
2.3.3. Lớp xử lý chuỗi String	44
2.3.4. Lớp java.lang.Math	45
BÀI TẬP CHƯƠNG 2	46
CHƯƠNG 3. TÍNH KẾ THỪA VÀ ĐA HÌNH TRONG JAVA	47
3.1. Mối quan hệ giữa các lớp	47
3.1.1. Mối quan hệ “Phụ thuộc” – Dependency	47
3.1.2. Mối quan hệ “Kết nối” – Association:	47
3.1.3. Mối quan hệ “Kết nối hai chiều” – Bidirectional Association	49
3.1.4. Mối quan hệ “Thành phần” – Aggregation	49
3.1.5. Mối quan hệ “Hợp thành” – Composition	50
3.1.6. Mối quan hệ “Kế thừa” – Inheritance	51
3.1.7. Kế thừa phương thức khởi tạo của lớp cha	53
3.1.8. Thiết kế cây kế thừa	54
3.2. Tính đa hình trong Java	57
3.2.1. Tính đa hình	57
3.2.2. Cài đè phương thức (Override)	59
3.2.3. Cài chồng phương thức (Overload)	60

3.2.4. Lớp trừu tượng và phương thức trừu tượng.....	61
3.2.5. Lớp giao diện (interface).....	62
BÀI TẬP CHƯƠNG 3	65
CHƯƠNG 4. XỬ LÝ NHẬP/XUẤT TRONG	67
4.1. File và luồng dữ liệu.....	67
4.2. Lớp File	68
4.3. Kiến trúc luồng xuất dữ liệu ra file	69
4.3.1. Dữ liệu dạng byte	70
4.3.2. Dữ liệu dạng văn bản	71
4.4. Ghi/đọc chuỗi ký tự ra tệp văn bản	71
4.5. Đọc/ghi dữ liệu luồng byte	73
4.5.1. Đọc/ghi dữ liệu dạng cơ bản	73
4.5.2. Đọc/ghi dữ liệu là các đối tượng.....	74
BÀI TẬP CHƯƠNG 4	78
CHƯƠNG 5. XỬ LÝ NGOẠI LỆ TRONG JAVA	79
5.1. Xử lý ngoại lệ	79
5.2. Khối try/catch.....	83
5.2.1. Hoạt động của khối try/catch	83
5.2.2. Xử lý nhiều ngoại lệ.....	85
5.3. Ném ngoại lệ	86
5.4. Lan truyền lỗi ngoại lệ.....	86
5.5. Ngoại lệ được kiểm tra và không được kiểm tra	87
5.6. Lỗi do người dùng định nghĩa	88
BÀI TẬP CHƯƠNG 5	89
CHƯƠNG 6. LẬP TRÌNH ĐA LUỒNG	90
6.1. Đa luồng (Multithreading).....	90
6.2. Các cách tạo luồng	90
6.3. Vòng đời của một luồng.....	94
6.4. Tạm ngừng, chạy lại và ngừng luồng.....	95
6.5. Mức độ ưu tiên và lập lịch cho các luồng	96
6.6. Đồng bộ (Synchronization)	98
6.7. Tương tác giữa các luồng	100

BÀI TẬP CHƯƠNG 6	104
CHƯƠNG 7. LẬP TRÌNH GIAO DIỆN TRONG JAVA	105
7.1. Lập trình giao diện với các đối tượng cơ bản.....	105
7.2. Container	106
7.2.1. Frame	107
7.2.2. Panel.....	107
7.2.3. Dialog.....	108
7.3. Component	109
7.3.1. Label	109
7.3.2. TextField	110
7.3.3. TextArea	111
7.3.4. Button.....	111
7.3.5. Checkbox và RadioButton	113
7.3.6. Choice	113
7.4. Layout.....	115
7.4.1. FlowLayout Manager.....	115
7.4.2. BorderLayout Manager.....	115
7.4.3. GridLayout Manager.....	116
7.5. Xử lý sự kiện	118
7.5.1. Khái niệm xử lý sự kiện.....	118
7.5.2. Các bước lập trình xử lý sự kiện	120
7.5.3. Các lớp xử lý sự kiện	120
7.6. Applet	123
7.6.1. Khái niệm Applet.....	123
7.6.2. Cấu trúc của Applet	123
7.7. Lập trình giao diện với Swing.....	127
7.7.1. Tạo các thành phần giao diện với Java Swing	127
7.7.2. Một số ví dụ với Swing.....	128
BÀI TẬP CHƯƠNG 7	129
TÀI LIỆU THAM KHẢO	130

DANH SÁCH HÌNH VẼ

Hình 1. 1. Biên dịch chương trình theo cách truyền thống	11
Hình 1. 2 Biên dịch và chương trình Java	11
Hình 1. 3 Cấu trúc chương trình Java.....	13
Hình 1. 4 Cách khai báo một lớp.....	13
Hình 1. 5 Cách khai báo một phương thức	14
Hình 1. 6 Các lệnh trong một phương thức.....	14
Bảng 2. 1 Các từ khóa mô tả phạm vi truy nhập	33
Bảng 2. 2 Các lớp bao ngoài của dữ liệu cơ bản.....	42
Bảng 2. 3 So sánh Array và ArrayList	44
Bảng 4. 1 Một số phương thức của lớp file.....	68
Bảng 5. 1 Một số lớp ngoại lệ thường gặp	83
Bảng 5. 2 Luồng xử lý try/catch/finally	85
Bảng 7. 1 Một số phương thức của Label	110
Bảng 7. 2 Một số phương thức của TextField.....	111
Bảng 7. 3 Một số phương thức của TextArea	111
Bảng 7. 4 Một số lớp xử lý sự kiện	121

CHƯƠNG 1. TỔNG QUAN VỀ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VÀ NGÔN NGỮ LẬP TRÌNH JAVA

1.1. Các phương pháp tiếp cận trong lập trình ứng dụng

Lập trình là công đoạn quan trọng chủ chốt và không thể thiếu để tạo ra sản phẩm phần mềm. Phần mềm càng trở nên đa dạng và ngành công nghiệp phần mềm càng phát triển thì người ta càng thấy rõ tầm quan trọng của phương pháp lập trình.

Phương pháp lập trình tốt không chỉ đảm bảo tạo ra phần mềm tốt mà còn hỗ trợ thiết kế phần mềm có tính mở và hỗ trợ khả năng sử dụng lại các module. Nhờ đó chúng ta có thể dễ dàng bảo trì, nâng cấp phần mềm cũng như giảm chi phí phát triển phần mềm.

Trong những thập kỷ 1970, 1980, phương pháp phát triển phần mềm chủ yếu là lập trình có cấu trúc (structured programming). Cách tiếp cận cấu trúc đối với việc thiết kế chương trình dựa trên chiến lược chia để trị: Để giải một bài toán lớn, chúng ta tìm cách chia nó thành các bài toán nhỏ hơn và giải riêng từng bài; để giải mỗi bài, chúng ta coi nó như một bài toán mới và có thể tiếp tục chia nó thành các bài toán nhỏ hơn; cuối cùng, ta sẽ đi đến những bài toán có thể giải ngay được mà không cần phải chia tiếp. Cách tiếp cận này được gọi là lập trình từ trên xuống (top-down programming).

Lập trình từ trên xuống là một phương pháp tốt và đã được áp dụng thành công cho phát triển rất nhiều phần mềm. Tuy nhiên, cùng với sự đa dạng và phức tạp của phần mềm, phương pháp này bộc lộ những hạn chế. Trước hết, nó hầu như chỉ đáp ứng việc tạo ra các lệnh hay là các quy trình để giải quyết một bài toán. Dần dần, người ta nhận ra rằng thiết kế các cấu trúc dữ liệu cho một chương trình có tầm quan trọng không kém việc thiết kế các hàm/thủ tục và các cấu trúc điều khiển. Lập trình từ trên xuống không quan tâm đủ đến dữ liệu mà chương trình cần xử lý.

Thứ hai, với lập trình từ trên xuống, chúng ta khó có thể tái sử dụng các phần của chương trình này cho các chương trình khác. Bằng việc xuất phát từ một bài toán cụ thể và chia nó thành các mảnh sao cho thuận tiện, cách tiếp cận này có xu hướng tạo ra một thiết kế đặc thù cho chính bài toán đó. Chúng ta khó có thể nâng lấy một đoạn mã lớn từ một chương trình cũ lắp vào một dự án mới mà không phải sửa đổi.

Thứ ba, môi trường hoạt động trong thực tế của các ứng dụng luôn thay đổi. Dẫn đến việc yêu cầu phần mềm cũng phải liên tục thay đổi theo để đáp ứng nhu cầu của người dùng nếu không muốn phần mềm bị đào thải. Do đó, một thiết kế linh hoạt mềm dẻo là cái mà các nhà phát triển phần mềm mong muốn. Phương pháp tiếp cận từ dưới lên (bottom-up) hỗ trợ tốt hơn cho tính linh hoạt mềm dẻo đó. Trong thực tế, thiết kế và lập trình từ trên xuống thường được kết hợp với thiết kế và lập trình từ dưới lên. Trong tiếp cận từ dưới lên, từ các vấn đề mà ta đã biết cách giải và có thể đã có sẵn các thành

phần tái sử dụng được chúng ta xây dựng dần theo hướng lên trên, hướng đến một giải pháp cho bài toán tổng.

Các thành phần tái sử dụng được nên có tính module hóa cao nhất có thể. Mỗi module là một thành phần của một hệ thống lớn hơn, nó tương tác với phần còn lại của hệ thống theo một cách đơn giản và được quy ước chặt chẽ. Chi tiết về những gì xảy ra bên trong module không cần được xem xét đến đối với hệ thống nói chung. Đây được gọi là che giấu thông tin, một trong những nguyên lý quan trọng nhất của công nghệ phần mềm. Các module hỗ trợ dạng che giấu thông tin này bắt đầu trở nên phổ biến trong các ngôn ngữ lập trình đầu thập kỷ 80. Từ đó, một hình thức tiên tiến hơn của chính ý tưởng đó đã lan rộng trong ngành công nghệ phần mềm. Cách tiếp cận đó được gọi là lập trình hướng đối tượng (object-oriented programming).

1.2. Các khái niệm cơ bản của lập trình hướng đối tượng

1.2.1. Lớp và đối tượng

Trong thực tế, các sự vật có cùng thuộc tính hay trạng thái thì ta có thể nhóm chung các sự vật đó vào cùng một nhóm hay còn gọi là **lớp** sự vật. Ví dụ lớp động vật bốn chân gồm những động vật bốn chân, lớp phương tiện giao thông gắn máy gồm những phương tiện gắn máy, v.v. Trong các lớp đó, mỗi sự vật được coi là một đối tượng cụ thể của lớp đó.

Trong mỗi ứng dụng Java, các đối tượng có đặc điểm tương tự nhau được xếp vào cùng một nhóm, đó là lớp (class). Mỗi lớp mô tả các đặc điểm chung cho các đối tượng của lớp đó. Cụ thể, một định nghĩa lớp mô tả tất cả các thuộc tính của các đối tượng thành viên của lớp đó và các phương thức thực thi hành vi của các đối tượng đó.

Các đối tượng được tạo ra khi chương trình chạy và lớp là khuôn mẫu mà từ đó có thể tạo ra các đối tượng thuộc lớp đó. Mỗi đối tượng được tạo ra từ lớp được gọi là một thực thể (instance) của lớp đó.

1.2.2. Phương thức và thông điệp

Trong Java, mỗi hành vi của một đối tượng trong một lớp được gọi là phương thức của lớp đó. Ví dụ, chúng ta thấy các đối tượng ô tô đều có chức năng chuyển số thì trong lớp ô tô chúng ta sẽ tạo một phương thức chuyển số cho lớp này.

Các đối tượng dùng các thông điệp (message) để liên lạc với nhau. Nhìn từ phương diện lập trình, việc gửi một thông điệp tới một đối tượng chính là gọi một phương thức của đối tượng đó, còn việc một đối tượng nhận được một thông điệp chính là việc một phương thức của nó được một đối tượng khác gọi. Ví dụ khi người dùng click chuột vào một nút bấm tại cửa sổ chương trình thì một thông điệp sẽ được gửi đến đối tượng điều khiển nút bấm đó để thông báo rằng nút bấm đó đã được click.

1.2.3. Tính đóng gói

Dữ liệu của một đối tượng chỉ được thay đổi bởi chính đối tượng đó. Đặc tính này gọi là tính bao đóng. Chúng ta có thể tạo một lớp được bao đóng hoàn toàn trong Java bằng việc tạo tất cả thành viên dữ liệu của lớp chỉ có thể hoạt động trong phạm vi của lớp đó. Đối tượng có thể giao tiếp với các phần khác của chương trình thông qua các phương thức cụ thể gọi là giao diện (interface).

1.2.4. Kế thừa

Là quan hệ mang tính phân cấp mà trong đó các hành vi của một lớp được kế thừa bởi các lớp được dẫn xuất trực tiếp hoặc gián tiếp từ lớp đó. Đây là cơ chế cho phép định nghĩa một lớp mới dựa trên định nghĩa của một lớp có sẵn, sao cho tất cả các thành viên của lớp cha cũng có mặt trong lớp con và các đối tượng thuộc lớp con có thể được sử dụng thay cho đối tượng của lớp cha ở bất cứ đâu.

1.2.5. Đa hình

Đa hình là khả năng tồn tại ở nhiều hình thức khác nhau. Trong hướng đối tượng, đa hình đi kèm với quan hệ kế thừa và nó có nghĩa rằng cùng một cái tên có thể được hiểu theo các cách khác nhau tùy từng tình huống. Ví dụ chúng ta có các lớp Hình Vuông, Hình Tròn là các lớp con của lớp Hình Học. Khi ta sử dụng một đối tượng Hình Học, đối tượng này có thể được hiểu là Hình Vuông hoặc Hình Tròn tùy theo từng ngữ cảnh cụ thể.

1.3. Giới thiệu về ngôn ngữ lập trình hướng đối tượng Java

1.3.1. Lịch sử của Java

Cho đến tận ngày nay, đóng góp quan trọng nhất của cuộc cách mạng vi xử lý chính là việc cho ra đời các thế hệ máy tính cá nhân. Ngoài ra, bộ vi xử lý còn có tác động mạnh mẽ tới việc phát triển các thiết bị điện tử dân dụng thông minh. Nhận thức được điều này, Sun microsystems vào năm 1991 đã phát triển dự án nghiên cứu nội bộ của công ty do James Gosling dẫn đầu, kết quả là ngôn ngữ lập trình hướng đối tượng dựa trên C++ mà Sun gọi là Java ra đời.

Mục tiêu chính của Java là có thể viết các chương trình sẽ chạy trên nhiều hệ thống máy tính và thiết bị điện tử khác nhau. Điều này được gọi là “viết một lần, chạy ở bất cứ đâu”.

Công nghệ Web phát triển bùng nổ vào những năm đầu thập niên 90 và Sun đã thấy được tiềm năng của việc sử dụng Java để thêm nội dung động như tính tương tác và hình ảnh động vào các trang web. Nhờ vào sự phát triển đúng hướng này, Java đã thu hút sự chú ý của cộng đồng doanh nghiệp, những khách hàng thường xuyên phải sử dụng web. Java hiện được sử dụng để phát triển các ứng dụng doanh nghiệp quy mô lớn, để tăng cường chức năng của máy chủ web, để cung cấp ứng dụng cho thiết bị tiêu dùng (ví dụ: điện thoại di động, điện thoại thông minh, thiết bị thu và giải mã tín hiệu truyền

hình (set-top box),...) và cho nhiều mục đích khác. Sun microsystems đã được Oracle mua lại vào năm 2009. Tại hội nghị JavaOne 2010, Oracle đã thông báo rằng 97% máy tính để bàn doanh nghiệp, ba tỷ thiết bị cầm tay và 80 triệu thiết bị truyền hình được sử dụng Java. Hiện tại có hơn 9 triệu nhà phát triển Java so với 4,5 triệu trong năm 2005. Java hiện là ngôn ngữ phát triển phần mềm được sử dụng rộng rãi nhất trên thế giới.

1.3.2. Đặc trưng của ngôn ngữ Java

Đơn giản:

Các nhà thiết kế ra ngôn ngữ Java cố gắng phát triển ngôn ngữ lập trình dễ dàng cho người học. Họ muốn tạo ra ngôn ngữ thân thiện với hầu hết các lập trình viên. Do vậy, họ loại bỏ những khái niệm phức tạp của C và C++ như con trỏ, struct, union,...

Hướng đối tượng:

Tất cả mọi thứ đề cập đến trong Java đều liên quan đến các đối tượng được định nghĩa trước, thậm chí hàm chính của một chương trình viết bằng Java (đó là hàm main) cũng phải đặt bên trong một lớp. Dữ liệu (Data) và phương thức (Method) xử lý dữ liệu là những khái niệm được nhắc nhiều đến trong Java. Dữ liệu mô tả trạng thái của đối tượng, trong khi phương thức mô tả hành vi của đối tượng.

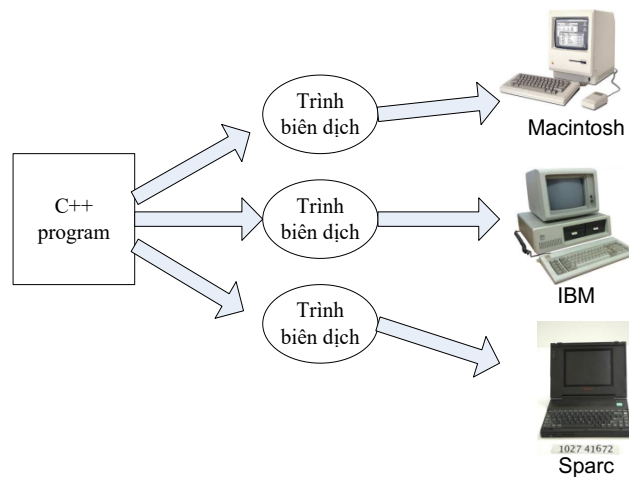
Độc lập với các nền tảng (Platform independent):

Độc lập với nền tảng được coi là khả năng chương trình có thể chạy được trên các hệ điều hành khác nhau (Windows, Unix, Linux,...). Java độc lập với nền tảng tại mức chương trình (source code level) và mức nhị phân (byte code).

Độc lập ở mức chương trình có nghĩa là chương trình có thể chuyển từ hệ điều hành này sang một hệ điều hành khác, biên dịch và chạy luôn trên hệ điều hành đó.

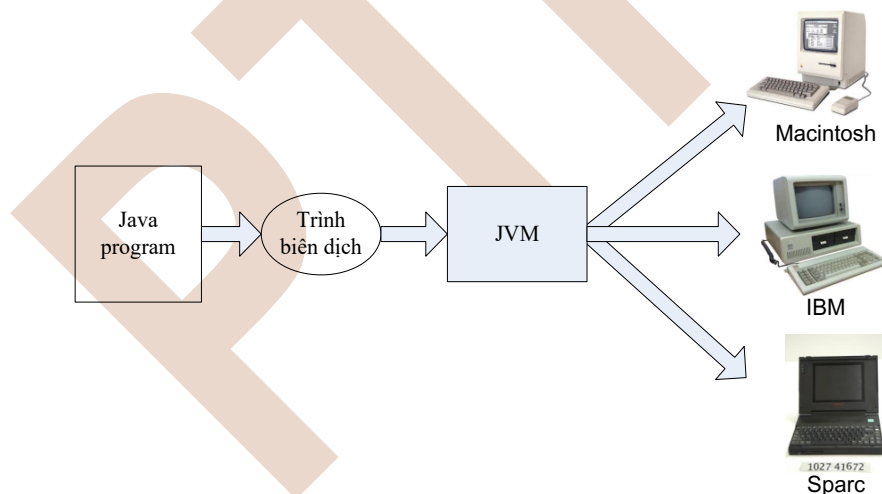
Độc lập ở mức nhị phân có nghĩa là có thể chạy một file đã được biên dịch trên nhiều hệ điều hành khác nhau mà không cần phải biên dịch lại.

Đối với các ngôn ngữ trước đây như C và C++, trình biên dịch làm nhiệm vụ chuyển các lệnh của chương trình thành các lệnh của vi xử lý trong máy tính hay còn gọi là mã máy. Các mã máy này thay đổi tùy theo từng bộ vi xử lý. Do vậy, nếu một chương trình muốn chạy trên một nền tảng khác thì cần phải biên dịch lại chương trình cho phù hợp với nền tảng đó. Hình 1.1 mô tả việc biên dịch một chương trình C++ để chạy trên các nền tảng khác nhau.



Hình 1. 1. Biên dịch chương trình theo cách truyền thống

Đối với Java, môi trường phát triển được chia thành hai phần : Trình biên dịch Java và trình thông dịch. Trình biên dịch có nhiệm vụ dịch mã nguồn sang mã máy, nhưng đây là mã máy dành cho máy tính trung gian của Java gọi là Máy ảo Java (Java Virtual Machine – JVM). Ngôn ngữ dành cho máy ảo này được gọi là Java bytecode. Máy ảo Java có chức năng thông dịch từ bytecode sang mã máy tương ứng với nền tảng máy tính thực tế đang sử dụng để chạy chương trình Java. Hình 2.2 mô tả quá trình biên dịch và thông dịch ngôn ngữ Java sang ngôn ngữ máy tính.



Hình 1. 2 Biên dịch và chương trình Java

Java là một ngôn ngữ lập trình vừa biên dịch vừa thông dịch. Chương trình nguồn viết bằng ngôn ngữ lập trình Java có đuôi *.java đầu tiên được biên dịch thành tập tin có đuôi *.class và sau đó sẽ được trình thông dịch thông dịch thành mã máy. Chính vì có máy ảo trung gian làm nhiệm vụ thông dịch từ bytecode sang mã máy tính nên Java được coi là độc lập với nền tảng.

Đa nhiệm - đa luồng (MultiTasking - Multithreading):

Java hỗ trợ lập trình đa nhiệm, đa luồng cho phép nhiều tiến trình, tiểu trình có thể chạy song song cùng một thời điểm và tương tác với nhau.

Chắc chắn (Robust):

Java là ngôn ngữ có tính khắt khe. Vì vậy, nó yêu cầu phải khai báo các phương thức một cách tường minh. Java kiểm tra mã nguồn tại thời điểm biên dịch và thông dịch. Do vậy, nó loại bỏ hầu hết các lỗi tiềm tàng trước khi chạy.

Trong các ngôn ngữ truyền thống, lập trình viên phải quan tâm đến việc cấp phát và giải phóng bộ nhớ. Trong trường hợp lập trình viên quên giải phóng bộ nhớ sẽ nảy sinh vấn đề tràn bộ nhớ. Trong Java, lập trình viên không cần quan tâm đến các vấn đề này vì Java tự động xử lý vấn đề cấp phát và giải phóng bộ nhớ.

Phân tán (Distributed):

Một ứng dụng viết bằng Java có thể được thiết kế để chạy trên mạng máy tính bao gồm nhiều nền tảng, nhiều hệ điều hành và nhiều giao diện người dùng khác nhau. Do vậy, Java được sử dụng rộng rãi trong môi trường Internet.

1.3.3. Các công nghệ Java

Công nghệ Java phát triển mạnh mẽ nhờ vào việc hãng Sun Microsystem cung cấp nhiều công cụ, thư viện lập trình phong phú hỗ trợ cho việc phát triển nhiều loại hình ứng dụng khác nhau cụ thể như:

- J2SE (Java 2 Standard Edition) hỗ trợ phát triển những ứng dụng đơn, ứng dụng client-server.
- J2EE (Java 2 Enterprise Edition) hỗ trợ phát triển các ứng dụng thương mại.
- J2ME (Java 2 Micro Edition) hỗ trợ phát triển các ứng dụng trên các thiết bị di động, không dây, ...

1.3.4. Áp dụng Java trong thực tế

Java được sử dụng ở rất nhiều nơi trong thế giới thực, từ các trang web thương mại điện tử đến các ứng dụng Android, từ ứng dụng khoa học đến ứng dụng tài chính như các hệ thống giao dịch điện tử, từ các trò game như Minecraft cho đến các ứng dụng desktop như Eclipse, Netbeans và IntelliJ, từ thư viện mã nguồn mở tới các ứng dụng J2ME v.v... Một số ứng dụng điển hình của Java bao gồm các ứng dụng trên điện thoại di động trên nền tảng Android, các ứng dụng trên máy chủ, các ứng dụng trên Web,...

1.3.5. Môi trường lập trình Java

Môi trường lập trình Java thường bao gồm một số chương trình thực hiện các nhiệm vụ khác nhau để phục vụ công việc biên soạn, dịch và chạy một chương trình Java. Cụ thể một số bước cơ bản để xây dựng và chạy một chương trình Java :

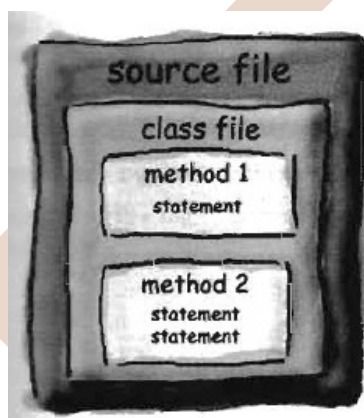
- Soạn thảo: Mã nguồn chương trình có thể được soạn thảo trên các phần mềm soạn thảo văn bản như Notepad, emacs và được tên file với phần mở rộng là .java.

- Biên dịch : Chương trình biên dịch javac của Java biên dịch file mã nguồn chương trình thành các lệnh để máy ảo Java có thể hiểu được. File này có phần mở rộng là .class
- Chạy chương trình : Chương trình java sẽ dùng máy ảo Java để chạy file .class.

Để thuận tiện trong việc soạn thảo, biên dịch và chạy chương trình, hiện nay người ta dùng các chương trình tích hợp các tác vụ như trên và gọi chung là môi trường lập trình tích hợp (IDE- Intergrated Development Environment). Một số IDE phổ biến hiện nay là Eclipse, NetBean, JBuilder.

1.3.6. Cấu trúc chương trình Java

Cấu trúc tổng quát của một chương trình Java được mô tả trong hình 1.3 :



Hình 1.3 Cấu trúc chương trình Java

File mã nguồn (với phần mở rộng là .java) gồm có định nghĩa một lớp (class). Một chương trình Java có thể chứa một hoặc nhiều lớp. Lớp phải được khai báo kèm theo một cặp dấu ngoặc ({ }) như mô tả trong hình 1.4.

```

public class Dog {

}
class
  
```

Hình 1.4 Cách khai báo một lớp

Mỗi lớp có một hoặc nhiều phương thức (method). Trong lớp *Student*, phương thức *subjectRegister* sẽ chứa các lệnh cho phép sinh viên đăng ký môn học. Các phương thức phải được khai báo bên trong một lớp (tức là bên trong hai dấu ngoặc { } của lớp). Cách khai báo phương thức được mô tả trong hình 1.5.

```
public class Dog {
    void bark() {

    }
}
```

method

Hình 1. 5 Cách khai báo một phương thức

Giữa hai dấu ngoặc {} của phương thức là các lệnh của phương thức đó. Như vậy, mã nguồn của một phương thức là một tập các lệnh để máy tính theo đó thực hiện và hoàn thành chức năng của phương thức đó.

```
public class Dog {
    void bark() {
        statement1;
        statement2;
    }
}
```

statements

Hình 1. 6 Các lệnh trong một phương thức

Phân tích cấu trúc của một lớp :

Khi JVM bắt đầu chạy, nó sẽ tìm tới lớp có tên được chỉ ra tại dòng lệnh. Sau đó, nó tìm tới phương thức đặc biệt được gọi là phương thức *main*. Phương thức này được khai báo như sau :

```
public class Baitap {
    public static void main(String[] args) {
        // TODO code application logic here
    }
}
```

Tiếp theo, JVM sẽ chạy các lệnh giữa hai dấu ngoặc {} của phương thức *main*. Tất cả các ứng dụng Java phải có ít nhất một lớp và ít nhất một phương thức *main*.

1.3.7. Chương trình Java đầu tiên :

Chúng ta sẽ xem xét chương trình dưới đây :

```

1 package first;
2 //Chương trình Java đơn giản
3 /*Comment
4  | nhiều dòng */
5 public class First {
6     | public static void main(String[] args) {
7     |     System.out.println("Chương trình Java đầu tiên");
8     | }
9 }

```

Tên file đóng vai trò rất quan trọng trong Java. Vì mã nguồn chương trình nằm trong một lớp nên lớp và file phải cùng tên với nhau. Ngôn ngữ Java phân biệt chữ hoa và chữ thường nên tên file phải trùng với tên của lớp. Ví dụ, 'First' và 'first' được coi là hai tên khác nhau.

Để biên dịch chương trình, chúng ta có thể sử dụng trình biên dịch javac. Giả sử JDK được cài trong thư mục Program Files. Ta sử dụng lệnh như sau :

C:\Program Files\Java\jdk1.8.0_144\bin>javac First.java

Trình biên dịch sẽ tạo ra file First.class chứa chương trình dạng bytecode. Tuy nhiên, chương trình này chưa thể chạy trực tiếp trên các nền tảng máy tính mà phải sử dụng thêm trình thông dịch java để thực thi chương trình bằng lệnh sau :

C:\Program Files\Java\jdk1.8.0_144\bin>java First

Chương trình sẽ có kết quả như sau :

Chương trình Java đầu tiên

Sau đây chúng ta sẽ phân tích các lệnh trong chương trình First :

- Dòng 1: Khai báo tên gói chứa chương trình First. Gói là một thư mục chứa các lớp có cùng mối liên hệ nào đó. Một chương trình Java có thể có nhiều gói.
- Dòng 2: Ký hiệu // được sử dụng để báo cho Java biết đây là dòng comment. Chương trình sẽ bỏ qua dòng này khi biên dịch. Trong trường hợp có nhiều dòng comment liên nhau, chúng ta có thể sử dụng ký hiệu /* để bắt đầu khối và ký hiệu */ ở cuối khối như mô tả ở dòng 3 và 4.
- Dòng 5 : Từ khóa 'public' dùng để khai báo phạm vi truy cập của lớp. Với từ khóa này, lớp có thể được sử dụng ở bên ngoài phạm vi của gói. Từ khóa 'class' khai báo định nghĩa lớp. 'First' là tên của lớp.
- Dòng 6 :
 - Từ khóa 'public' dùng để khai báo phạm vi truy cập của phương thức. Với từ khóa này, phương thức có thể được truy cập từ bên ngoài lớp.

- Từ khóa 'static' cho phép phương thức main được gọi mà không cần phải tạo một đối tượng cụ thể của lớp First. Từ khóa này sẽ được đề cập kỹ hơn trong phần sau của bài giảng.
 - Từ khóa 'void' báo cho trình biên dịch biết là phương thức main sẽ không trả về bất cứ giá trị nào khi chạy chương trình.
 - Phương thức 'main()' là phương thức đặc biệt của Java. Nó được gọi đầu tiên khi chương trình chạy.
 - Tham số 'String args[]' được truyền vào cho phương thức main. Phương thức bao gồm tên phương thức và dấu ngoặc đơn. Trong dấu ngoặc đơn chứa các tham số cần được truyền vào cho phương thức đó. Trong trường hợp này, args là một mảng có kiểu là String. Như vậy, tham số truyền vào cho phương thức main phải là một chuỗi các ký tự.
- Dòng 7 : Đây là câu lệnh dùng để hiển thị một chuỗi ký tự lên màn hình. Phương thức println() có tham số truyền vào là chuỗi 'Chương trình Java đầu tiên'. 'System' là tên của một lớp có sẵn trong Java cho phép truy cập tới hệ thống và 'out' là một luồng ra (output stream) kết nối với màn hình được tạo sẵn của hệ thống.

1.4. Các kiểu dữ liệu trong Java

Dữ liệu bao gồm đầu vào, đầu ra, và các kết quả của các phép toán trung gian trong chương trình. Trong Java, dữ liệu được chia thành hai loại : dữ liệu cơ bản (Primitive data) và dữ liệu tham chiếu (Reference data). Dữ liệu cơ bản dành cho các giá trị cơ bản như các số hay các ký tự. Ví dụ như các kiểu int (số nguyên), float (số thực), char (ký tự),... Còn dữ liệu tham chiếu dành cho các dữ liệu là các tham chiếu đến đối tượng. Ví dụ dữ liệu kiểu đối tượng Student, Person,...

Java cung cấp 8 kiểu dữ liệu cơ bản :

Loại dữ liệu	Kích thước (bit)	Khoảng giá trị	Mô tả
byte	8	-128 đến 127	Kiểu số nguyên có độ dài 1 byte
char	16	0 đến 65.535	Kiểu ký tự đơn trong bảng mã Unicode
boolean	1	True hoặc False	Kiểu logic, mang giá trị đúng hoặc sai
short	16	-32768 đến 32767	Kiểu số nguyên có độ dài 2 byte

int	32	-2.147.483.648 đến 2.147.483.648	Kiểu số nguyên có độ dài 4 byte
long	64	-9.223.372.036.854.775.808 đến 9.223.372.036.854.775.807	Kiểu số nguyên có độ dài 8 byte
float	32	-3.34028E+38 tới +3.34028E+38	Số thực có độ dài 4 byte
double	64	-1.7976E+308 tới +1.7976 ^E +308	Số thực có độ dài 8 byte

Bảng 1. 1 Các kiểu dữ liệu cơ bản của Java

Java cung cấp 3 loại dữ liệu tham chiếu :

Kiểu dữ liệu	Mô tả
Array	Kiểu mảng : là một tập các biến có cùng kiểu. Ví dụ mảng các biến kiểu nguyên, kiểu Student,...
Class	Kiểu lớp : là một tập các biến và phương thức của một lớp nào đó. Ví dụ, lớp Student gồm các biến chứa tên tuổi, địa chỉ,... và các phương thức như học, chơi,... của đối tượng sinh viên.
Interface	Kiểu lớp giao diện : Giống như kiểu lớp nhưng đây là lớp trừu tượng.

Bảng 1. 2 Các kiểu dữ liệu tham chiếu

1.5. Các phép toán cơ bản trong Java

1.5.1. Biến

Trong chương trình, biến là tên của một vùng nhớ được dùng để lưu dữ liệu khi chạy chương trình. Dữ liệu lưu trong một biến được gọi là giá trị của biến đó. Để sử dụng biến trong chương trình, chúng ta phải khai báo biến đó trước khi sử dụng.

Định danh (identifier) là thuật ngữ chỉ tên (tên biến, tên hàm, tên lớp,...). Java quy định định danh là một chuỗi ký tự viết liên nhau bao gồm các chữ cái a...z, A...Z, chữ số 0...9, dấu gạch chân '_'. Định danh không được bắt đầu bằng chữ số và không được trùng với các từ khóa (key word). Từ khóa là từ mang ý nghĩa đặc biệt của ngôn ngữ lập trình, ví dụ như public, private, static, void, int,... Lưu ý Java phân biệt chữ cái hoa và chữ cái thường.

Ví dụ :

```
int a,b; // khai báo 2 biến sẽ chứa dữ liệu kiểu nguyên
```

Các biến được khai báo trong một hàm được gọi là biến địa phương và biến này chỉ có hiệu lực trong phạm vi của hàm đó. Một biến địa phương trước khi sử dụng trong biểu thức phải được gán một giá trị khởi tạo.

Phép gán (=) là cách gán giá trị cho một biến. Java cho phép gán giá trị cho nhiều biến đồng thời. Lệnh gán trong Java :

Ví dụ : gán giá trị cho một biến có kiểu là int:

```
int num = 100;  
int a,b,c;  
a = b = c = 200;
```

1.5.2. Phép toán số học

Trong Java, các toán hạng (operand) của các toán tử (operator) phải có kiểu số hoặc kiểu ký tự. Các toán hạng kiểu boolean không được phép sử dụng trong các phép toán. Một số phép toán số học của Java bao gồm :

Phép toán	Mô tả
+	Cộng : trả về giá trị là tổng 2 toán hạng. Ví dụ : 3 + 4 trả về giá trị 7
-	Trừ : trả về giá trị là hiệu của hai toán hạng hoặc giá trị âm của một toán hạng. Ví dụ : 4 - 3 trả về giá trị 1 hoặc -9 trả về giá trị âm của 9
*	Nhân : Trả về giá trị là tích của hai toán hạng. Ví dụ : 4*3 trả về giá trị 12.
/	Chia lấy phần nguyên: Trả về thương của phép chia hai toán hạng. Ví dụ : 9/4 trả về giá trị 2
%	Chia lấy phần dư : Trả về phần dư của phép chia hai toán hạng. Ví dụ : 9/4 trả về giá trị 1
++	Tăng : Tăng giá trị của toán hạng lên 1 đơn vị. Ví dụ : a++ có kết quả tương đương với phép toán a = a + 1
--	Giảm : Giảm giá trị của toán hạng xuống 1 đơn vị

	Ví dụ : $a--$ có kết quả tương đương với phép toán $a = a - 1$
$+=$, $-=$, $*=$, $/=$, $\%=$	Các phép toán cộng rồi gán, trừ rồi gán, nhân rồi gán, chia lấy phần nguyên rồi gán, chia lấy phần dư rồi gán. Ví dụ : $c+=a$ tương đương với phép toán $c = c + a$.

Bảng 1. 3 Một số phép toán số học của Java

1.5.3. Phép toán trên các bit

Các phép toán trên bit (bitwise) cho phép thao tác với các bit giá trị của dữ liệu cơ bản dựa trên đại số logic bao gồm các phép toán NOT, AND, OR, XOR. Cụ thể như sau :

Phép toán	Mô tả
$\sim$	NOT : trả về giá trị phủ định Ví dụ : $a = 1$ thì $\sim a = 0$ và ngược lại.
$\&$	AND : phép tính ‘và’ của 2 bit. Ví dụ : $1 \text{ AND } 0 = 0$, $0 \text{ AND } 1 = 0$, $0 \text{ AND } 0 = 0$, $1 \text{ AND } 1 = 1$
$ $	OR : phép tính ‘hoặc’ của 2 bit. Ví dụ : $1 \text{ OR } 1 = 1$, $0 \text{ OR } 0 = 0$, $1 \text{ OR } 0 = 1$, $0 \text{ OR } 1 = 1$
$\wedge$	XOR : phép tính ‘hoặc tuyệt đối’ của 2 bit Ví dụ : $1 \text{ XOR } 1 = 0$, $0 \text{ XOR } 0 = 0$, $1 \text{ XOR } 0 = 1$, $0 \text{ XOR } 1 = 1$
$>>$	Phép dịch phải : Dịch các bit của một số sang phải một vị trí. Toán hạng bên trái là số bị dịch, toán hạng bên phải là số lần dịch. Ví dụ : $x = 28$ (nhị phân : 0001 1100) thì phép toán $x >> 2$ trả về giá trị là 7 (nhị phân : 0000 0111)
$<<$	Phép dịch trái : Dịch các bit của một số sang trái một vị trí.

Bảng 1. 4 Một số phép toán trên bit

1.5.4. Phép toán chỉ mỗi quan hệ

Phép toán	Mô tả
$==$	Bằng : Dùng để kiểm tra xem hai số có bằng nhau hay không.

	Ví dụ : câu lệnh <i>if (a==b) return true</i> ; trả về giá trị true nếu a bằng b.
!=	Khác : Dùng để kiểm tra xem hai số có khác nhau hay không Ví dụ : câu lệnh <i>if (a !=b) return true</i> ; trả về giá trị true nếu a không bằng b.
>	Lớn hơn : Dùng để kiểm tra xem số thứ bên trái có lớn hơn số bên phải hay không. Ví dụ : câu lệnh <i>if (a >b) return true</i> ; trả về giá trị true nếu a lớn hơn b.
<	Nhỏ hơn : Dùng để kiểm tra xem số thứ bên trái có nhỏ hơn số bên phải hay không. Ví dụ : câu lệnh <i>if (a <b) return true</i> ; trả về giá trị true nếu a nhỏ hơn b.
<=	Nhỏ hơn hoặc bằng : Dùng để kiểm tra xem số thứ bên trái có nhỏ hơn hoặc bằng số bên phải hay không. Ví dụ : câu lệnh <i>if (a <=b) return true</i> ; trả về giá trị true nếu a nhỏ hơn b.
>=	Lớn hơn hoặc bằng : Dùng để kiểm tra xem số thứ bên trái có lớn hơn hoặc bằng số bên phải hay không. Ví dụ : câu lệnh <i>if (a >=b) return true</i> ; trả về giá trị true nếu a nhỏ hơn b.

Bảng 1. 5 Phép toán chỉ mối quan hệ

1.5.5. Phép toán logic

Phép toán	Mô tả
&&	AND : Trả về giá trị TRUE nếu cả hai giá trị đều là TRUE. Ví dụ : <i>if ((a>b) && (c>d)) return 10</i> : trả về giá trị bằng 10 nếu a>b và c>d
	OR : Trả về giá trị TRUE nếu một trong hai hoặc cả hai giá trị đều là TRUE. Ví dụ : <i>if(a>b) (c >d) return 10</i> : trả về giá trị bằng 10 nếu a>b hoặc c>d hoặc đồng thời a >b và c>d

^^	XOR : Trả về giá trị TRUE nếu chỉ một trong hai giá trị là TRUE Ví dụ : : if(a>b) (c >d) return 10 : trả về giá trị bằng 10 nếu chỉ a>b hoặc chỉ c>d.
!	Phủ định : Chuyển giá trị từ TRUE sang FALSE hoặc ngược lại. Ví dụ if (!(a>b)) return 10 : trả về giá trị 10 nếu a nhỏ hơn hoặc bằng b.

Bảng 1. 6 Phép toán logic

1.5.6. Độ ưu tiên của các phép toán

Mức độ ưu tiên của các phép toán theo thứ tự như sau : Mức độ ưu tiên cao nhất là các toán tử đơn (+, -, !, ++, --), sau đến là các toán tử đôi (*, /, %, +, -) và cuối cùng là các phép so sánh (<, >, <=, >=).

Ví dụ : b=3, biểu thức a = ++b * -4 - 5%3 sẽ trả về kết quả là -18.

Ngoài ra, để xác định rõ thứ tự ưu tiên của các phép tính trong biểu thức, ta có thể dùng ngoặc đơn (). Nếu có nhiều ngoặc lồng nhau thì biểu thức của ngoặc trong cùng sẽ được tính trước.

Ví dụ : b=3, biểu thức a = (++b * (-4) - 5)%3 sẽ trả về kết quả là 0.

1.6. Cấu trúc lệnh trong Java

1.6.1. Lệnh in dữ liệu ra màn hình

System.out.println : Hiển thị text trên màn hình

Ví dụ :

```
System.out.println("Hoc vien CN BCVT - Khoa DPT");
```

Kết quả :

```
Hoc vien CN BCVT - Khoa DPT
```

Để hiển thị nhiều dòng text hoặc các ký tự đặc biệt, ta có thể dùng một số ký hiệu sau :

Ký hiệu	Ví dụ	Ý nghĩa
\n	<pre>System.out.println("Hoc vien CN BCVT \nKhoa DPT");</pre> Kết quả:	Xuống dòng từ vị trí có ký hiệu.

	Hoc vien CN BCVT Khoa DPT	
\r	<pre>System.out.println("Hoc vien CN BCVT \rKhoa DPT");</pre> Kết quả: Hoc vien CN BCVT Khoa DPT	Xuống dòng từ vị trí có ký hiệu. Tuy nhiên, những ký tự trước đó sẽ bị ghi đè.
\t	<pre>System.out.println("Hoc vien CN BCVT \tKhoa DPT");</pre> Kết quả: Hoc vien CN BCVT Khoa DPT	Các ký tự sau ký hiệu được lùi một tab.
\\	<pre>System.out.println("Hoc vien CN BCVT \\Khoa DPT");</pre> Kết quả: Hoc vien CN BCVT \Khoa DPT	Hiển thị dấu \
\'	<pre>System.out.println("Hoc vien CN BCVT \'Khoa DPT");</pre> Kết quả: Hoc vien CN BCVT 'Khoa DPT	Hiển thị dấu ngoặc đơn ‘
\“	<pre>System.out.println("Hoc vien CN BCVT \"Khoa DPT\");</pre> Kết quả: Hoc vien CN BCVT "Khoa DPT"	Hiển thị dấu ngoặc kép “

Bảng 1. 7 Ký hiệu định dạng chuỗi ký tự

System.out.printf(format, arguments): Hiển thị dữ liệu (arguments) với định dạng được chỉ ra bởi format. Các định dạng bao gồm :

- s – Định dạng cho chuỗi ký tự
- d – Định dạng cho số nguyên
- f – Định dạng cho số thực
- t – Định dạng cho dữ liệu thời gian

Ví dụ :

```

public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    System.out.println("Nhap tu so:");
    int n = s.nextInt();
    System.out.println("Nhap mau so:");
    int d = s.nextInt();
    float r = (float)n/d;
    System.out.printf("%.2f", r);
}

```

1.6.2. Lệnh nhập dữ liệu từ bàn phím

Để nhập một số hoặc một chuỗi ký tự từ bàn phím, chúng ta sử dụng lớp Scanner có sẵn trong thư viện của Java. Để sử dụng gói Scanner, ta phải sử dụng lệnh *import java.util.Scanner*. Sau đó ta phải tạo ra một đối tượng cụ thể của lớp Scanner bằng cách sử dụng lệnh *Scanner s = new Scanner(System.in)*.

Lớp Scanner của Java có sẵn một số phương thức cho phép người dùng nhập vào một số, một chuỗi ký bao gồm *next()*, *nextLine()*, *nextInt()*, *nextByte()*, *nextShort()*, *nextLong()*, *nextFloat()*, *nextDouble()*.

Ví dụ :

```

1 package first;
2 import java.util.Scanner;
3 public class First {
4     public static void main(String[] args) {
5         int num1,num2;
6         Scanner s = new Scanner(System.in);
7         System.out.println("Nhap so thu nhat: ");
8         num1 = s.nextInt();
9         System.out.println("Nhap so thu hai: ");
10        num2 = s.nextInt();
11        System.out.println("Tong hai so la: " + (num1 + num2));
12    }
13 }

```

Kết quả :

```

Nhap so thu nhat:
4
Nhap so thu hai:
5
Tong hai so la: 9

```

1.6.3. Các lệnh điều khiển trong Java

Lệnh if – else

Lệnh if – else cho phép rẽ nhánh bằng cách lựa chọn thực hiện một trong hai hành động. Cú pháp của lệnh :

```
if(điều kiện) {  
    Hành động 1 ;  
}  
else{  
    Hành động 2 ;  
}
```

Ví dụ :

```
package first;  
import java.util.Scanner;  
public class First {  
    public static void main(String[] args) {  
        int num1,num2;  
        Scanner s = new Scanner(System.in);  
        System.out.println("Nhap so thu nhat: ");  
        num1 = s.nextInt();  
        System.out.println("Nhap so thu hai: ");  
        num2 = s.nextInt();  
        System.out.println("Tong hai so la: " + (num1 + num2));  
        if(num1 > num2){  
            System.out.println("Số thứ nhất lớn hơn số thứ hai");  
        }  
        else{  
            System.out.println("Số thứ nhất bé hơn số thứ hai");  
        }  
    }  
}
```

Kết quả :

```
Nhap so thu nhat:  
4  
Nhap so thu hai:  
5  
Tong hai so la: 9  
Số thứ nhất bé hơn số thứ hai
```

Lệnh switch – case

Lệnh switch – case được sử dụng để thay thế cho lệnh if – else trong trường hợp kết quả của một biểu thức có thể nhận một trong nhiều giá trị khác nhau. Cú pháp của lệnh như sau :

```
switch (biểu thức) {  
    case 'giá trị 1' :  
        công việc 1 ;  
        break ;  
    case 'giá trị 2' :  
        công việc 2 ;  
        break ;  
    -----  
    case 'giá trị N' :  
        công việc N ;  
        break ;  
    default :  
        công việc mặc định ;  
}
```

Ví dụ :

```
package first;  
import java.util.Scanner;  
public class First {  
    public static void main(String[] args) {  
        int day;  
        Scanner s = new Scanner(System.in);  
        System.out.println("Nhập ngày trong tuần: ");  
        day = s.nextInt();  
        switch(day) {  
            case 0: System.out.println("Chủ nhật"); break;  
            case 1: System.out.println("Thứ 2"); break;  
            case 2: System.out.println("Thứ 3"); break;  
            case 3: System.out.println("Thứ 4"); break;  
            case 4: System.out.println("Thứ 5"); break;  
            case 5: System.out.println("Thứ 6"); break;  
            case 6: System.out.println("Thứ 7"); break;  
            default: System.out.println("Ngày không hợp lệ");  
        }  
    }  
}
```

Kết quả :

```
Nhập ngày trong tuần:  
5  
Thứ 6
```

Lệnh lặp while :

Vòng lặp while thực hiện lặp đi lặp lại các công việc trong thân vòng lặp nếu như điều kiện lặp vẫn còn thỏa mãn. Cú pháp của lệnh như sau :

```
while (điều kiện) {  
    thực hiện công việc ;  
}
```

Ví dụ :



```
package first;  
import java.util.Scanner;  
public class First {  
    public static void main(String[] args) {  
        int num,giaithua=1;  
        Scanner s = new Scanner(System.in);  
        System.out.println("Nhập số cần tính giai thừa: ");  
        num = s.nextInt();  
        System.out.print("Giai thua cua " + num + " là: ");  
        while(num>=1){  
            giaithua*=num;  
            num--;  
        }  
        System.out.println(giaithua);  
    }  
}
```

Kết quả :



```
Nhập số cần tính giai thừa:  
5  
Giai thua cua 5 là: 120
```

Lệnh lặp do – while :

Vòng lặp do – while thực hiện các công việc trong vòng lặp cho đến khi điều kiện được thỏa mãn. Vòng lặp này tương tự như vòng lặp while ngoại trừ là vòng lặp do – while luôn thực hiện công việc ít nhất một lần cho dù điều kiện đúng hay sai. Cú pháp của lệnh :

```
do {
```

công việc ;

}

while (điều kiện) ;

Ví dụ :

```
package first;
public class First {
    public static void main(String[] args) {
        int num =1, sum = 0;
        do {
            sum += num;
            num++;
        }
        while (num<=100);
        System.out.println("Tổng từ 1 đến 100 là: " + sum);
    }
}
```

Kết quả :

Tổng từ 1 đến 100 là: 5050

Lệnh lặp for :

Vòng lặp for cho phép thực hiện công việc với số lần lặp được kiểm soát bởi một biến đếm. Cú pháp của vòng lặp như sau :

```
for (giá trị khởi tạo; điều kiện lặp; cập nhật){
    thực hiện công việc ;
}
```

Ví dụ :

```
public class First {
    public static void main(String[] args) {
        int sum = 0;
        for(int i=1;i<=100;i++){
            sum += i;
        }
        System.out.println("Tổng từ 1 đến 100 là: " + sum);
    }
}
```

Kết quả :

Tổng từ 1 đến 100 là: 5050

BÀI TẬP CHƯƠNG 1

Bài 1: Viết một giao diện Game đưa ra thông báo có các tùy chọn để người chơi chọn lựa bằng cách nhập chữ cái đầu. Sau đó hiển thị chữ cái đầu của người chơi đã nhập.

GAME OPTIONS

(A)dd a new player

(L)oad a saved game

(S)ave game

(Q)uit game

Bài 2: Viết chương trình để người dùng nhập hai số. Chương trình tính tổng hai số và kết quả được gán vào biến thứ nhất. Chương trình có sử dụng hàm `calSum` để tính tổng.

Bài 3: Cho phép người dùng nhập tuổi để tham gia bầu cử. Nếu tuổi < 18 : hiển thị thông báo “Không hợp lệ”. Nếu tuổi ≥ 18 hiển thị thông báo “Hợp lệ”. Chương trình dùng phương thức `ageCheck` để kiểm tra tuổi.

Bài 4: Viết chương trình hỏi người chơi có tiếp tục chơi hay thoát khỏi chương trình. Nếu người chơi chọn “q” hoặc “Q” thì thoát.

Bài 5: Viết chương trình nhập vào số phần tử của mảng, sau đó khởi tạo mảng có số phần tử do người dùng nhập vào. In số phần tử của mảng ra màn hình (`firstArray.length`).

CHƯƠNG 2. LỚP VÀ ĐỐI TƯỢNG TRONG JAVA

2.1. Cơ bản về lớp

2.1.1. Khai báo lớp

Trong Java, **lớp (class)** được coi là một khuôn mẫu của một nhóm đối tượng có cùng tính chất, đặc điểm và hành động. Việc định nghĩa một lớp thực chất là định nghĩa ra một kiểu dữ liệu bên cạnh các kiểu dữ liệu cơ bản như kiểu int, float,... Kiểu dữ liệu mới này có thể được sử dụng để định nghĩa ra các biến và các biến này được gọi là các **đối tượng (object)**. Nói cách khác, đối tượng là một thực thể cụ thể (instance) của lớp. Tất cả các đối tượng của lớp đều có cùng **đặc tính** và **hành động**.

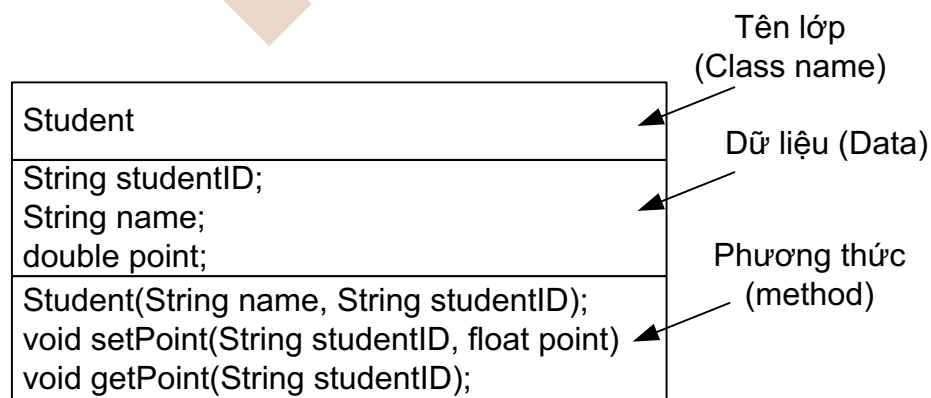
Ví dụ, trong bài toán quản lý nhân sự của một trường đại học chúng ta thấy có thể các nhân sự bao gồm giảng viên, sinh viên,... Các cá nhân trong mỗi nhóm nhân sự lại có những đặc điểm, hành động giống nhau. Vì vậy ta có thể định nghĩa các lớp đối tượng là Giảng viên, Sinh viên. Các cá nhân trong lớp Giảng viên có cùng đặc tính là có họ tên, trình độ, số năm giảng dạy và có các hoạt động chung là giảng dạy, nghiên cứu. Lớp Sinh viên sẽ có những đặc điểm là họ tên, số hiệu sinh viên, điểm thi,... và có các hoạt động chung là tạo sinh viên, nhập điểm, xem điểm thi,...

Cấu trúc của một lớp gồm 3 phần : tên lớp, dữ liệu của lớp và phương thức của lớp. Dữ liệu của lớp mô tả các đặc tính của các đối tượng trong khi phương thức mô tả các hành động mà các đối tượng của lớp có thể thực hiện.

Cú pháp khai báo lớp như sau :

```
class name {  
    kiểu_dữ_liệu tên biến ;  
    kiểu_phương_thức tên phương thức (tham số) ;  
}
```

Ví dụ với lớp Sinh viên đề cập ở trên có cấu trúc như sau :



Hình 2. 1 Cấu trúc của lớp

Khai báo cho lớp Sinh viên như sau :

```
public class Student {
    String studentID;
    String name;
    double point;
    Student(String n, String sID){
        name = n;
        studentID = sID;
    }
    void setPoint(String sID, double pnt){
        point = pnt;
    }
    void getPoint(String sID){
        System.out.println("Điểm của sinh viên " + this.name + " là: " + point);
    }
}
```

Vậy làm thế nào để kiểm tra hoạt động của lớp *Student*? Để làm việc này, ngoài lớp *Student* như đề cập ở trên ta cần thêm một lớp chứa hàm *main* để tạo ra các đối tượng có kiểu *Student*. Lớp này ta đặt tên là *StudentManage*, dùng để tạo và quản lý các đối tượng *Student*.

```
public class StudentManage {
    public static void main(String[] args) {
        Student s1 = new Student("Nguyen Van A", "PTIT1234");
        s1.setPoint("PTIT1234", 8.5);
        s1.getPoint("PTIT1234");
    }
}
```

2.1.2. Mô tả lớp bằng UML

Một lớp bao gồm:

- Một tên lớp duy nhất (thường bắt đầu bằng chữ hoa).
- Một danh sách các thuộc tính (int, double, boolean, String,...)
- Một danh sách các phương thức.

Ví dụ cách mô tả một lớp:

ClassName
attributes
methods()

- Attribute. Ví dụ: Name, Address,...
- Operations (hành vi/thao tác). Ví dụ: Running, Playing, Shrink,...
- Visibility (phạm vi truy cập).
 - Public: Mọi lớp khác trong hệ thống đều có thể nhìn thấy.
 - Private hay Protected: Chỉ có thể nhìn thấy bên trong lớp hoặc từ lớp friend.

- Package hay Implementation: Chỉ các lớp trong cùng gói mới nhìn thấy.

Ví dụ mô tả lớp Book:

Book
- title: String - author: String
+ setTitle() + getTitle() + setAuthor() + getAuthor() + toString()

2.1.2.1. Mô tả thuộc tính có cú pháp như sau:

visibility name: type multiplicity

visibility (khả năng truy cập/nhìn thấy đối tượng tại một vị trí nào đó trong chương trình). Phạm vi truy cập (modifier) của các thuộc tính và phương thức được biểu diễn bởi dấu + (với phạm vi public) hoặc dấu – (với phạm vi private). Với các thuộc tính thông thường phạm vi truy cập mặc định là private, với các phương thức thì phạm vi truy cập mặc định là public. Phạm vi truy cập có thể là:

- ‘+’ public
- ‘-’ private
- ‘#’ protected
- ‘~’ default

type: là các loại dữ liệu cơ bản trong Java.

multiplicity có thể là:

- ‘n’ mô tả chính xác một giá trị n nào đó
- ‘*’ mô tả giá trị 0 hoặc lớn hơn 0
- ‘m..n’ mô tả giá trị nằm giữa m và n.

Multiplicity	Ý nghĩa
n (mặc định)	Nhiều
0..0	Không
0..1	Không hoặc 1
0..n	Không hoặc nhiều
1..1	Chính xác 1
1..n	Một hoặc nhiều

Ví dụ một số cách mô tả thuộc tính:

- custRef: int [1]

Thuộc tính custRef là có kiểu truy cập là private và là một số nguyên.

#itemCode: String[1..*]

Thuộc tính itemCode có kiểu truy cập là protected và là một hoặc nhiều chuỗi ký tự.

validCard: boolean

Thuộc tính validCard không chỉ ra kiểu truy cập và cũng không chỉ ra tính nhiều.

2.1.2.2. Mô tả hành vi (operation) có cú pháp như sau:

Visibility name (par1: type1, par2: type2): returntype

Ví dụ:

+addName (newName: String): boolean

Khai báo trên mô tả phương thức “addName” có phạm vi truy cập là public, kiểu của tham số newName là String (chuỗi ký tự) và phương thức trả về giá trị kiểu boolean (True hoặc False).

2.1.3. Phương thức của lớp

Phương thức là một nhóm các lệnh chỉ ra cách thức thực hiện hoạt động nào đó của đối tượng. Ví dụ phương thức *setPoint* ở trên chỉ ra cách thức vào điểm của sinh viên bằng cách gán cho biến điểm *point* bằng với giá trị của tham số truyền vào là *pnt* của sinh viên có mã là *sID*.

Cú pháp của phương thức :

```
phạm_vi_truy_cập  thuộc_tính_của_phương_thức      kiểu_trả_về
tên_phương_thức  (danh_sách_tham_số) {
}
```

Ví dụ :

```
public final void setPoint(String sID, double pnt) {
    point = pnt;
}
```

Trong cú pháp trên :

- **phạm_vi_truy_cập (Access Specifier)** : mô tả phạm vi cho phép truy cập tới phương thức. Java cung cấp một số phạm vi như sau :
 - o **public** : Có thể truy cập phương thức từ bên ngoài gói hoặc bên ngoài lớp chứa phương thức đó.

- **default** : Chỉ có thể truy cập phương thức từ các lớp cùng một gói với lớp chứa phương thức.
- **protected** : Chỉ có thể truy cập phương thức trong lớp khai báo hoặc lớp con của lớp chứa phương thức, kể cả lớp con thuộc một gói khác.
- **private** : Chỉ có thể truy cập phương thức ngay trong chính lớp chứa phương thức.

Trong ví dụ trên, phương thức *setPoint* không có phạm vi truy cập thì JVM mặc định là public. Như vậy ta có thể truy cập phương thức này của lớp *Student* từ lớp bên ngoài là lớp *StudentManage* bằng câu lệnh *s1.setPoint('PTIT1234',8.5)*.

Khả năng truy cập	Public	Protected	Default	Private
Cùng lớp	Có	Có	Có	Có
Khác lớp, cùng gói	Có	Có	Có	Không
Khác lớp, không cùng gói	Có	Không	Không	Không
Lớp con, cùng gói	Có	Có	Có	Không
Lớp con, khác gói	Có	Có	Không	Không

Bảng 2. 1 Các từ khóa mô tả phạm vi truy cập

- **Thuộc tính của phương thức (modifier)** : Mô tả tính chất và cách thức sử dụng phương thức. Java cung cấp một số thuộc tính cho phương thức như sau :
 - **static** : Phương thức được dùng chung cho tất cả các đối tượng của lớp. Ngay cả khi chúng ta không cần tạo một đối tượng cụ thể của lớp, chúng ta vẫn dùng được phương thức này.
 - **abstract** : Phương thức chỉ có khai báo mà không có code bên trong. Code của phương thức sẽ được viết tại lớp con của lớp chứa phương thức abstract.
 - **final** : Phương thức có tiền tố final này sẽ không được khai báo chồng (overriding) ở lớp con.

- **synchronized** : Được sử dụng trong trường hợp lập trình đa luồng. Từ khóa này cho phép chỉ có một luồng được truy cập tới phương thức.
- **kiểu_trả_về** : Là kiểu dữ liệu cuối cùng được trả về cho phương thức đó. Trong trường hợp kiểu void có nghĩa là phương thức đó không trả về kiểu giá trị cụ thể nào. Trong trường hợp kiểu dữ liệu cụ thể (ví dụ *int*, *float*, *double*, ...) thì cuối phương thức phải có lệnh *return* kèm theo giá trị trả về cho phương thức. Trong trường hợp lệnh *return* không kèm theo giá trị nào thì nó mang ý nghĩa thoát khỏi phương thức.

Ví dụ :

```
package chapter2_vidu2.pkg2;
import java.util.Scanner;
public class Chapter2_vidu22 {
    public static void main(String[] args) {
        int num1, num2;
        Scanner s = new Scanner(System.in);
        System.out.println("Nhập số thứ nhất: ");
        num1 = s.nextInt();
        System.out.println("Nhập số thứ hai: ");
        num2 = s.nextInt();
        System.out.println("Giá trị nhỏ nhất là: " + minValue(num1, num2));
    }
    static int minValue(int num1, int num2){
        int min;
        if(num1 >= num2){
            min = num2;
        }
        else
            min = num1;
        return min;
    }
}
```

- **tên_phương_thức** : Được đặt theo các nguyên tắc giống với cách đặt tên biến.
- **danh_sách_tham_số** : Danh sách tham số của phương thức là các biến của phương thức và nó yêu cầu phải được truyền giá trị cụ thể cho các biến đó khi ta sử dụng phương thức. Các tham số này được khai báo ngăn cách bởi dấu phẩy. Trong ví dụ trên, phương thức *minValue* có hai tham số *num1*, *num2* có kiểu là *int*. Khi sử dụng phương thức ở hàm *main*, ta phải truyền cho hai tham số này các giá trị tương ứng của chúng do người dùng nhập vào.

2.1.4. Truyền tham số cho phương thức

Như đã đề cập ở khái niệm về lớp trong mục 2.1.1, lớp có chức năng mô tả những gì một đối tượng biết (thông qua thuộc tính) và những gì đối tượng có thể thực hiện (thông qua các phương thức). Mặc dù tất cả các thực thể của lớp có chung phương thức

nhưng phương thức của mỗi thực thể lại trả về những kết quả khác nhau vì nó còn phụ thuộc vào giá trị của các biến của thực thể đó.

Song
title artist
setTitle() setArtist() play()

Hình 2. 2 Lớp Song

Xét ví dụ trong hình 2.2. Lớp Song có hai biến thực thể là title và artist. Ngoài ra, lớp Song có phương thức play() có chức năng phát ra một bản nhạc nào đó tùy theo tên bài hát được truyền vào. Mã của phương thức play() được viết như sau :

```
void play(){  
    soundPlayer.playSound(title) ;  
}
```

Như ta thấy, phương thức play() có chức năng chơi một bản nhạc nhưng thực tế phương thức này phát bản nhạc nào thì nó phụ thuộc vào giá trị của biến title của thực thể đó.

Phương thức play() trong ví dụ trên không yêu cầu truyền trực tiếp giá trị vào trong phương thức nhưng có thể có những phương thức yêu cầu truyền giá trị vào trong phương thức. Ví dụ như phương thức *minValue (int num1, int num2)* ở ví dụ trên. Trong phương thức này, các biến *num1, num2* được gọi là các **tham số (parameter)** của phương thức. Trong khi các giá trị được truyền vào, ví dụ *minValue (4,5)*, gồm hai giá trị cụ thể 4 và 5 được gọi là các **đối số (argument)**.

Chúng ta cần lưu ý là nếu một phương thức yêu cầu một tham số thì ta phải truyền cho nó một giá trị nào đó và giá trị đó phải thuộc đúng kiểu được khai báo của tham số.

Phương thức có thể có nhiều tham số. Khi khai báo, ta dùng dấu phẩy để tách giữa chúng. Khi gọi hàm, ta phải truyền các đối số thuộc đúng kiểu dữ liệu và theo đúng thứ tự đã khai báo.

Phương thức có thể trả về giá trị. Khi phương thức khai báo sẽ trả về giá trị thuộc kiểu dữ liệu gì thì phải trả về giá trị thuộc kiểu dữ liệu đó. Ví dụ trong phương thức *minValue*, giá trị trả về là *min* thì *min* phải có cùng kiểu *int* với phương thức như đã khai báo.

2.1.5. Phương thức khởi tạo (Constructor)

Phương thức khởi tạo của một lớp là một phương thức đặc biệt so với các phương thức thông thường như đề cập ở trên. Phương thức này có tên giống với tên lớp và nó không có kiểu trả về. Phương thức khởi tạo thường được sử dụng để khởi tạo các giá trị ban đầu cho các biến thành viên của một lớp và phương thức này được gọi ngay sau khi một đối tượng được tạo ra. Phương thức khởi tạo có hai loại :

- Phương thức khởi tạo cụ thể : Phương thức khởi tạo này có các tham số cụ thể. Khi khởi tạo đối tượng, các giá trị cụ thể phải được truyền cho các tham số này. Trong ví dụ lớp *Student*, hàm khởi tạo của lớp được viết như sau :

```
Student(String n, String sID){  
    name = n;  
    studentID = sID;  
}
```

- Phương thức khởi tạo mặc định : Phương thức khởi tạo này không cần phải khai báo trong lớp. Khi khởi tạo một đối tượng của lớp, JVM sẽ mặc định tạo ra một đối tượng rỗng. Trong ví dụ trên, nếu ta không viết hàm khởi tạo thì trình biên dịch sẽ tự động tạo một hàm khởi tạo mặc định có dạng như sau :

```
Student() {  
}
```

Lưu ý rằng trong một lớp có thể có nhiều hàm khởi tạo với số lượng các tham số khác nhau. Khi khởi tạo, tùy theo số lượng tham số truyền vào mà JVM sẽ tự động gọi tới hàm khởi tạo tương ứng.

2.1.6. Khai báo và khởi tạo đối tượng

Như phần 2.1.1 đã đề cập, để sử dụng lớp *Student*, ta phải cần tới một lớp khác là lớp *StudentManage* để khai báo và tạo ra các đối tượng *Student*.

Để khai báo và khởi tạo một đối tượng cụ thể của *Student* là đối tượng *s1*, ta dùng lệnh như sau :

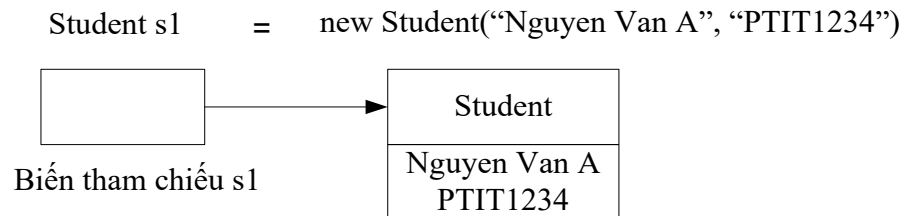
```
Student s1 = new Student("Nguyen Van A", "PTIT1234");
```

Dòng lệnh trên gồm ba bước : khai báo, tạo và gán đối tượng, tham chiếu đối tượng.

- Bước 1 : Lệnh *Student s1* có tác dụng khai báo một biến tham chiếu *s1* có kiểu là *Student*.
- Bước 2 : *new Student ("Nguyen Van A", "PTIT1234")* có tác dụng yêu cầu máy ảo Java cấp phát một ô nhớ chứa dữ liệu khởi tạo là *Nguyen Van A* và *PTIT1234*. Nói cách khác, với lệnh này, JVM sẽ gọi tới hàm khởi tạo của lớp

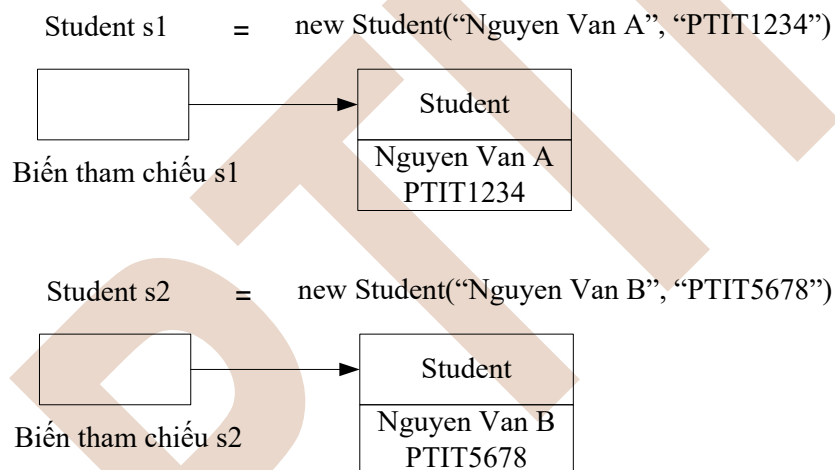
Student và lưu các dữ liệu được truyền cho các tham số của hàm khởi tạo vào trong ô nhớ vừa được cấp phát.

- Bước 3 : Phép gán (=) gắn biến tham chiếu *s1* với ô nhớ vừa được cấp phát. Nếu ta chỉ khai báo bằng lệnh *Student s1* mà không gán thì đồng nghĩa với việc JVM chỉ tạo ra biến *s1* và biến này sẽ không trỏ tới bất kỳ ô nhớ nào.

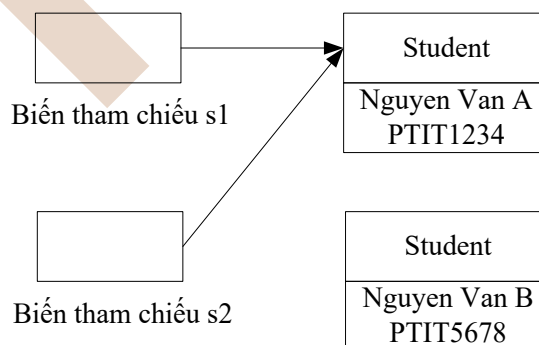


Hình 2. 3 Biến s1 tham chiếu tới đối tượng kiểu Student

Trong trường hợp ta khai báo thêm một đối tượng *s2* và gán *s1 = s2* thì *s2* và *s1* sẽ cùng tham chiếu tới ô nhớ của *s1*. Trong khi đó, ô nhớ của *s2* sẽ được JVM tự động giải phóng.



Sau khi gán *s1 = s2* :



Hình 2. 4 Gán hai biến tham chiếu

2.1.7. Lớp lồng nhau và lớp nội bộ

Trong trường hợp một lớp được khai báo bên trong một lớp khác thì ta gọi là lớp lồng nhau (Inner class). Chúng ta sử dụng inner class để nhóm các lớp và các interface một cách logic lại với nhau ở một nơi để giúp cho code dễ đọc và dễ bảo trì hơn.

Một số ưu điểm của lớp lồng nhau bao gồm :

- Inner class biểu diễn cho một kiểu đặc biệt của mối quan hệ đó là nó có thể truy cập tất cả các thành viên (các thành viên dữ liệu và các phương thức) của lớp ngoài bao gồm cả private.
- Inner class được sử dụng để giúp code dễ đọc hơn và dễ bảo trì bởi vì nó nhóm các lớp và các interface một cách logic vào trong một nơi.
- Code được tối ưu hóa: tiết kiệm code hơn.

Cú pháp :

```
class Java_Outer_class {  
    // code  
    class Java_Inner_class {  
        // code  
    }  
}
```

Có hai loại lớp lồng nhau :

- **Non-static class:** Một lớp non-static được tạo bên trong một lớp được gọi là lớp non-static lồng hay non-static class. Có ba kiểu non-static class:
 - o **Member Inner Class:** Một lớp được tạo ra bên trong một lớp và bên ngoài phương thức.

```
public class OuterClass {  
    int num = 10;  
    class InnerClass{  
        void printMsg() {  
            System.out.println("Giá trị của num: " + num);  
        }  
    }  
    public static void main(String[] args) {  
        OuterClass test = new OuterClass();  
        OuterClass.InnerClass in = test.new InnerClass();  
        in.printMsg();  
    }  
}
```

- **Anonymous Inner Class:** Một lớp được tạo ra để implements interface hoặc extends class. Tên của nó được quyết định bởi trình biên dịch java.

```
abstract class InnerClass{
    abstract void printMsg();
}

public class OuterClass {
    public static void main(String[] args) {
        int num = 10;
        InnerClass in = new InnerClass() {
            void printMsg() {
                System.out.println("Giá trị của num: " + num);
            }
        };
        in.printMsg();
    }
}
```

- **Local Inner Class:** Một lớp được tạo ra bên trong một phương thức. Nếu muốn gọi các phương thức của lớp được khai báo bên trong một phương thức, chúng ta phải tạo ra thể hiện của lớp này bên trong phương thức chứa nó.

```
public class OuterClass {
    int num = 10;
    void printMsg() {
        class InnerClass{
            void display() {
                System.out.println("Giá trị của num: " + num);
            }
        }
        InnerClass in = new InnerClass();
        in.display();
    }
    public static void main(String[] args) {
        OuterClass out = new OuterClass();
        out.printMsg();
    }
}
```

- **Static :** Một lớp static được tạo bên trong một lớp được gọi là lớp static lồng hay **static nested class**. Nó không thể truy cập các thành viên và phương thức non-static. Nó có thể được truy cập bởi tên lớp bên ngoài.

- Nó có thể truy cập các thành viên dữ liệu tĩnh (static data members) của lớp ngoài bao gồm cả private.
- Static nested class không thể truy cập thành viên dữ liệu hoặc phương thức non-static (instance).

```
public class OuterClass {
    static int num = 10;

    static class InnerClass{
        void printMsg() {
            System.out.println("Giá trị của num: " + num);
        }
    }

    public static void main(String[] args) {
        OuterClass.InnerClass test = new OuterClass.InnerClass();
        test.printMsg();
    }
}
```

2.2. Phân loại biến trong Java

Xét theo khía cạnh kiểu dữ liệu có thể chia biến thành hai loại : Biến thuộc một kiểu dữ liệu cơ bản và biến tham chiếu đối tượng.

- Biến thuộc một kiểu dữ liệu cơ bản có kích thước cố định tùy theo kiểu dữ liệu đó là gì. Các kiểu dữ liệu cơ bản của Java được mô tả chi tiết trong bảng 1.1.
- Biến tham chiếu đối tượng đóng vai trò như một trỏ trỏ tới một đối tượng nào đó.

Xét theo khía cạnh phạm vi truy nhập có thể chia biến thành ba loại : biến địa phương, biến thực thể và biến lớp.

- Biến địa phương được khai báo bên trong một phương thức. Nó được sinh ra khi phương thức được gọi và dòng lệnh khai báo nó được thực thi. Nó hết hiệu lực khi phương thức kết thúc.

```
int minValue(int num1, int num2){
    int min;
    if(num1>=num2){
        min=num2;
    }
    else
        min=num1;
    return min;
}
```


Trong ví dụ trên, biến `min` chỉ có hiệu lực trong phạm vi của phương thức `minValue`.

- Biến thực thể được khai báo bên trong một lớp nhưng không nằm trong một phương thức nào. Biến này được sinh ra khi đối tượng được tạo ra và hết hiệu lực khi đối tượng đó bị hủy.

```
public class Student {
    String studentID;
    String name;
    double point;
    Student(String n, String sID){
        name = n;
        studentID = sID;
    }
    void setPoint(String sID, double pnt){
        point = pnt;
    }
    public void getPoint(String sID){
        System.out.println("Điểm của sinh viên " + this.name + " là: " + point);
    }
}

public class StudentManage {
    public static void main(String[] args) {
        Student s1 = new Student("Nguyen Van A", "PTIT1234");
        s1.setPoint("PTIT1234", 8.5);
        s1.getPoint("PTIT1234");
    }
}
```

Trong ví dụ trên, các biến `studentID`, `name`, `point` là các biến thực thể và được khai báo trong lớp `Student`. Các biến này chỉ có hiệu lực khi ta tạo ra một thực thể `s1` và chỉ được truy nhập thông qua thực thể `s1`. Giá trị của biến chỉ có hiệu lực đối với một thực thể nào đó. Ví dụ, giá trị `'PTIT1234'` chỉ có hiệu lực với thực thể `s1`. Với thực thể khác, giá trị này sẽ thay đổi.

- Biến lớp : Đôi khi ta muốn có một biến dùng chung cho tất cả các đối tượng của một lớp nào đó. Ta gọi các biến dùng chung này là biến của lớp (class variable) hay gọi tắt là biến lớp. Biến này không gắn với bất cứ một đối tượng nào mà chỉ gắn với lớp đối tượng. Để phân biệt giữa biến thực thể và biến lớp khi khai báo trong một lớp, ta dùng từ khóa **static** cho biến lớp.

```

public class Student {
    String studentID;
    String name;
    double point;
    static int numOfStudent;
    Student(String n, String sID){
        name = n;
        studentID = sID;
        numOfStudent++;
    }
}

public class StudentManage {
    public static void main(String[] args) {
        Student s1 = new Student("Nguyen Van A", "PTIT1");
        Student s2 = new Student("Nguyen Van B", "PTIT2");
        System.out.println("Tổng số sinh viên: " + Student.numOfStudent);
    }
}

```

2.3. Một số thư viện cơ bản trong Java

2.3.1. Lớp bao ngoài (Wrapper class)

Dữ liệu cơ bản không phải là các đối tượng. Do đó chúng ta không thể sử dụng những biến dữ liệu cơ bản để truy cập đến các phương thức có sẵn của Java. Để truy cập được các phương thức có sẵn, ta phải sử dụng lớp bao ngoài của các dữ liệu này để tạo đối tượng. Bảng dưới đây cung cấp tên các lớp bao ngoài của các loại dữ liệu cơ bản

Loại dữ liệu	Lớp bao ngoài
boolean	Boolean
byte	Byte
char	Character
double	Double
int	Integer
long	Long
short	Short

Bảng 2. 2 Các lớp bao ngoài của dữ liệu cơ bản

Ví dụ :

```
Boolean bl = new Boolean("Sai");
Integer num = new Integer("20");
int a = num.intValue();
```

2.3.2. ArrayList

Khác với khai báo mảng là ta phải biết trước số phần tử của mảng, khi khai báo ArrayList, ta không cần biết một ArrayList có kích thước bao nhiêu. Khi tạo ArrayList, nó sẽ tự tăng lên hoặc giảm đi khi các đối tượng được thêm vào hoặc lấy ra. Cách khai báo ArrayList :

```
ArrayList<type> list = new ArrayList<type>();
```

Lưu ý: Phải sử dụng thư viện ArrayList trong gói util: `import java.util.ArrayList;`

Ví dụ:

```
ArrayList<String> list = new ArrayList<String>();
```

```
ArrayList<Student> stl = new ArrayList<Student>();
```

Một số phương thức của ArrayList:

- `add(Object name)`: gắn đối tượng vào cuối danh sách.
- `add(int i, Object name)`: gắn đối tượng vào vị trí *i* trong danh sách.
- `get(int i)`: trả về đối tượng tại vị trí *i* trong danh sách
- `remove(int i)`: xóa đối tượng tại vị trí *i* trong danh sách.
- `remove(Object name)`: xóa đối tượng nếu nó nằm trong danh sách.
- `contains(Object name)`: trả về giá trị true nếu có trong danh sách
- `isEmpty()`: trả về true nếu ds rỗng
- `size()`: trả về số phần tử của danh sách

So sánh ArrayList và mảng thông thường:

ArrayList	Array
<pre>ArrayList<String> arl = new ArrayList<>();</pre>	<pre>String[] ar = new String[10]</pre>
<pre>String a = new String("PTIT"); arl.add(a);</pre>	<pre>String a = new String("PTIT"); ar[0] = a;</pre>
<pre>int arlSize = arl.size();</pre>	<pre>int arSize = a.length();</pre>

<code>String str = arl.get(0);</code>	<code>String str = ar[0];</code>
<code>arl.remove(1);</code>	<code>ar[1] = null;</code>
<pre>String b = "PTIT"; boolean isIn = arl.contains(b); System.out.println(isIn);</pre>	<pre>String b = "PTIT"; boolean isIn = false; for(String item: ar){ if(b.equals(item)){ isIn = true; } } System.out.println(isIn);</pre>

Bảng 2. 3 So sánh Array và ArrayList

2.3.3. Lớp xử lý chuỗi String

Lớp String cung cấp một số phương thức để xử lý các chuỗi ký tự. Một số hàm khởi tạo của lớp này bao gồm :

```
//Khởi tạo một chuỗi rỗng
String st1 = new String();

//Khởi tạo một chuỗi có nội dung
String st2 = new String("PTIT");

//Khởi tạo từ một mảng các ký tự
char ch[] = {'P','T','I','T'};
String st3 = new String(ch);
String st4 = new String(ch,0,2);
```

Một số phương thức của lớp String :

- `charAt()` : Trả về ký tự tại vị trí được chỉ ra.

```
String st5 = new String("Multimedia");
char ch1 = st5.charAt(1);
```

- `startsWith()` : So sánh chuỗi được truyền vào phương thức với đoạn đầu tiên của chuỗi đã cho. Phương thức trả về giá trị kiểu boolean.

```
String st6 = new String("PTIT Multimedia");
boolean fl = st6.startsWith("PTIT");
```

- `endsWith()` : Tương tự như `startsWith()` nhưng so sánh ở cuối chuỗi.

- copyValueOf() : Hàm trả về một chuỗi được copy từ một mảng các ký tự.

```
char name[] = {'P', 'T', 'I', 'T'};  
String subname= String.valueOf(name, 1, 3);  
System.out.println(subname);
```

Kết quả chuỗi subname = 'TIT'.

2.3.4. Lớp *java.lang.Math*

Lớp *java.lang.Math* bao gồm một số phương thức hỗ trợ các hàm toán học bao gồm :

- abs() : hàm trả về giá trị tuyệt đối của đối số truyền vào.
- ceil() : hàm trả về giá trị nguyên lớn hơn hoặc bằng đối số truyền vào.
- floor() : hàm trả về giá trị nguyên nhỏ hơn hoặc bằng đối số truyền vào.
- max() : hàm trả về giá trị lớn nhất của hai đối số truyền vào.
- min() : hàm trả về giá trị nhỏ nhất của hai đối số truyền vào.
- round() : hàm trả về giá trị nguyên gần nhất với số thực được truyền vào.
- random() : hàm trả về một số kiểu double trong khoảng từ 0 đến 1.
- sqrt() : hàm trả về giá trị căn bậc hai.

BÀI TẬP CHƯƠNG 2

Bài 1. Tạo lớp Student với thiết kế như sau :

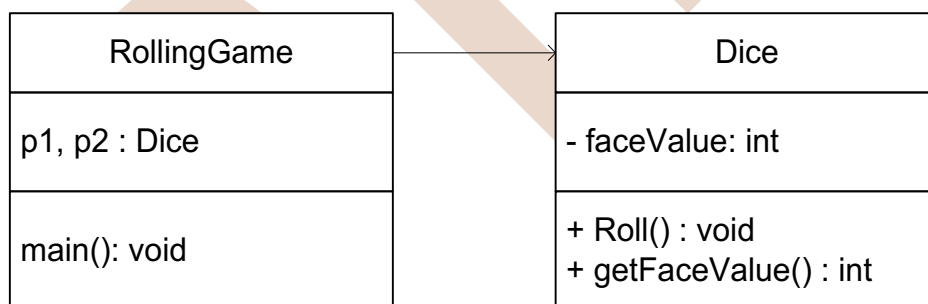
Student
- name : String - age : int - stdID : String - point : double
+ setName(String name) : void + getName() : String + setPoint(double point): void + getPoint() : double

Bài 2. Khởi tạo mảng gồm 5 sinh viên. Cho người dùng nhập dữ liệu của 5 sinh viên.

- In ra danh sách của các sinh viên sau khi nhập.
- Tính điểm trung bình của 5 sinh viên đó.

Bài 3. Tạo một mảng ArrayList và cho phép người dùng nhập dữ liệu sinh viên liên tục. Sau mỗi lần nhập, chương trình hỏi người dùng có muốn nhập nữa hay không (Y/N). Sắp xếp điểm của sinh viên tăng dần và in ra danh sách đã sắp xếp.

Bài 4 : Viết chương trình chơi xúc sắc. Hai người chơi lần lượt tung xúc sắc. Nếu ai tung được mặt nhiều điểm thì người đó thắng. Chương trình sử dụng lớp xúc sắc Dice.



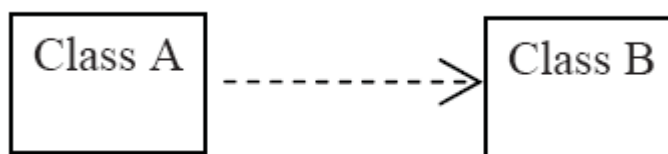
Gợi ý: Phương thức Roll() là phương thức tạo giá trị ngẫu nhiên và lưu vào biến faceValue. Cách tạo số ngẫu nhiên trong khoảng [min, max] :

$$faceValue = (int)(Math.random() * (max - min + 1)) + min$$

CHƯƠNG 3. TÍNH KẾ THỪA VÀ ĐA HÌNH TRONG JAVA

3.1. *Mối quan hệ giữa các lớp*

3.1.1. *Mối quan hệ “Phụ thuộc” – Dependency*



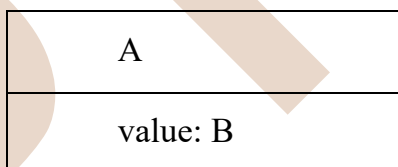
Hình 3. 1 Mối quan hệ phụ thuộc

Phụ thuộc là quan hệ chỉ một lớp tham chiếu tới một lớp khác. Phụ thuộc là mối quan hệ giữa hai lớp đối tượng: một lớp đối tượng B có tính độc lập và một lớp đối tượng A phụ thuộc vào lớp B. Khi thay đổi lớp tham chiếu B sẽ ảnh hưởng tới lớp sử dụng nó là lớp A.

Ví dụ điển hình của mối quan hệ này là lớp A có một phương thức được truyền một tham số là đối tượng của lớp B hoặc lớp A sử dụng biến hoặc phương thức “static” trong lớp B.

3.1.2. *Mối quan hệ “Kết nối” – Association:*

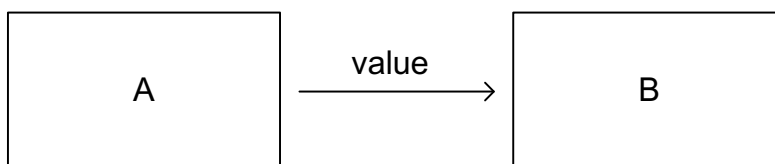
Trong một lớp, các thuộc tính có thể có các kiểu dữ liệu nguyên tố (int, boolean,...) nhưng cũng có thể có kiểu dữ liệu là một đối tượng được định nghĩa bởi một lớp khác.



Hình 3. 2 Mối quan hệ kết nối

Hình trên mô tả lớp “A” có thuộc tính là “value”. Thuộc tính value không phải là kiểu dữ liệu cơ bản mà nó có kiểu là một đối tượng của lớp “B”. Như vậy lớp A “uses” lớp B. Động từ “uses” được sử dụng để mô tả mối quan hệ này.

Một cách khác để biểu diễn thông tin trên bằng biểu đồ mối quan hệ kết nối như sau:

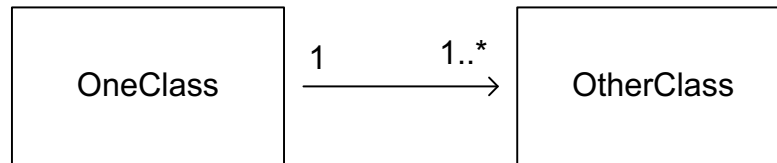


Hình 3. 3 Mối quan hệ kết nối

Mối quan hệ được biểu diễn bằng mũi tên một chiều có nghĩa là lớp A có thể truy cập lớp B nhưng ngược lại không đúng.

Về nguyên tắc, chúng ta cũng phải sử dụng mối quan hệ này nếu trong lớp có biến kiểu String vì kiểu String không phải là kiểu cơ bản. Tuy nhiên, lớp String được định nghĩa sẵn tại Java platform nên nó có thể được coi như kiểu cơ bản.

Một cách khác dùng để biểu diễn mối quan hệ “Kết nối” với “Tính nhiều”:



Hình 3. 4 Mối quan hệ kết nối với tính nhiều

Mô tả trên nghĩa là lớp “OneClass” chứa một mảng các đối tượng có kiểu “OtherClass”.

Ví dụ: Lớp Catalogue sử dụng lớp ItemForSale



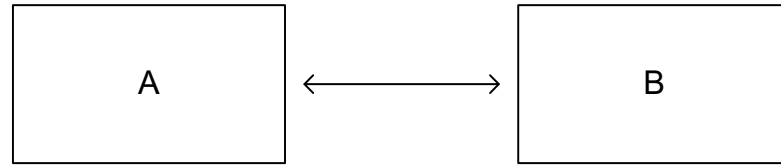
Biểu đồ trên mô tả như sau: Hệ thống gồm 1 lớp “Catalogue” và 1 lớp “ItemForSale”.

Lớp “ItemForSale” có thuộc tính “name” kiểu String và thuộc tính “price” kiểu int. Lớp này cũng bao gồm phương thức setPrice() với một tham số newPrice kiểu integer.

Lớp “Catalogue” có một thuộc tính “listOfItem” với kiểu là “ItemForSale”. Biến listOfItem có thể chứa 0 hoặc nhiều các mặt hàng để bán nên biến này sẽ là một mảng có kiểu là ItemForSale. Lớp “Catalogue” bao gồm một phương thức addItem() với tham số “item” có kiểu là ItemForSale. Phương thức này có chức năng thêm mặt hàng vào danh sách listOfItems.

Như vậy, về nguyên tắc lớp Catalogue được truy cập tới lớp ItemForSale để xem, thêm, xóa các mặt hàng. Tuy nhiên, một mặt hàng không thể truy cập tới lớp Catalogue để thêm mặt hàng.

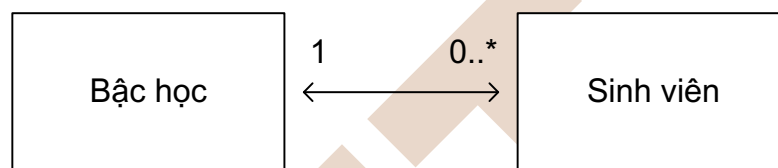
3.1.3. *Mối quan hệ “Kết nối hai chiều” – Bidirectional Association*



Hình 3. 5 Mối quan hệ kết nối hai chiều

Biểu đồ trên mô tả mối quan hệ hai chiều giữa A và B. Lớp A sử dụng lớp B. Tuy nhiên, lớp B có thể truy cập tới lớp A.

Ví dụ:



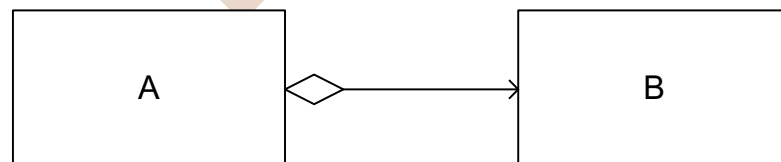
Với một lớp Bậc học, chúng ta có thể xem được các sinh viên đang theo học ở bậc học đó. Ngược lại, với một Sinh viên, chúng ta cũng có thể xem được sinh viên đó đang học ở bậc học nào.

Với quan hệ trên, một bậc học có thể có nhiều sinh viên nhưng một sinh viên chỉ có thể học tại một bậc học.

3.1.4. *Mối quan hệ “Thành phần” – Aggregation*

Mối quan hệ Aggregation là mối quan hệ giữa tổng thể và bộ phận (Whole – Parts). Ví dụ mối quan hệ giữa lớp Car (là lớp tổng thể) và các lớp Lốp, Động cơ,... (là các lớp bộ phận).

Trong mối quan hệ này, một lớp biểu diễn cái lớn hơn còn lớp kia biểu diễn cái nhỏ hơn.

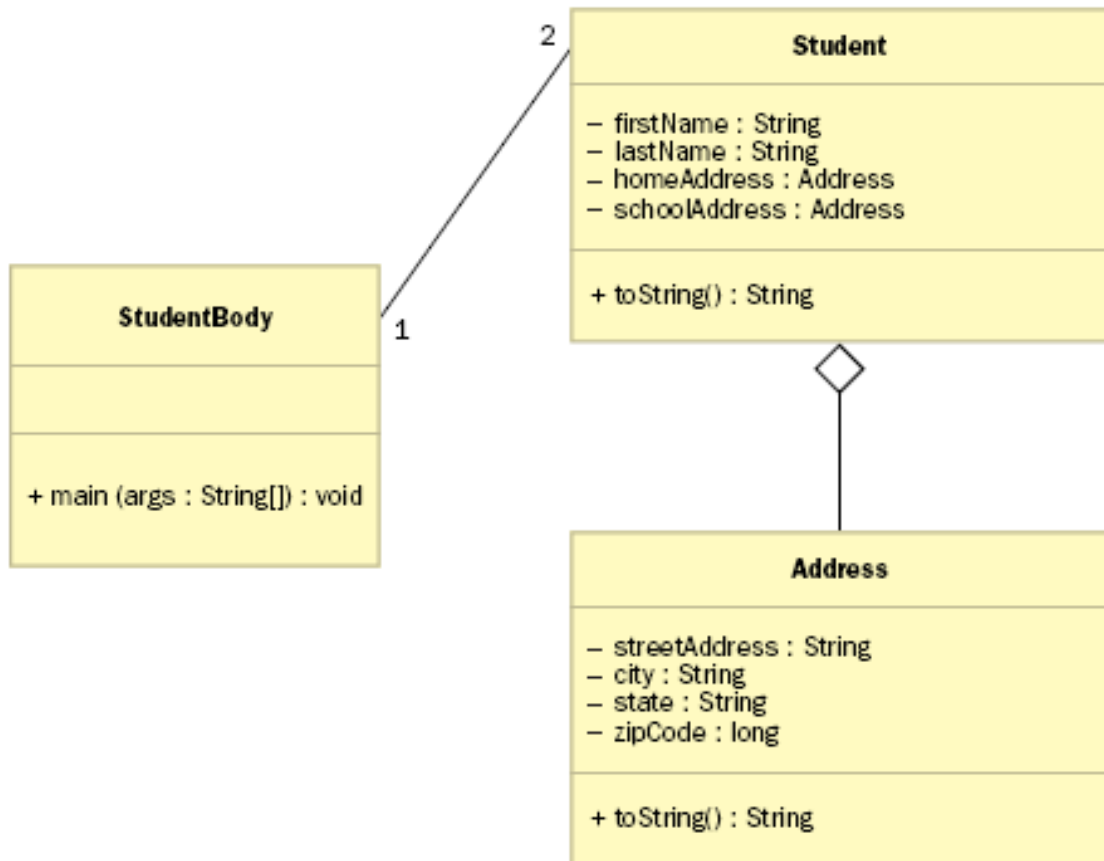


Hình 3. 6 Mối quan hệ thành phần

Mối quan hệ này được mô tả bởi động từ “**Có một**” – **Has a**. Trong biểu đồ trên, mối quan hệ được phát biểu là A có một B.

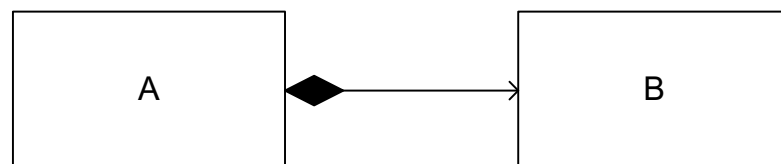
Ví dụ: Xe Car có một Động cơ. Mỗi quan hệ Aggregation được coi là không có sự khác biệt nhiều với mỗi quan hệ Association. Ví dụ, chúng ta cũng có thể nghĩ rằng mỗi quan hệ giữa Car và Engine có thể chỉ cần dùng mỗi quan hệ Association. Tức là Car sử dụng Engine.

Ví dụ 2 :



Hình 3. 7 Ví dụ về mối quan hệ giữa các lớp

3.1.5. Mối quan hệ “Hợp thành” – Composition

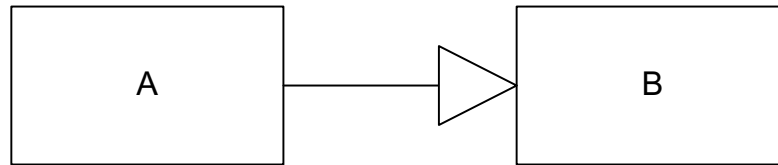


Hình 3. 8 Mối quan hệ hợp thành

Mối quan hệ hợp thành là mối quan hệ mạnh hơn mỗi quan hệ thành phần. Điều này có nghĩa là lớp B là một thành phần không thể tách rời của lớp A. Nếu lớp A không tồn tại thì lớp B cũng sẽ không tồn tại.

Ví dụ các Điểm, Đường và Hình là các thành phần của một bức tranh. Nếu bức tranh bị xóa thì các lớp thành phần cũng sẽ bị xóa.

3.1.6. Mối quan hệ “Kế thừa” – Inheritance

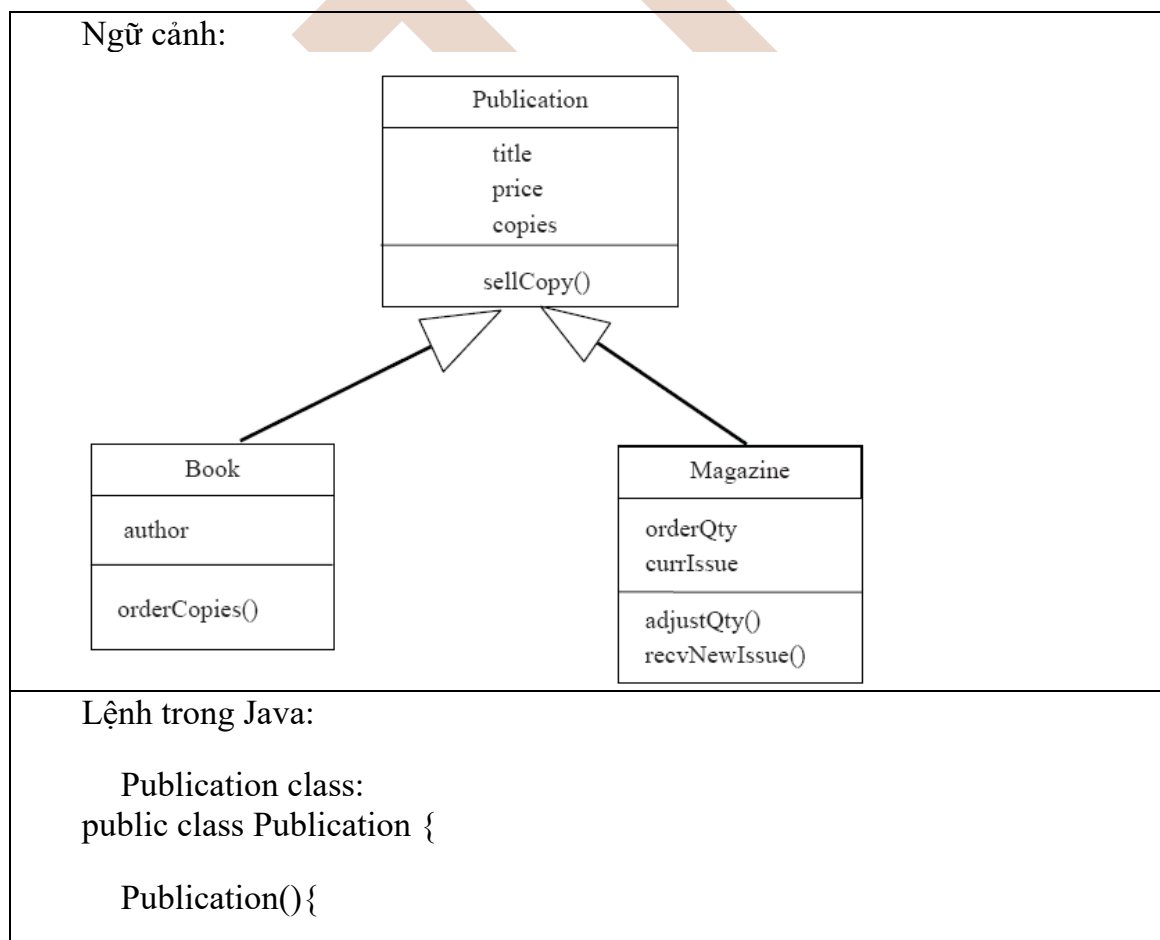


Hình 3. 9 Mối quan hệ kế thừa

Biểu đồ trên mô tả lớp A kế thừa lớp B. Nói cách khác, lớp B là lớp cha và lớp A là lớp con. Kế thừa cho phép chia sẻ các phương thức của lớp cha với lớp con. Lớp con có thể bổ sung thêm phương thức mới hoặc cài chồng (override) phương thức đã có ở lớp cha. Cụm từ “**is – a**” được dùng để mô tả mối quan hệ này.

Ví dụ: Lớp Oto kế thừa từ lớp Phuongtien. Khi đó ta có thể nói, Oto là một loại Phuongtien.

Ví dụ 1 :



```

        System.out.println("Publication");
    }
}

Book class:

public class Book extends Publication{

    Book(){

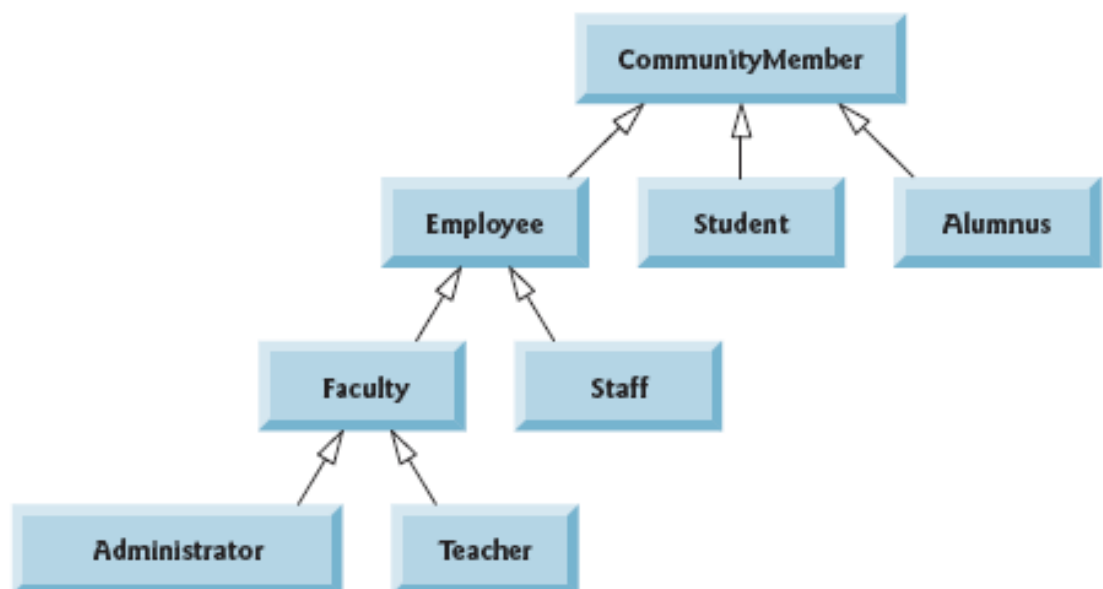
        System.out.println("Book");

    }

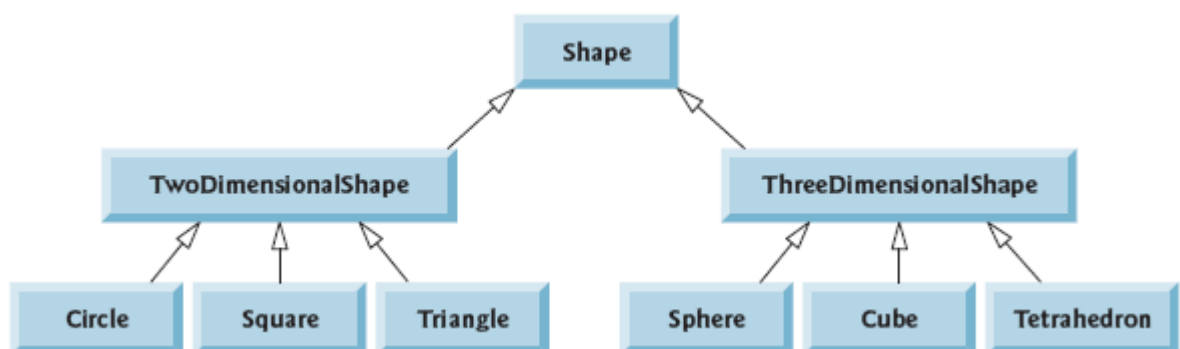
}

```

Ví dụ 2 :



Ví dụ 3 :



3.1.7. Kế thừa phương thức khởi tạo của lớp cha

Mỗi lớp (cho dù là lớp con hay lớp cha) đều có riêng một hàm khởi tạo để thiết lập các trạng thái ban đầu của đối tượng được tạo ra.

Hàm khởi tạo của lớp cha có trách nhiệm khởi tạo trạng thái chung. Mỗi lớp con có hàm khởi tạo riêng để tạo ra trạng thái cụ thể của đối tượng con. Thông thường, hàm khởi tạo lớp con sẽ gọi hàm khởi tạo lớp cha. Việc này thực hiện bằng cách sử dụng từ khóa `super`.

Nếu từ khóa `super` được gọi, nó phải đứng ở dòng đầu tiên trong hàm khởi tạo của lớp con.

Thông thường, một số biến của lớp con sẽ trùng với các biến của lớp cha. Do đó, các biến trùng này sẽ được truyền vào hàm `super`.

Ví dụ:

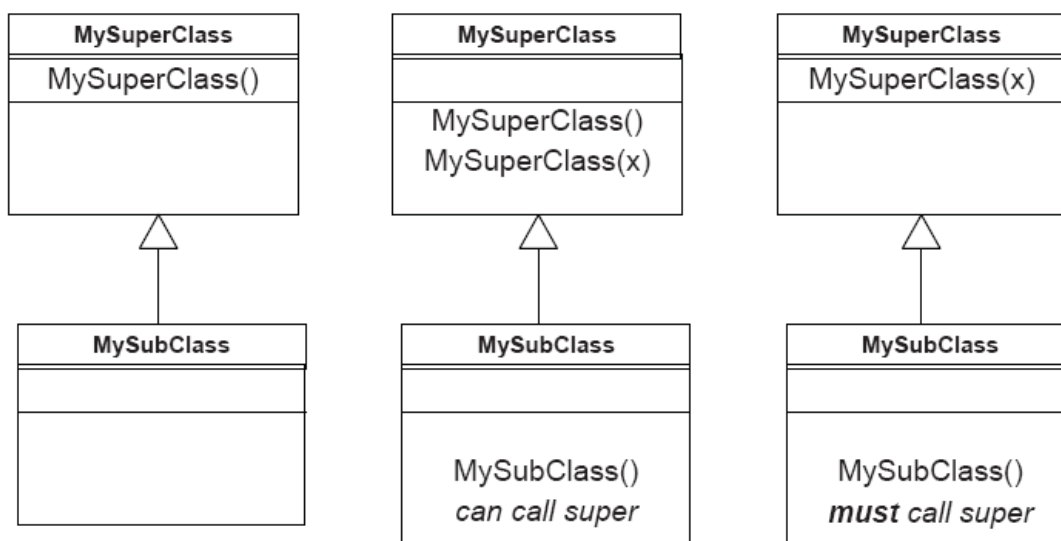
```
public class Publication {
    String title;
    double price;
    int copies;
    Publication(String pTitle, double pPrice, int pCopies) {
        title = pTitle;
        price = pPrice;
        copies = pCopies;
    }
}

public class Book extends Publication{
    String author;
    Book(String pTitle, String pAuthor, double pPrice, int
pCopies) {
        super(pTitle, pPrice, pCopies);
        author = pAuthor;
    }
}
```

Nguyên tắc gọi hàm khởi tạo lớp cha:

- Nếu lớp cha có một hàm khởi tạo không có tham số thì hàm khởi tạo này của lớp cha sẽ mặc định được gọi trong hàm khởi tạo của lớp con.
- Nếu lớp cha có duy nhất một hàm khởi tạo có tham số thì hàm `super` **bắt buộc** phải được gọi tại lớp con.

Ví dụ:



Hình bên trái: Lớp con không có hàm khởi tạo nhưng thay vào đó hàm khởi tạo lớp cha sẽ mặc định được gọi tại lớp con. Tuy nhiên khuyến nghị là nên có hàm khởi tạo tại lớp con.

Hình giữa: Lớp cha có 2 hàm khởi tạo. Nếu hàm khởi tạo lớp con không có từ khóa `super` thì hàm khởi tạo không tham số của lớp cha sẽ mặc định được gọi tại lớp con.

Hình bên phải: Lớp cha có 1 hàm khởi tạo có tham số. Do đó từ khóa `super` bắt buộc phải được sử dụng trong hàm khởi tạo của lớp con. Từ khóa `super` được dùng trong một số tình huống sau :

- Dùng trong phương thức khởi tạo của lớp con trong trường hợp hàm khởi tạo lớp cha có tham số.
- Dùng để gọi biến của lớp cha. Ví dụ lớp cha và lớp con có biến `count` và cùng kiểu `int`. Trong lớp con nếu chúng ta muốn sử dụng biến của lớp cha thì ta dùng từ khóa `super` như sau : `super.hunger`.

3.1.8. Thiết kế cây kế thừa

Giả sử ta phải thiết kế một chương trình mô phỏng các động vật trong rừng. Chúng ta được cung cấp thông tin về một số loại vật ban đầu bao gồm sư tử, hà mã, hổ, chó, mèo, sói và chúng ta phải sắp xếp chúng thành các nhóm động vật có cùng tính chất. Sau đó, người lập trình khác có thể thêm các động vật khác vào chương trình và được tận dụng những đoạn mã đã được viết sẵn cho nhóm của động vật được thêm vào đó. Việc thiết kế bao gồm các bước sau :

Bước 1 : Xác định các đối tượng có chung thuộc tính và hành động.

Giả sử nhóm động vật được cung cấp có những thuộc tính chung bao gồm :

- `Picture` : tên file ảnh (.jpeg) đại diện của động vật.
- `Food` : loại thức ăn động vật ăn : `meat` (thịt) hoặc `grass` (cỏ).

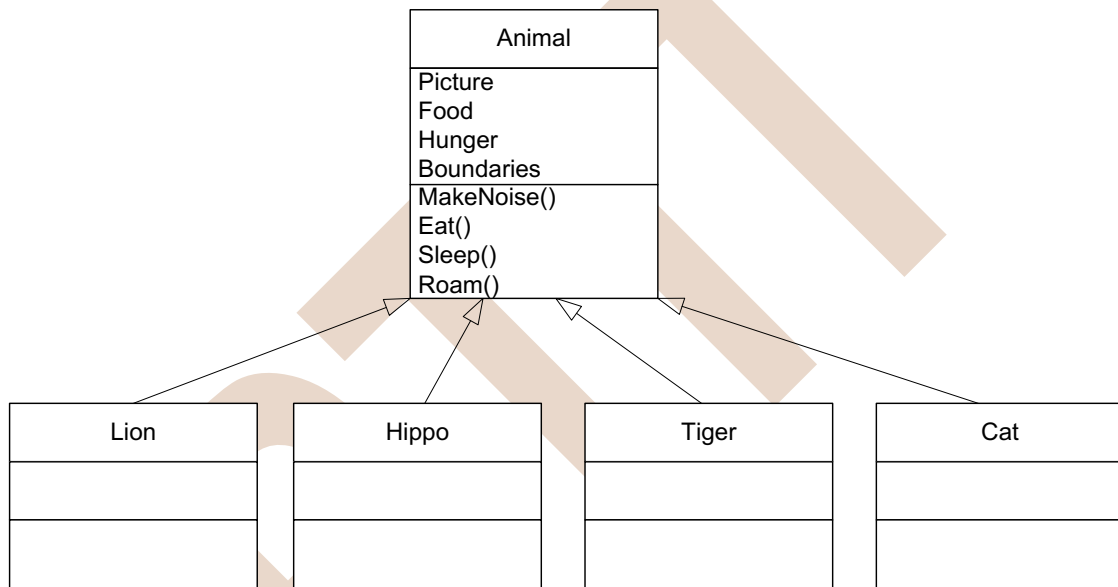
- Hunger : biến kiểu int biểu diễn mức độ đói của con vật.
- Boundaries : Giá trị biểu diễn khu vực con vật di chuyển
- Location : Tọa độ con vật di chuyển trong khu vực của nó.

Các phương thức chung bao gồm :

- Makenoise() : Hành vi con vật phát ra tiếng kêu
- Eat() : Hành vi khi con vật gắp cỏ hoặc thịt
- Sleep() : Hành vi khi con vật ngủ
- Roam() : Hành vi khi con vật di chuyển

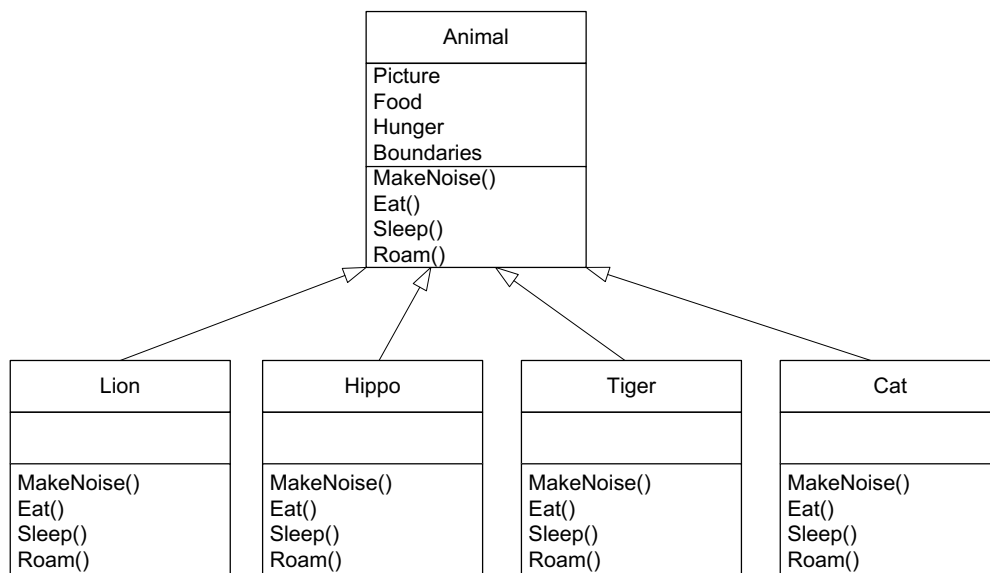
Bước 2 : Thiết kế lớp chung để mô tả các thuộc tính và phương thức chung.

Do các con vật trên đều là động vật nên ta gọi lớp chung là lớp Animal. Lớp Animal có các biến và phương thức được mô tả ở bước 1.



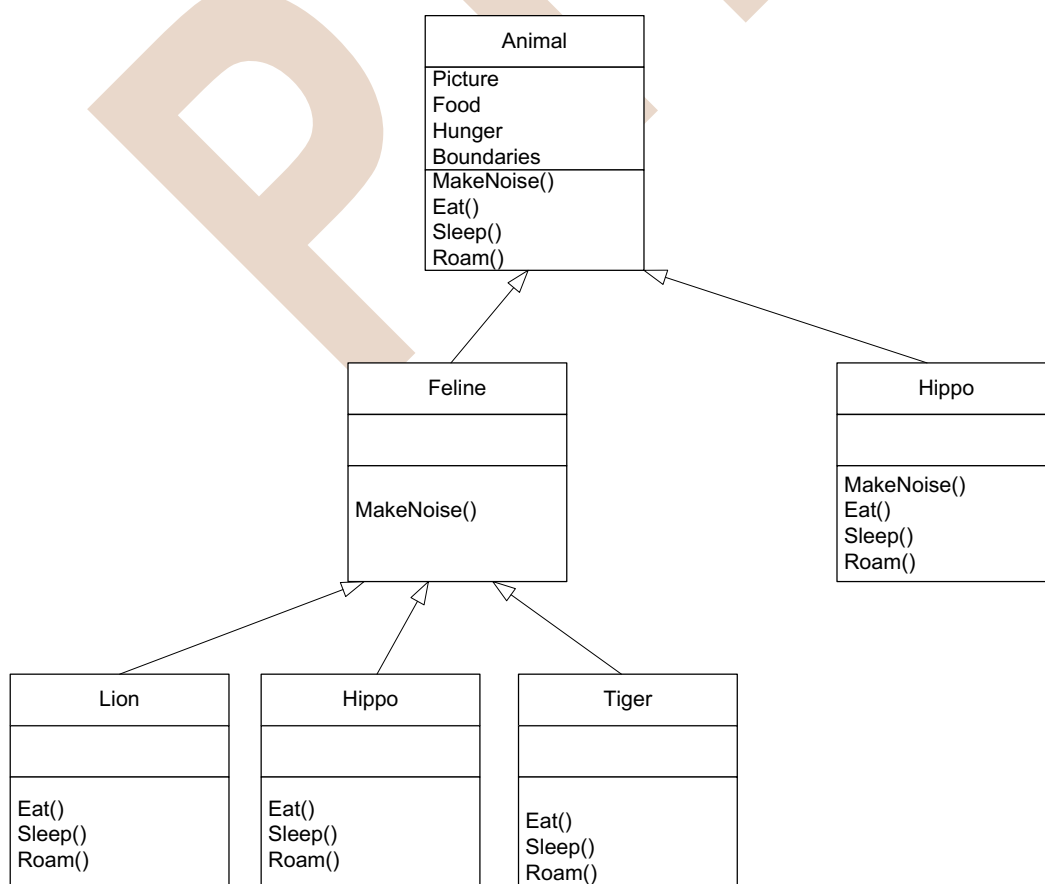
Bước 3 : Xác định xem một lớp con nào đó có cần những hành vi (phương thức) đặc thù của lớp con đó không.

Chúng ta thấy rằng, các động vật trên đều có hành vi kêu, nhưng tiếng kêu của sư tử phải khác với tiếng kêu của chó. Hoặc kiểu ăn của sư tử phải khác với kiểu ăn của hà mã. Vì vậy, mỗi động vật sẽ phải cài đè (override) phương thức ăn và phương thức kêu để mô tả các đặc trưng riêng của nó.



Bước 4 : Tiếp tục dùng trừu tượng hóa để tìm các nhóm động vật có thể có cùng hành vi giống nhau để phân nhóm mịn hơn.

Trong nhóm các động vật trên, chúng ta có thể thấy Sói và Chó có thể được nhóm vào một nhóm, Sư tử, Hổ và Mèo có thể nhóm vào một nhóm. Trong nhóm thuộc họ Chó (canines), có thể Chó và Sói cùng kiểu kêu nhưng kiểu di chuyển thì khác nhau (chó Sói thường di chuyển theo bầy). Vì vậy, phương thức `roam()` có thể được chuyển lên lớp Canines. Tương tự với lớp thuộc họ Mèo (Feline). Vì vậy, chúng ta có cây kế thừa được phát triển như sau :



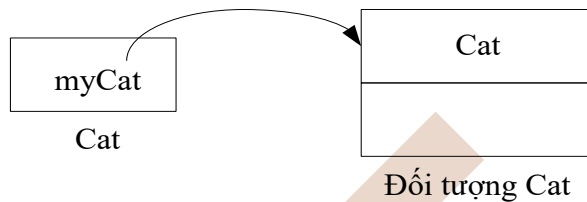
3.2. Tính đa hình trong Java

3.2.1. Tính đa hình

Giả sử ta khai báo và khởi tạo đối tượng Cat như sau :

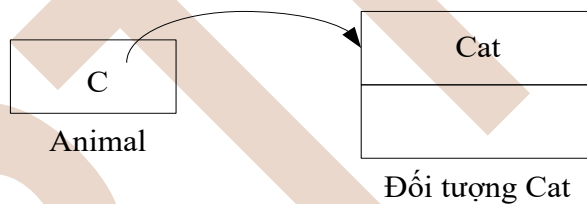
Cat myCat = new Cat() ;

Như phần 2.1.6, việc khai báo và khởi tạo ở trên đồng nghĩa với việc JVM tạo ra một biến tham chiếu myCat có kiểu Cat và tham chiếu đến đối tượng Cat.



Như vậy, ta thấy rằng kiểu của biến tham chiếu và kiểu của đối tượng cùng là Cat. Tuy nhiên, Java cho phép khai báo kiểu của biến tham chiếu khác với kiểu của đối tượng. Ví dụ, ta có thể khai báo như sau :

Animal C = new Cat() ;



Với đa hình, biến tham chiếu có thể thuộc kiểu lớp cha của lớp đối tượng khởi tạo. Khi ta khai báo một biến tham chiếu thuộc kiểu lớp cha, nó có thể được gán với bất cứ đối tượng nào thuộc một trong các lớp con.

Như vậy, **để dễ quản lý nhiều loại đối tượng của các lớp con, ta sử dụng tính đa hình để định nghĩa một mảng lớp cha. Mỗi phần tử của mảng sẽ tham chiếu tới đối tượng của lớp con.** Ví dụ, ta có thể khai báo một mảng kiểu Animal. Mỗi phần tử của mảng sẽ tham chiếu các đối tượng Dog, Cat, Wolf,...

```

public static void main(String[] args) {
    Animal[] animals = new Animal[5];
    animals[0] = new Dog();
    animals[1] = new Cat();
    animals[2] = new Lion();
    animals[3] = new Wolf();
    animals[4] = new Hippo();

    for(int i=0;i<animals.length;i++){
        animals[i].eat();
    }
}

```

Kết quả :

```

Dog eating!
Cat eating!
Lion eating!
Wolf eating!
Hippo eating!

```

Lưu ý : Trong việc gọi các phương thức của lớp con bằng tham chiếu lớp cha, các biến tham chiếu này chỉ gọi được những phương thức mà lớp cha có hoặc những phương thức lớp con cài đặt. Biến tham chiếu không thể gọi những phương thức của lớp con có mà lớp cha không có. Ví dụ : Trong ví dụ trên, tham chiếu `animals` không thể gọi phương thức `eat()` nếu lớp `Animal` không có phương thức này. Nhưng nếu lớp `Animal` có thêm phương thức `makeSound()` mà các lớp con không có thì các tham chiếu vẫn có thể gọi được phương thức `makeSound()` của lớp `Animal`.

Ngoài ra, một tính năng khác nữa của đa hình trong Java là tham số của một phương thức có kiểu lớp cha nhưng có thể nhận giá trị truyền vào là các đối tượng của lớp con. Như vậy, ta chỉ cần viết phương thức đó một lần nhưng có thể dùng cho bất cứ lớp con nào của lớp cha.

Ví dụ : Ta cần tạo thêm một lớp nhân viên quản lý vườn thú `AnimalManager`. Trong lớp này có phương thức là cho thú ăn `AnimalFeed`. Vì mỗi động vật có kiểu ăn khác nhau nên mỗi khi ăn, phương thức `eat()` của động vật đó được gọi. Vì vậy, phương thức `AnimalFeed` phải nhận giá trị truyền vào là đối tượng động vật cụ thể nào đó như là `Dog`, `Cat`,... để sử dụng phương thức `eat()` của đối tượng đó. Để làm được điều này, ta phải sử dụng tính đa hình của Java trong đó tham số của phương thức `AnimalFeed` có kiểu là `Animal`.

```

public class AnimalManager {
    void AnimalFeed(Animal a){
        a.eat();
    }
}

public static void main(String[] args) {
    Animal[] animals = new Animal[5];
    animals[0] = new Dog();
    animals[1] = new Cat();
    animals[2] = new Lion();
    animals[3] = new Wolf();
    animals[4] = new Hippo();
    AnimalManager am = new AnimalManager();
    for(int i=0;i<animals.length;i++){
        am.AnimalFeed(animals[i]);
    }
}

```

Kết quả :

```

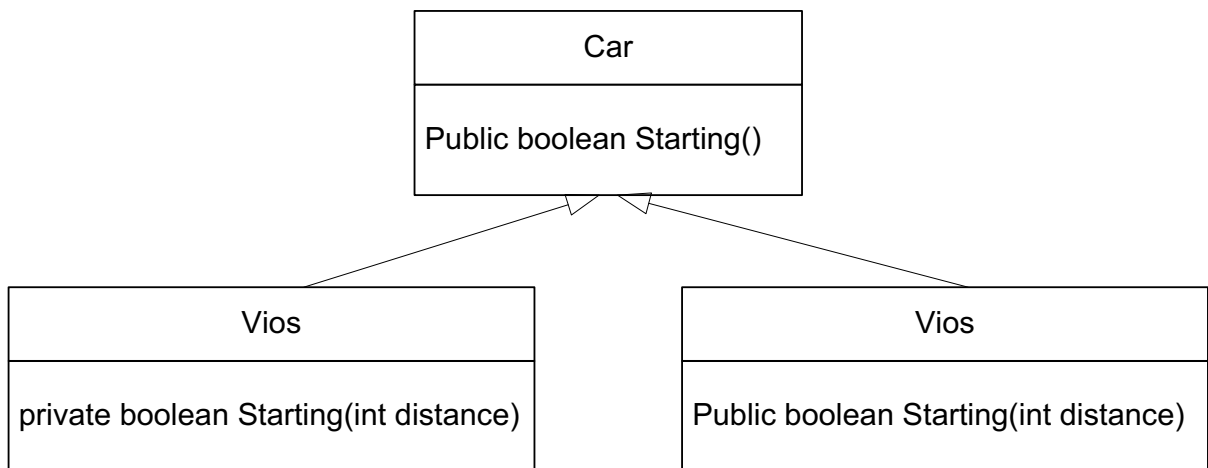
Dog eating!
Cat eating!
Lion eating!
Wolf eating!
Hippo eating!

```

3.2.2. Cài đặt phương thức (Override)

Khi cài đặt một phương thức của lớp cha, lớp con phải tuân thủ một số quy tắc sau :

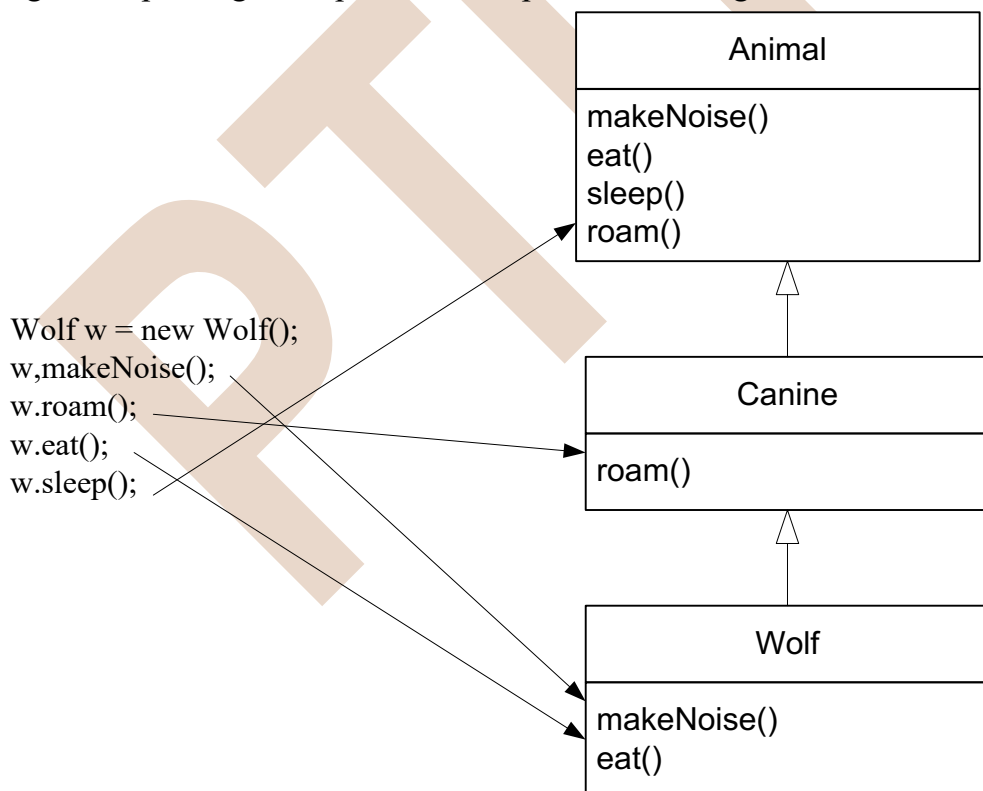
- Danh sách tham số phải trùng nhau, kiểu giá trị trả về phải tương thích : Phương thức của lớp cha có danh sách đối số như thế nào thì danh sách đối số của lớp con cũng phải như vậy. Phương thức của lớp cha khai báo trả về kiểu nào thì phương thức lớp con cũng phải khai báo kiểu trả về như vậy.
- Phương thức cài đặt không được giảm quyền truy nhập so với phiên bản của lớp cha. Ví dụ, phương thức lớp cha là public thì lớp con không được là private. Ngược lại thì được phép.



Không hợp lệ vì phương thức override không được giảm quyền truy nhập. Trường hợp này cũng không phải overload vì không được sửa tham số

Không hợp lệ vì phương thức override được sửa tham số. Trường hợp này là overload

- Nếu trong cây kế thừa có các lớp cha ở phía trên còn các lớp con ở phía dưới thì khi gọi một phương thức, phiên bản thấp nhất sẽ được gọi.



3.2.3. Cài chồng phương thức (Overload)

Cài chồng phương thức là tình huống trong một lớp có nhiều phương thức cùng tên nhưng khác danh sách đối số. Cài chồng phương thức có một số đặc điểm sau :

- Kiểu trả về có thể khác nhau với điều kiện là danh sách đối số cũng phải khác nhau.
- Có thể thay đổi mức truy nhập của các phương thức cài chõng một cách tùy ý.

3.2.4. Lớp trừu tượng và phương thức trừu tượng

Trong ví dụ về nhóm động vật ở phần trước, ta hoàn toàn có thể khai báo và khởi tạo ra các đối tượng cho các lớp với tên và tuổi. Ví dụ :

Dog d = new Dog("Jack ", 3) ;

Cat c = new Cat("Tom",4) ;

Trong trường hợp ta muốn tạo một đối tượng kiểu *Animal* thì hàm khởi tạo phải như thế nào ? Rõ ràng, *Animal* ở đây là một động vật trừu tượng không có tên, tuổi, đặc điểm cụ thể để ta có thể khởi tạo một đối tượng cụ thể kiểu *Animal*. Trong trường hợp này, ta có thể gọi *Animal* là lớp trừu tượng.

Ví dụ khai báo lớp trừu tượng :

```
public abstract class Animal {
    void eat() {

    }
    void makeSound() {
        System.out.println("Animal Sound");
    }
}
```

Như vậy, việc sử dụng lớp trừu tượng có mục đích như sau :

- Khi ta muốn tạo một lớp nhưng không muốn ai đó sử dụng để tạo các đối tượng cụ thể.
- Khi ta muốn sử dụng tính đa hình cho một nhóm các lớp thì ta cho các lớp đó cùng kế thừa một lớp trừu tượng.

Không chỉ có lớp, ta còn có thể khai báo các phương thức trừu tượng trong lớp trừu tượng. Một lớp trừu tượng có nghĩa phải tạo lớp con cho nó, còn một phương thức trừu tượng có nghĩa nó phải được cài đặt ở lớp con. Như vậy, khi ta muốn tất cả các lớp con của lớp trừu tượng bắt buộc phải viết lại cụ thể phương thức nào đó của lớp cha thì ta để phương thức này là phương thức trừu tượng. Ví dụ, nếu là lớp con của lớp *Animal* thì đều có phương thức là “kêu”. Tuy nhiên, phương thức “kêu” của các lớp *Dog*, *Cat*, *Lion*,... đều khác nhau. Do đó ta có thể để phương thức “kêu” của lớp *Animal* là phương thức trừu tượng.

Java quy định phương thức trừu tượng không có thân phương thức. Dòng khai báo phương thức kết thúc bằng dấu chấm phẩy và không có cặp ngoặc {}.

Ví dụ khai báo phương thức trừu tượng :

```
public abstract void eat() ;
```

Trong một lớp trừu tượng có thể có cả phương thức cụ thể và phương thức trừu tượng.

3.2.5. Lớp giao diện (interface)

Lớp giao diện là một tập các phương thức public được đưa ra để người dùng có thể tương tác với đối tượng. Lớp giao diện bao gồm các hằng số và các phương thức trừu tượng. Phương thức trừu tượng là phương thức chỉ có khai báo mà không có triển khai.

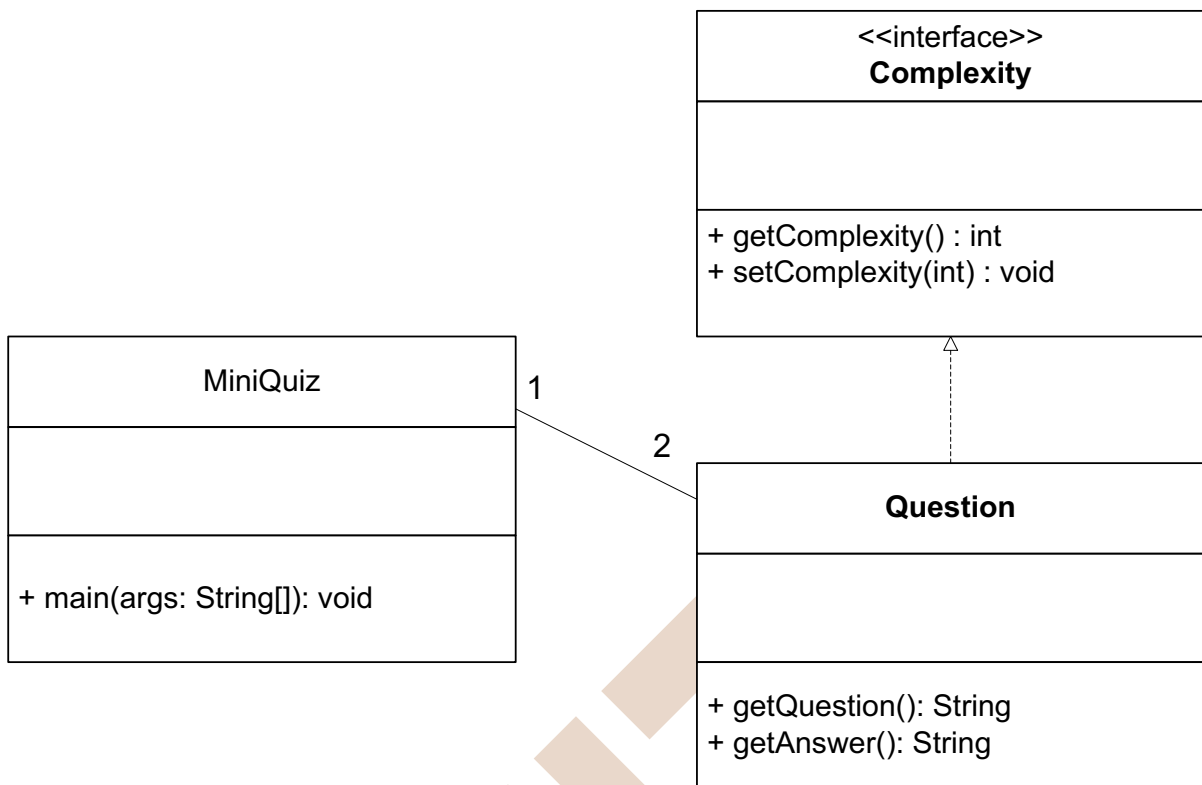
Lớp giao diện được dùng khi thiết kế một lớp có các phương thức chung. Các lớp triển khai lớp giao diện này bắt buộc phải tuân thủ các phương thức chung đó. Ví dụ: Lớp giao diện Động vật có phương thức kêu() có nghĩa là tất cả các lớp động vật đều phải kêu nhưng có thể kêu với các cách implement khác nhau. Lớp người là nói, lớp chó là sủa, lớp hổ là gầm,...

Khai báo lớp giao diện :

```
interface MyInterface {  
    public void method1() ;  
    public void method2() ;  
}
```

Lớp triển khai :

```
class Demo implements MyInterface {  
    public void method1() {  
        System.out.println("implementation of method1");  
    }  
    public void method2() {  
        System.out.println("implementation of method2");  
    }  
    public static void main(String arg[]) {  
        MyInterface obj = new Demo(); obj.method1();  
    }  
}
```



Hình 3. 10 Ví dụ về mô tả lớp giao diện bằng UML

Lớp giao diện có một số đặc điểm sau :

- Lớp giao diện chỉ cho một lớp giao diện khác kế thừa.
- Lớp giao diện chỉ chứa các phương thức trừu tượng mà không có các phương thức cụ thể.
- Các phương thức trừu tượng trong lớp giao diện chỉ có thể là public.
- Lớp giao diện chỉ chứa các biến loại public static final.

Vấn đề đa kế thừa : Ta có lớp b và lớp c cùng được kế thừa từ lớp a. Java không cho phép một lớp kế thừa từ hai lớp đồng thời. Cụ thể, lớp d không thể kế thừa đồng thời lớp b và lớp c. Lý do là nếu d muốn sử dụng dữ liệu hay phương thức của lớp a thì Java không biết dữ liệu đó được kế thừa theo hướng từ b xuống đến d hay từ c xuống đến d. Để tránh tình trạng kế thừa đồng thời nhiều lớp, đồng thời vẫn đảm bảo việc sử dụng hiệu quả của tính kế thừa, Java cung cấp khái niệm lớp giao diện (Interface). Cụ thể, trong trường hợp này, một trong hai lớp b hoặc c phải là lớp giao diện. Giả sử lớp c là lớp giao diện thì khi đó lớp d vẫn được kế thừa từ a và b, đồng thời vẫn có những phương thức của lớp c bằng cách cài đặt các phương thức của lớp c.

Ví dụ về sử dụng đa hình thông qua lớp giao diện : Trong ví dụ về mô phỏng động vật ở trên, ta đã biết về vai trò của đa hình thông qua việc truyền tham số Animal cho phương thức AnimalFeed(). Giả sử, bài toán phát sinh thêm tình huống là có 2 động vật

Dog và Cat có thêm phương thức là `playing()`. Khi đó ta có thể nhóm 2 lớp này vào một nhóm bằng cách tạo thêm lớp giao diện `Pet` và cho 2 lớp này kế thừa lớp `Pet`.

Khai báo lớp giao diện `Pet` :

```
interface Pet{  
    void playing();  
}
```

Định nghĩa lớp `feedingAnimal` :

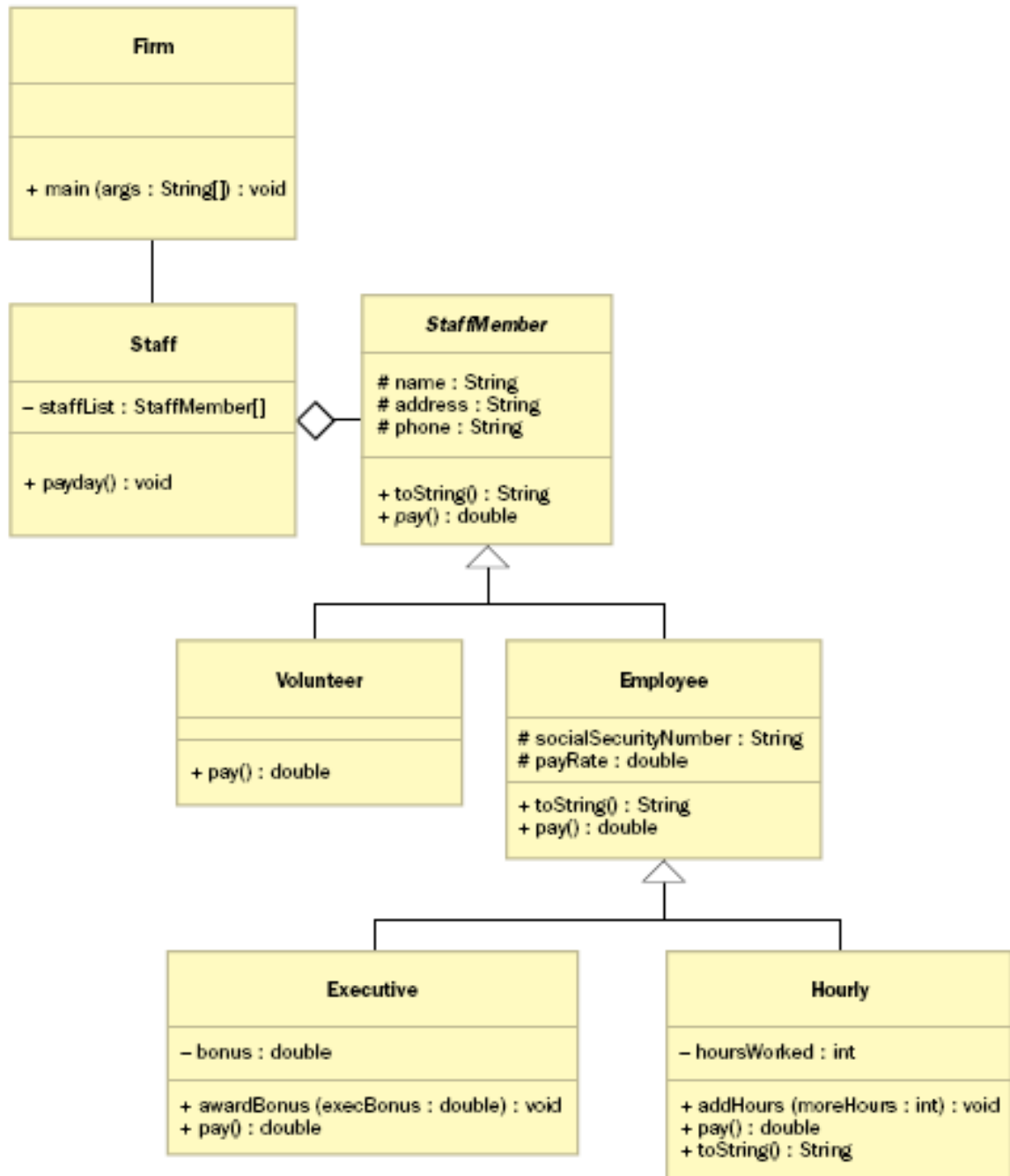
```
class feedingAnimal{  
    void animalFeeding(Animal a){  
        a.makeSound();  
    }  
    void animalPlaying(Pet p){  
        p.playing();  
    }  
}
```

Tại phương thức `main` của lớp `AnimalManager` :

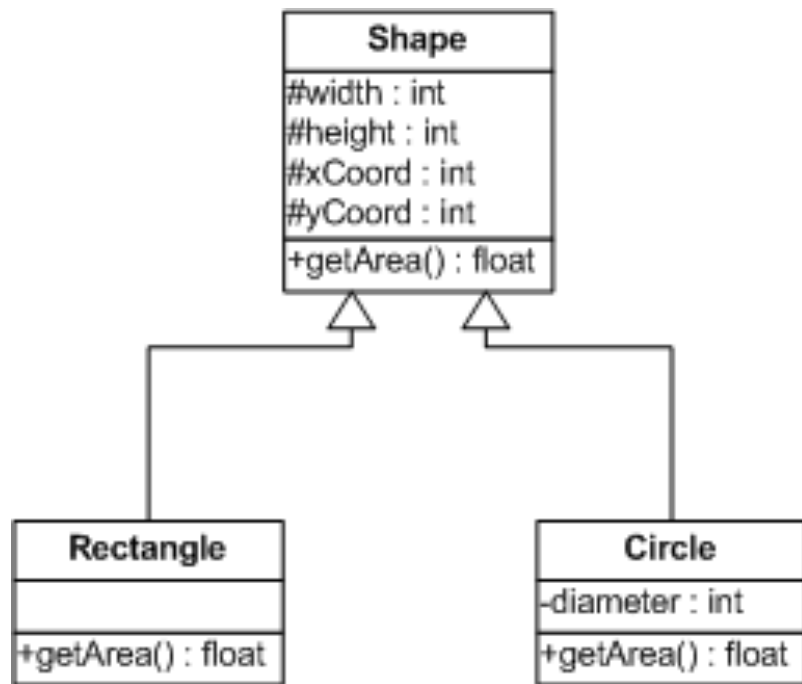
```
Pet[] petAnimal = new Pet[2];  
petAnimal[0] = new Dog();  
petAnimal[1] = new Cat("Tom", 3);  
  
for (Pet animal : petAnimal) {  
    fa.animalPlaying(animal);  
}
```


BÀI TẬP CHƯƠNG 3

Bài 1 : Viết chương trình quản lý lương nhân viên công ty theo biểu đồ lớp sau :



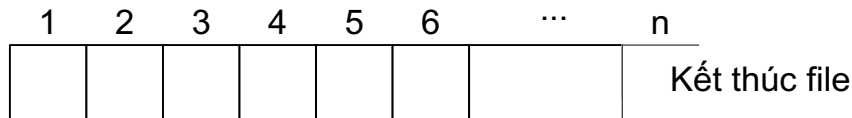
Bài 2: Viết chương trình tính diện tích và chu vi hình chữ nhật. Lớp **Shape** là lớp trừu tượng.



CHƯƠNG 4. XỬ LÝ NHẬP/XUẤT TRONG

4.1. File và luồng dữ liệu

Dữ liệu được lưu trữ trong các biến và mảng là tạm thời, nó bị mất khi một biến cục bộ bị mất phạm vi hoặc khi chương trình kết thúc. Để lưu giữ dữ liệu lâu dài, ngay cả sau khi chương trình kết thúc, máy tính sử dụng tập tin (file). Máy tính lưu trữ file trên các thiết bị lưu trữ thứ cấp như đĩa cứng, USB, đĩa CD,...



Hình 4. 1 Kích thước file n byte

Java xem mỗi tệp như một luồng byte liên tiếp (Hình 4.1). Mỗi hệ điều hành cung cấp một cơ chế để xác định kết thúc của một tệp, chẳng hạn như điểm đánh dấu cuối tệp hoặc số đếm trong tổng số byte trong tệp được ghi lại trong cấu trúc file. Một chương trình Java xử lý một luồng byte chỉ đơn giản là nhận dữ liệu từ hệ điều hành khi đến cuối luồng thì chương trình dừng mà không cần để biết file hoặc luồng được biểu diễn như thế nào.

Các luồng dữ liệu được biểu diễn bằng **chuỗi nhị phân định dạng theo byte (byte based stream) hoặc chuỗi các ký tự (character stream)**. Ví dụ, số 5 nếu được lưu dưới dạng nhị phân sẽ là 0000.0101. Nếu số 5 được lưu dưới dạng ký tự thì nó sẽ là các số nhị phân biểu diễn giá trị mã Unicode dùng để mô tả ký tự 5. Cụ thể, ký tự 5 trong bảng mã Unicode có mã là 53. Vì vậy, chuỗi số nhị phân được lưu vào file sẽ là 0000.0000.0011.0101. Sự khác biệt giữa hai cách lưu số 5 này là trong cách thứ nhất, 5 được hiểu là một số nguyên và có thể đọc ra để tính toán còn trong cách thứ hai, 5 được hiểu là ký tự và được sử dụng trong các chuỗi. Ví dụ “Tom is 5 years old”. Các file sử dụng luồng nhị phân được gọi là **file nhị phân (binary file)**, còn các file sử dụng luồng ký tự được gọi là **file văn bản (text file)**. File văn bản có thể được đọc bởi các chương trình soạn thảo văn bản, trong khi file nhị phân chỉ có thể đọc bởi các chương trình có thể hiểu được cấu trúc file đó.

Chương trình Java mở file bằng cách tạo ra một đối tượng, sau đó đối tượng đó được kết hợp với một luồng byte hoặc luồng ký tự. Chương trình Java xử lý file bằng cách sử dụng các lớp trong gói Java.io. Gói này cung cấp các lớp xử lý luồng dữ liệu như **FileInputStream** (dùng để ghi luồng byte vào một file), **FileOutputStream** (dùng để đọc luồng byte từ một file) và **FileWriter** (dùng để ghi luồng ký tự vào file) và **FileReader** (dùng để đọc luồng ký tự từ file) được kế thừa từ các lớp InputStream, OutputStream, Reader và Writer tương ứng.

Java cũng cung cấp các lớp dùng để xử lý dữ liệu vào/ra là các đối tượng hoặc các dữ liệu cơ bản. Các dữ liệu này về bản chất vẫn được lưu dưới dạng byte hoặc ký tự nhưng đối với người lập trình chúng ta có thể đọc dữ liệu dưới dạng cơ bản int, float,... hoặc String mà không cần quan tâm chúng được chuyển sang dạng byte hoặc dạng ký tự như thế nào. Để xử lý các dữ liệu này, đối tượng của các lớp **ObjectInputStream** và **ObjectOutputStream** được dùng cùng với các lớp luồng byte **FileInputStream** và **FileOutputStream**.

4.2. Lớp File

Đối tượng của lớp `java.io.File` biểu diễn cho một file hoặc một thư mục mà không biểu diễn nội dung của file. Trong chương trình ta dùng một đối tượng của lớp này để thay cho một chuỗi biểu diễn tên file hoặc tên thư mục. Hầu hết các lớp sử dụng tham số là trên file trong hàm khởi tạo như `FileWriter` hoặc `FileInputStream` có thể sử dụng đối tượng `File` để làm đối số.

- Tạo một đối tượng `File` đại diện cho một file :

```
File f = new File("MyCode.txt");
```

- Tạo một thư mục mới :

```
File dir = new File("Code");  
dir.mkdir();
```

- Một số phương thức của lớp `File` :

Phương thức	Mô tả
<code>boolean canread()</code>	Trả về giá trị True nếu file có thể đọc bởi chương trình, giá trị False nếu không đọc được.
<code>boolean canwrite()</code>	Trả về giá trị True nếu file có thể ghi bởi chương trình, giá trị False nếu không ghi được.
<code>boolean exists()</code>	Trả về giá trị True nếu file hoặc thư mục tồn tại, giá trị False nếu file hoặc thư mục không tồn tại.
<code>String getName()</code>	Trả về tên của file hoặc thư mục đã được biểu thị bởi <code>pathname</code> trừu tượng này
<code>String getParent()</code>	Trả về đường dẫn của thư mục chứa file
<code>String getPath()</code>	Trả về đường dẫn của file
<code>boolean isDirectory()</code>	Trả về giá trị True nếu đối tượng là tên của thư mục, giá trị False nếu đối tượng không phải là tên thư mục.
<code>boolean isFile()</code>	Trả về giá trị True nếu đối tượng là tên của một file, giá trị False nếu đối tượng không phải là tên file.
<code>long lastModified()</code>	Trả về thời điểm sửa file lần cuối cùng.
<code>long length()</code>	Trả về số byte dữ liệu của file.
<code>String[] list()</code>	Trả về một mảng các chuỗi chỉ các file và thư mục trong thư mục.

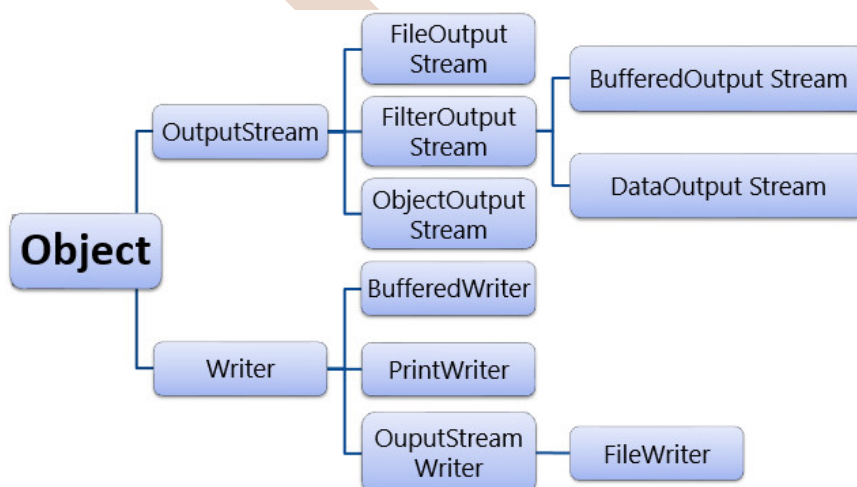
Bảng 4. 1 Một số phương thức của lớp file

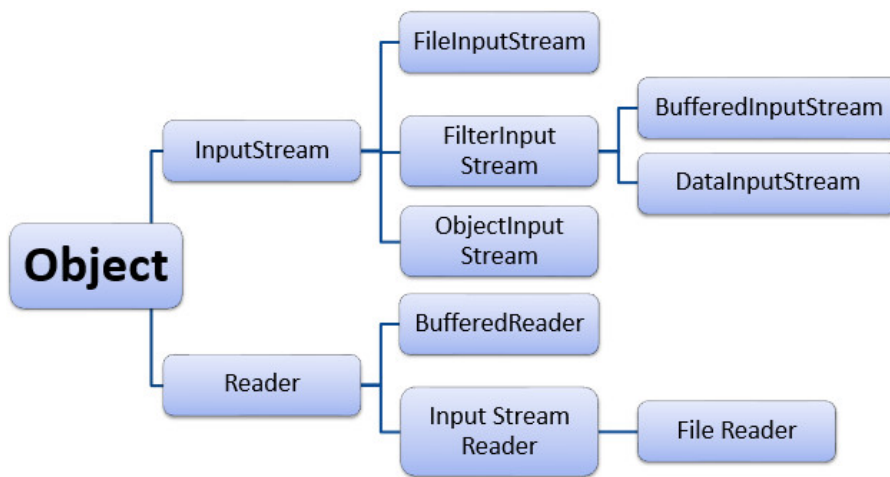
Ví dụ : Yêu cầu người dùng nhập tên file hoặc thư mục và in ra các thông tin của file hoặc thư mục đó.

```
public class FileProcess1 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Nhap ten file: ");
        String fn = s.nextLine();
        fileAnalyzer(fn);
    }

    public static void fileAnalyzer(String filename){
        File f = new File(filename);
        if(f.isFile()){
            System.out.println("Ten file: " + f.getName());
            System.out.println("Do dai cua file: " + f.length());
            System.out.println("Last modified: " + f.lastModified());
        }
        else if(f.isDirectory()){
            System.out.println("Danh sach file trong thu muc:");
            String[] filelist = f.list();
            for(String fl: filelist){
                System.out.println(fl);
            }
        }
        else
            System.out.println("Ten file khong ton tai");
    }
}
```

4.3. Kiến trúc luồng xuất dữ liệu ra file

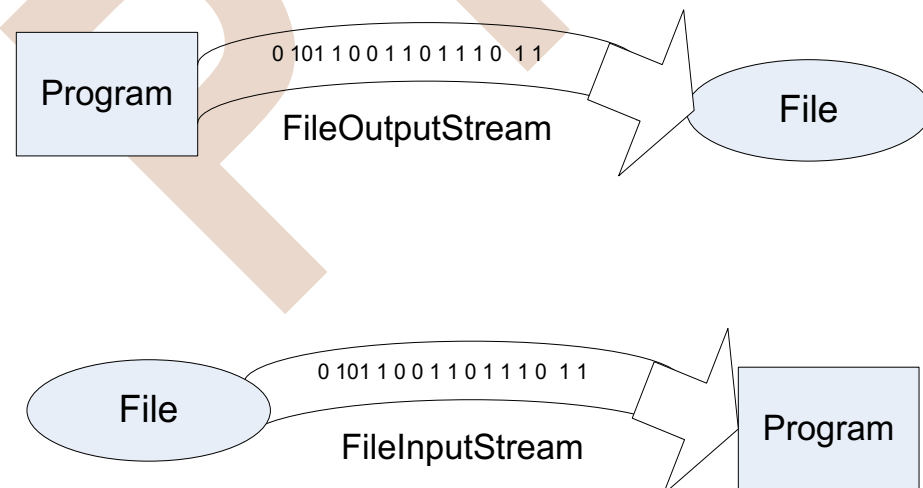




Hình 4. 2 Cấu trúc cây gia phả của lớp xử lý vào/ra dữ liệu

Hình trên mô tả cấu trúc cây kế thừa của các lớp xử lý vào/ra dữ liệu. Các lớp dùng để xử lý dữ liệu luồng byte thuộc lớp cha là `InputStream` (đọc dữ liệu từ file) và `OutputStream` (ghi dữ liệu ra file). Các lớp dùng xử lý dữ liệu luồng văn bản thuộc lớp cha là `Reader` (đọc dữ liệu từ file văn bản) và `Writer` (ghi dữ liệu ra file văn bản). Các lớp này chỉ cung cấp các phương thức cho phép đọc/ghi dữ liệu dạng ký tự còn các lớp con của nó cung cấp các phương thức cho phép đọc/ghi tốc độ cao hơn.

4.3.1. Dữ liệu dạng byte



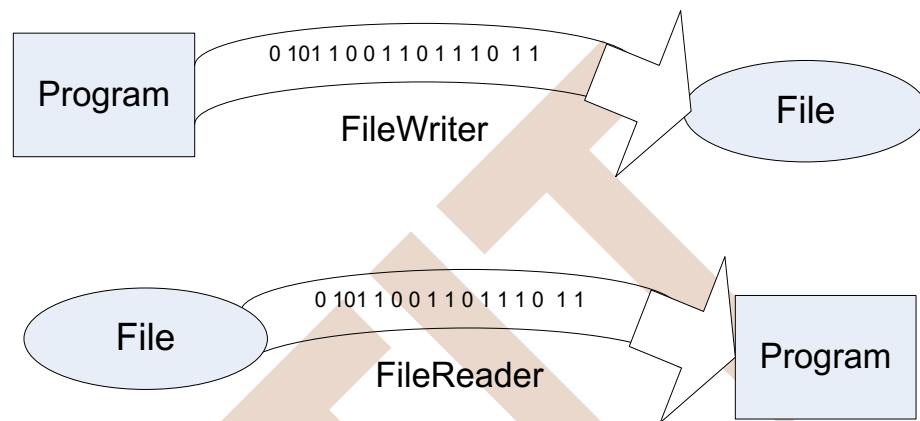
Hình 4. 3 Trao đổi dữ liệu dạng byte

Để trao đổi dữ liệu với một file ta phải tạo một luồng kết nối giữa chương trình và file đó bằng cách sử dụng lớp `FileOutputStream/FileInputStream`. Ta có thể luồng kết nối này giống như một “con đường” để vận chuyển dữ liệu. Sau khi có “con đường”,

ta phải sử dụng tới các “phương tiện” để vận chuyển dữ liệu. Tùy theo loại dữ liệu khác nhau mà ta phải sử dụng đến “phương tiện” khác nhau. Cụ thể :

- Nếu dữ liệu là luồng byte chứa các dữ liệu cơ bản thì ta dùng các lớp `DataInputStream/DataOutputStream`.
- Nếu dữ liệu là luồng byte chứa các đối tượng đã được chuỗi hóa thì ta dùng các lớp `ObjectInputStream/ObjectOutputStream`.

4.3.2. Dữ liệu dạng văn bản



Hình 4. 4 Trao đổi dữ liệu dạng văn bản

Tương tự như dữ liệu dạng byte, để tạo “con đường” kết nối giữa chương trình và file lưu dữ liệu ta dùng lớp `FileWriter/FileReader`.

- Nếu ghi/đọc dữ liệu từng ký tự rời rạc trong văn bản thì ta có thể sử dụng trực tiếp các phương thức `write/read` của lớp này.
- Nếu ghi/đọc từng dòng trong văn bản thì ta dùng thêm lớp `BufferedWriter/BufferedReader`

4.4. Ghi/đọc chuỗi ký tự ra tệp văn bản

Quá trình ghi/đọc luồng ký tự hay luồng byte ra/vào tệp đều tuân theo 3 bước sau :

- Bước 1: Tạo đối tượng luồng và liên kết với nguồn dữ liệu là file chứa dữ liệu.
- Bước 2 : Đọc hoặc ghi dữ liệu
- Bước 3 : Đóng luồng.

Sử dụng lớp **`FileWriter`** để ghi dữ liệu vào file văn bản như sau :

```

public class FileProcess1 {
    public static void main(String[] args) {
        try {
            //Bước 1: Tạo đối tượng luồng và liên kết nguồn dữ liệu
            File f = new File("datafile.txt");
            FileWriter fw = new FileWriter(f);
            //Bước 2: Ghi dữ liệu
            fw.write("PTIT - Multimedia");
            //Bước 3: Đóng luồng
            fw.close();
        } catch (IOException ex) {
            System.out.println("Loi ghi file: " + ex);
        }
    }
}

```

Sử dụng lớp **FileReader** để đọc dữ liệu từ file văn bản :

Đọc từng ký tự :

```

public class FileProcess1 {
    public static void main(String[] args) {
        try {
            //Bước 1: Tạo đối tượng luồng và liên kết nguồn dữ liệu
            File f = new File("datafile.txt");
            FileReader fr = new FileReader(f);
            //Bước 2: Đọc dữ liệu
            char [] a = new char[50];
            fr.read(a); // doc noi dung toi mang
            for(char c : a)
                System.out.print(c); //in tung ky tu mot
            //Bước 3: Đóng luồng
            fr.close();
        } catch (Exception ex) {
            System.out.println("Loi doc file: "+ex);
        }
    }
}

```

Đọc từng dòng :


```

public class FileProcess1 {
    public static void main(String[] args) {
        try {
            //Bước 1: Tạo đối tượng luồng và liên kết nguồn dữ liệu
            File f = new File("datafile.txt");
            FileReader fr = new FileReader(f);
            //Bước 2: Đọc dữ liệu
            BufferedReader br = new BufferedReader(fr);
            String line;
            while ((line = br.readLine()) != null){
                System.out.println(line);
            }
            //Bước 3: Đóng luồng
            fr.close();
            br.close();
        } catch (Exception ex) {
            System.out.println("Loi doc file: "+ex);
        }
    }
}

```

4.5. Đọc/ghi dữ liệu luồng byte

4.5.1. Đọc/ghi dữ liệu dạng cơ bản

```

public class FileProcess1 {
    public static void main(String[] args) {
        try{
            //Tạo nguồn và kết nối tới nguồn
            File f = new File("datafile.dat");
            FileOutputStream fos = new FileOutputStream(f);
            DataOutputStream dos = new DataOutputStream(fos);
            //Ghi dữ liệu
            dos.write(100);
            dos.writeDouble(3.4);
            //Đóng luồng
            dos.close();
            fos.close();
        }
        catch(IOException e){
            System.out.println("Loi ghi file");
        }
    }
}

```

```

public class FileProcess1 {
    public static void main(String[] args) {
        try{
            File f = new File("datafile.dat");
            FileInputStream fis = new FileInputStream(f);
            DataInputStream dis = new DataInputStream(fis);
            int n = dis.read();
            double d = dis.readDouble();
            dis.close();
            fis.close();
            System.out.println("ket qua: " + n + d);
        }
        catch(IOException e){
            System.out.println("Loi ghi file");
        }
    }
}

```

4.5.2. Đọc/ghi dữ liệu là các đối tượng

Các đối tượng có trạng thái và hành vi. Hành vi tồn tại trong các lớp, còn trạng thái tồn tại trong mỗi đối tượng cụ thể. Trong nhiều trường hợp chúng ta cần phải lưu lại

trạng thái của một đối tượng để đến một lúc nào đó ta khôi phục lại. Ví dụ khi ta đang chơi game, ta có thể lưu lại trạng thái của các nhân vật. Sau đó, khi ta quay lại chơi tiếp game đó thì các trạng thái của nhân vật được lấy lại như lúc trước khi lưu. Trong Java cung cấp hai cách để lưu các đối tượng:

- Cách thứ nhất là chúng ta sẽ lưu giá trị của các trạng thái vào một file theo định dạng quy định. Khi khôi phục lại trạng thái đối tượng, ta sẽ đọc ra các giá trị đó và gán tương ứng vào các biến của đối tượng. Với cách này, ta sẽ dùng một file dạng text với cú pháp được quy định để lưu các giá trị trạng thái và như vậy các chương trình khác ngoài Java cũng có thể đọc được các giá trị của trạng thái. Ví dụ :

GameFile.txt

```
60,Elf,bow,sword,dust
200,Troll,bare hands,big ax
120,Magiclan,spells,invisibility
```

Với cách này, khi đọc dữ liệu dễ mắc phải lỗi đọc nhầm giữa các trường hoặc các dòng. Khi đó, chương trình dễ bị lỗi hoặc trạng thái của đối tượng không được khôi phục lại như ban đầu. Vì vậy, cách này ít được dùng để ghi/đọc trạng thái của đối tượng.

- Cách thứ hai là chúng ta “nén” đối tượng đó lại và “giải nén” đối tượng khi cần sử dụng trở lại. Với cách này, các chương trình khác ngoài Java khó có thể đọc được nội dung của file. Cách này được gọi là chuỗi hóa (serialization) đối tượng. Ví dụ :

```
GameCharacter:50LAAQ *I *healthI
/weaponCountI *typet :Ljava/lang/
String:[] weaponst [LWeapon;xp 2 *t
*Zombieur [LWeapon;2AFúCÁ * xp sr
*WeaponC *p *gn *L *n *n *q * *xpt
*teethsq * t *handsp *pp *pp *ppsq * 2
*t PeaShooteruq * * MagicMushroomuq * *
* 2 *t sq * t *poisonsq * t
*lightpp *pp *pp *pp
```

Tuy nhiên không phải đối tượng nào cũng có thể chuỗi hóa được. Để đối tượng thuộc một lớp nào đó có thể chuỗi hóa được, ta phải cho lớp đó triển khai lớp giao diện Serializable. Lớp Serializable không có phương thức nào để cài đặt. Mục đích của lớp này là để khai báo rằng lớp triển khai nó có thể chuỗi hóa được. Nếu một lớp chuỗi hóa được thì các lớp con của nó đều tự động chuỗi hóa được mà không cần phải khai báo lại.

Ví dụ : ghi đối tượng ra file

```

import java.io.Serializable;
public class Student implements Serializable{
    String name;
    int age;
    Student(String n,int a){
        name = n;
        age = a;
    }
    public String toString(){
        return ("Ten: " + name + ";" + "Tuoi: " + age);
    }
}

```

```

public class FileProcess1 {
    public static void main(String[] args) {
        try{
            File f = new File("student");
            FileOutputStream fos = new FileOutputStream(f);
            ObjectOutputStream oop = new ObjectOutputStream(fos);
            Student[] s = {new Student("A",20),new Student("B",30)};
            oop.writeObject(s);
            oop.close();
            fos.close();
        }
        catch(IOException e){
            System.out.println("Loi ghi file");
        }
    }
}

```

Đọc đối tượng từ file :

```
public class FileProcess1 {  
    public static void main(String[] args) {  
        try{  
            File f = new File("student");  
            FileInputStream fis = new FileInputStream(f);  
            ObjectInputStream ois = new ObjectInputStream(fis);  
            Student[] std = (Student[])ois.readObject();  
            for(Student st: std){  
                System.out.println(st.toString());  
            }  
            ois.close();  
            fis.close();  
        }  
        catch(Exception e){  
            System.out.println("Loi ghi file");  
        }  
    }  
}
```

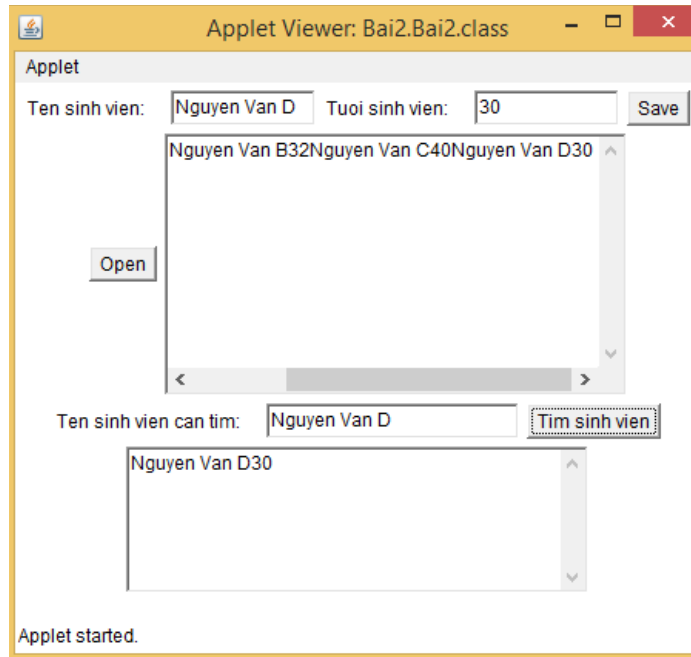


BÀI TẬP CHƯƠNG 4

Bài 1: Viết giao diện cho phép người dùng nhập họ tên, mã sv, tuổi sinh, lớp sinh viên.

- Ghi danh sách sv ra file **sinhvien.dat**
- Cho phép người dùng tìm theo tên của sinh viên

Bài 2: Viết một ứng dụng quản lý sinh viên có giao diện như sau:



Ứng dụng có các chức năng:

- Cho phép người dùng nhập tên và tuổi sinh viên. Sau đó lưu ra file “**sinhvien.txt**” khi người dùng chọn nút “Save”.
- Người dùng có thể xem danh sách sinh viên vừa nhập bằng cách chọn nút “Open”.
- Người dùng có thể tìm sinh viên bằng cách nhập tên sinh viên. Chương trình in ra tên và tuổi của sinh viên được tìm thấy.

CHƯƠNG 5. XỬ LÝ NGOẠI LỆ TRONG JAVA

5.1. Xử lý ngoại lệ

Ngoại lệ (exception) là trường hợp một sự cố bất thường xảy ra trong khi chương trình đang chạy. Ví dụ, ta có thể gặp tình huống chia cho 0, không tìm thấy file dữ liệu hoặc truy cập tới phần tử vượt quá giới hạn của mảng. Nếu người lập trình không lường hết các tình huống này và không viết các đoạn mã để chương trình xử lý khi gặp các lỗi này thì chương trình sẽ dừng đột ngột. Thông thường, để xử lý các tình huống này, người lập trình viết các lệnh rẽ nhánh để xử lý. Tuy nhiên, người lập trình không thể bao quát hết các tình huống xảy ra và việc viết thêm các lệnh rẽ nhánh như vậy sẽ làm chương trình trở nên phức tạp và khó kiểm soát.

Ví dụ : Viết chương trình cho người dùng nhập vào tử số, mẫu số và in ra kết quả của phân số đó.

```
public static void main(String[] args) {  
    int n,d;  
    Scanner s = new Scanner(System.in);  
    System.out.println("Nhập tử số:");  
    n = s.nextInt();  
    System.out.println("Nhập mẫu số:");  
    d = s.nextInt();  
    s.nextLine();  
    int f = n/d;  
    System.out.println("Kết quả là: " + f);  
}  
}
```

Chương trình trên được viết hoàn toàn đúng về cú pháp. Tuy nhiên, lỗi xảy ra khi người dùng nhập mẫu số bằng 0 :

```
Nhập tử số:  
4  
Nhập mẫu số:  
0  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at chuong4_vidul.Chuong4_Vidul.main(Chuong4_Vidul.java:30)  
Java Result: 1
```

Khi lỗi trên xảy ra, chương trình sẽ dừng đột ngột và người dùng không có cơ hội để sửa sai. Để giải quyết vấn đề trên ta có thể dùng lệnh rẽ nhánh để xử lý như sau :

```

public static void main(String[] args) {
    int n,d;
    boolean loop = false;
    Scanner s = new Scanner(System.in);
    while(!loop == true){
        System.out.println("Nhap tu so:");
        n = s.nextInt();
        System.out.println("Nhap mau so:");
        d = s.nextInt();
        if(d==0){
            System.out.println("Mau phai khac 0!");
            loop=false;
        }else{
            int f = n/d;
            System.out.println("Ket qua là: " + f);
            break;
        }
    }
}

```

Tuy nhiên, giả sử trong bài toán trên phát sinh tình huống ta phải in ra kết quả của nhiều phân số trong đó mỗi phân số lại có mẫu số là một biểu thức khác nhau chứa giá trị của d . Như vậy ta sẽ phải viết từng đó khối lệnh if – else như trên để tránh trường hợp mẫu số bằng 0.

Để giải quyết vấn đề trên, Java hỗ trợ người lập trình bằng cách cho phép người lập trình bắt một lỗi chung gọi “lỗi chia cho 0” để xử lý tất cả các tình huống trên thay vì phải xét từng trường hợp. Cụ thể là bất cứ khi nào xảy ra tình huống phân số bằng 0 thì Java sẽ tạo ra một đối tượng “*lỗi ngoại lệ chia cho 0*”. Đối tượng này sẽ được truyền xuống một phương thức để xử lý lỗi này. Quá trình tạo ra đối tượng lỗi và xử lý đối tượng đó gọi là xử lý ngoại lệ (Exception handling).

Để xử lý ngoại lệ có thể được tạo ra trong một đoạn mã, ta đưa đoạn mã đó vào trong khối try {}. Khi có đối tượng lỗi xuất hiện, đối tượng lỗi đó sẽ được truyền xuống khối catch {} để xử lý.

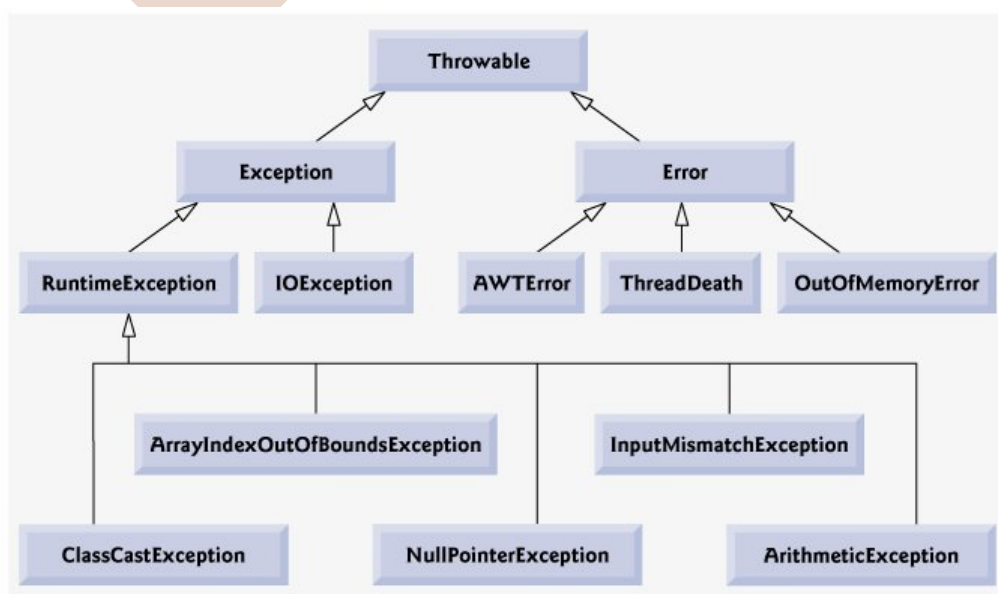
Ví dụ chương trình ở trên được viết lại như sau :


```

public static void main(String[] args) {
    int n,d;
    float f=0;
    boolean loop = false;
    Scanner s = new Scanner(System.in);
    while(!loop == true){
        try{
            loop=true;
            System.out.println("Nhap tu so:");
            n = s.nextInt();
            System.out.println("Nhap mau so:");
            d = s.nextInt();
            f = n/d;
        }
        catch(ArithmeticException e){
            System.out.println("Mau phai khac 0!");
            loop=false;
        }
    }
    System.out.println("Ket qua là: " + f);
}

```

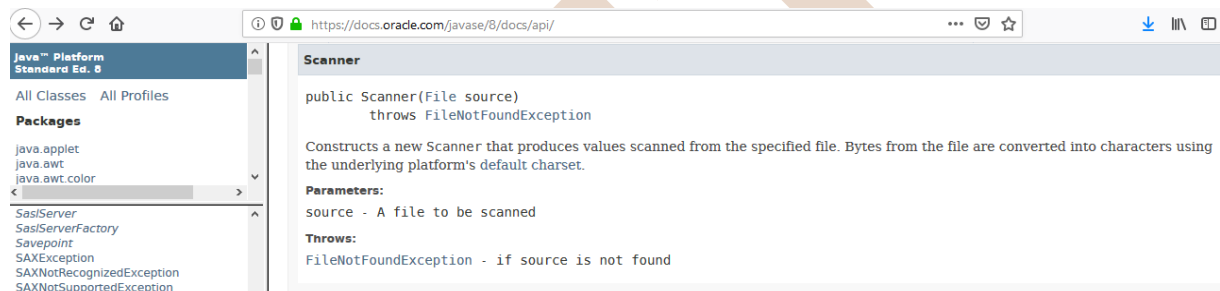
Khối try/catch gồm khối try chứa đoạn mã có thể phát sinh ngoại lệ và ngay sau đó là khối catch có nhiệm vụ “bắt” ngoại lệ được ném ra từ khối try và xử lý ngoại lệ đó. Cụ thể trong chương trình trên, khi gặp phép chia cho 0 thì chương trình sẽ ném ra đối tượng ngoại lệ và đối tượng này được truyền xuống khối catch để xử lý. Trong Java, mỗi đối tượng ngoại lệ là thực thể của một lớp ngoại lệ nào đó và lớp ngoại lệ này được kế thừa từ một lớp ngoại lệ là lớp Exception. Cây kế thừa của các lớp ngoại lệ như sau :



Hình 5. 1 Cây kế thừa của lớp ngoại lệ

Khối catch trong ví dụ trên có tham số e là tham chiếu được khai báo thuộc kiểu ArithmeticException. Mỗi khối catch khai báo tham số thuộc kiểu ngoại lệ nào thì sẽ bắt được đối tượng kiểu ngoại lệ đó. Tuy nhiên, theo nguyên tắc kế thừa và đa hình thì khối catch nếu khai báo tham số kiểu của lớp cha thì cũng có thể bắt được các đối tượng của lớp con. Ví dụ, nếu khai báo catch(Exception e) thì cũng có thể bắt được các đối tượng ngoại lệ kiểu ArithmeticException, ArrayIndexOutOfBoundsException,...

Vậy làm sao để biết một phương thức có thể ném ngoại lệ hay không và ngoại lệ nào nó có thể ném? Có hai cách để xử lý việc này. Cách thứ nhất là với bất kỳ phương thức nào ta cũng để vào khối try và khối catch(Exception e){}. Với cách này ta bắt được tất cả các lỗi ngoại lệ vì các lỗi này đều kế thừa từ Exception. Tuy nhiên ta không biết cụ thể lỗi gì để ta có phương án xử lý chuyên biệt cho loại lỗi đó. Cách thứ hai là tra đặc tả phương thức đó trong tài liệu API của Java đặt tại trang web của Oracle. Ví dụ hình sau là đặc tả của hàm Scanner(File). Đặc tả nói rằng hàm này có thể ném ra lỗi FileNotFoundException. Vì vậy, khi ta sử dụng hàm này, ta phải bắt lỗi catch(FileNotFoundException e){}.



Một số ngoại lệ thường gặp :

Exception	Ý nghĩa
RuntimeException	Lớp xử lý lỗi cho các lỗi của gói java.lang
ArithmeticException	Lỗi số học, ví dụ “divide by zero”
IllegalAccessException	Không truy cập được lớp
IllegalArgumentException	Tham số truyền vào phương thức bị sai
ArrayIndexOutOfBounds	Chỉ số của mảng nhỏ hơn 0 hoặc lớn hơn kích thước
NullPointerException	Truy cập một đối tượng “null”
SecurityException	Lỗi bảo mật
ClassNotFoundException	Không gọi được lớp
NumberFormatException	Lỗi khi chuyển từ string sang kiểu số
AWTException	Lỗi khi sử dụng thư viện AWT

IOException	Lớp xử lý lỗi vào ra
FileNotFoundException	Không tìm thấy file
EOFException	Lỗi khi đóng file
NoSuchMethodException	Lỗi khi gọi phương thức không tồn tại
InterruptedException	Lỗi khi luồng bị ngắt (interrupted thread)

Bảng 5. 1 Một số lớp ngoại lệ thường gặp

5.2. Khối try/catch

5.2.1. Hoạt động của khối try/catch

Khi ta chạy một đoạn mã chứa lệnh hoặc phương thức, một trong các trường hợp có thể xảy ra : (1) đoạn mã sẽ chạy thành công ; (2) đoạn mã sẽ ném ra ngoại lệ và được khối catch bắt xử lý ; (3) đoạn mã ném ra ngoại lệ nhưng không được khối catch bắt để xử lý.

- (1) Nếu đoạn mã chạy thành công, khối try được thực hiện đầy đủ cho đến lệnh cuối cùng, còn khối catch sẽ được bỏ qua. Sau đó, các lệnh phía sau khối catch sẽ được thực hiện.

```
public static void main(String[] args) {
    int n,d;
    int f=0;
    Scanner s = new Scanner(System.in);
    try{
        System.out.println("Nhap tu so:");
        n = s.nextInt();
        System.out.println("Nhap mau so:");
        d = s.nextInt();
        f = n/d;
    }
    catch(ArithmeticException e){
        System.out.println("Mau phai khac 0!");
    }
    System.out.println("Ket qua là: " + f);
}
```

Kết quả :

```
Nhap tu so:
4
Nhap mau so:
3
Ket qua là: 1
```

- (2) Đoạn mã ném ngoại lệ và khối catch bắt được ngoại lệ đó để xử lý. Khi đó các lệnh trong khối try ở sau lệnh phát sinh ngoại lệ bị bỏ qua, chương trình thực hiện các lệnh trong khối catch. Sau đó, các lệnh sau khối catch được thực hiện. Vẫn ví dụ chương trình trên nhưng nếu ta nhập mẫu số bằng 0, khối catch sẽ được thực hiện và sau đó là lệnh in ra kết quả :

```
Nhap tu so:
4
Nhap mau so:
0
Mau phai khac 0!
Ket qua la: 0
```

- (3) Đoạn mã ném ngoại lệ nhưng khối catch không bắt được ngoại lệ đó, chương trình sẽ ra khỏi khối try và báo lỗi.

```
Nhap tu so:
a
Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Scanner.java:864)
    at java.util.Scanner.next(Scanner.java:1485)
    at java.util.Scanner.nextInt(Scanner.java:2117)
    at java.util.Scanner.nextInt(Scanner.java:2076)
    at chuong4_vidul.Chuong4_Vidul.main(Chuong4_Vidul.java:28)
Java Result: 1
```

Để tránh tình trạng lỗi xảy ra không được xử lý ở tình huống (3), khối finally được sử dụng ở phần cuối cùng của try/catch. Khối finally là nơi ta đặt đoạn mã sẽ được thực thi bất kể ngoại lệ có xảy ra hay không.

```
public static void main(String[] args) {
    int n,d;
    int f=0;
    Scanner s = new Scanner(System.in);
    try{
        System.out.println("Nhap tu so:");
        n = s.nextInt();
        System.out.println("Nhap mau so:");
        d = s.nextInt();
        f = n/d;
        System.out.println("Ket qua la: " + f);
    }
    catch(ArithmeticException e){
        System.out.println("Mau phai khac 0!");
    }
    finally{
        System.out.println("Chương trình có lỗi");
    }
}
```

Kết quả :

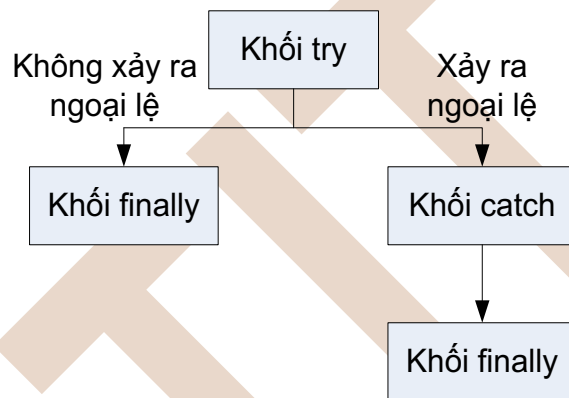
Nhap tu so:

a

Chương trình có lỗi

```
Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Scanner.java:864)
    at java.util.Scanner.next(Scanner.java:1485)
    at java.util.Scanner.nextInt(Scanner.java:2117)
    at java.util.Scanner.nextInt(Scanner.java:2076)
    at chuong4_vidu1.Chuong4_Vidu1.main(Chuong4_Vidu1.java:28)
Java Result: 1
```

Luồng thực hiện của các khối try, catch và finally như sau :



Bảng 5. 2 Luồng xử lý try/catch/finally

5.2.2. Xử lý nhiều ngoại lệ

Trong trường hợp khối try có thể xảy ra nhiều ngoại lệ, ta có thể dùng nhiều khối catch để bắt và xử lý. Ngoài ra, vì các ngoại lệ có tính kế thừa nên nó cũng có tính đa hình. Tức là khối catch dành cho ngoại lệ lớp cha cũng bắt được ngoại lệ lớp con. Ví dụ, theo như cây kế thừa ở trên, khối `catch(Exception e){...}` có thể bắt được ngoại lệ `RuntimeException`, `ArithmeticException`.

Vậy các khối catch nên được đặt theo thứ tự như thế nào? Khi một ngoại lệ được ném ra từ bên trong khối try, theo thứ tự từ trên xuống, khối catch nào bắt được ngoại lệ đó đầu tiên thì sẽ được chạy. Do đó ta nên để **khối catch của lớp ngoại lệ cha đứng sau khối catch của lớp ngoại lệ con**. Ví dụ, ta có ba khối catch với ba ngoại lệ `Exception`, `RuntimeException` và `ArithmeticException`, thứ tự đặt sẽ như sau :

```

try{
    //Do something
}
catch(ArithmeticException e){
    System.out.println("Lỗi số học");
}
catch(RuntimeException e){
    System.out.println("Lỗi phát sinh khi chương trình chạy");
}
catch(Exception e){
    System.out.println("Lỗi tổng quát, không xác định");
}

```

5.3. Ném ngoại lệ

Trong phần trên, chúng ta đều sử dụng các API đã được viết sẵn để ném ngoại lệ khi sự cố xảy ra. Java cũng cho phép chúng ta viết các phương thức có thể tự ném ngoại lệ. Nếu viết một phương thức có thể ném một ngoại lệ, ta phải thực hiện hai việc : (1) dùng từ khóa **throws** để tuyên bố phương thức có thể ném loại lệ tại dòng khai báo phương thức ; (2) dùng từ khóa **throw** tại tình huống có thể ném ngoại lệ trong nội dung của phương thức.

Ví dụ :

```

public static int fraction(int n, int d) throws ArithmeticException{
    if(d==0) throw new ArithmeticException();
    return n/d;
}

```

5.4. Lan truyền lỗi ngoại lệ

Giả sử phương thức Y() gọi một phương thức X():

```

public void Y(){
    X();
}

```

Nếu phương thức X() sử dụng try-catch thì nếu lỗi xảy ra sẽ được xử lý ngay tại trong X(). Nếu trong X() không sử dụng try-catch thì nếu lỗi xảy ra sẽ được truyền lên Y để xử lý.

```

public void Y(){
    try{
        X();
    }
    catch(Exception e){
    }
}

```

Trong trường hợp ta chưa muốn xử lý một ngoại lệ tại một phương thức, ta có thể né tránh việc xử lý này bằng cách khai báo **throws** cho ngoại lệ đó khi viết định nghĩa phương thức.

Ví dụ :

```
public static int arrayReading(int[] a, int index) throws
    ArrayIndexOutOfBoundsException{
    return a[index];
}
```

Khi đó, phương thức arrayReading không trực tiếp xử lý ngoại lệ ArrayIndexOutOfBoundsException mà phương thức nào gọi phương thức arrayReading sẽ phải xử lý ngoại lệ ArrayIndexOutOfBoundsException. Như vậy, bản chất của việc sử dụng throws là trì hoãn việc xử lý ngoại lệ và đẩy việc xử lý ngoại lệ lên trên. Hàm cuối cùng phải xử lý ngoại lệ sẽ là hàm main.

5.5. Ngoại lệ được kiểm tra và không được kiểm tra

Lỗi ngoại lệ có thể được tạo ra bởi người dùng, người lập trình hoặc bởi nguồn tài nguyên vật lý. Vì vậy, lỗi ngoại lệ được chia ra làm 3 loại:

- Checked exception: Xảy ra tại thời điểm biên dịch. Vì vậy nó còn có tên gọi là compile time exception. Đối với lỗi này, người lập trình cần phải xử lý “try – catch” ngay từ lúc viết chương trình.

Ví dụ:

```
import java.io.File;
import java.io.FileReader;
public class FileNotFound_Demo {
    public static void main(String args[]) {
        File file = new File("E://file.txt");
        FileReader fr = new FileReader(file);
    }
}
```

```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
    Unhandled exception type FileNotFoundException

    at Vidu1.FileNotFound.main(FileNotFound.java:7)
```

- Unchecked exception: Lỗi xảy ra trong quá trình chạy chương trình. Lỗi này còn có tên là RuntimeException. Lỗi này được trình biên dịch bỏ qua khi biên dịch nhưng khi thực thi chương trình sẽ báo lỗi.

Ví dụ:

```
int[] a = {1,2,3,4,5};
System.out.println(a[5]);
```

- Error: Đây là những lỗi vật lý xảy ra trong quá trình chạy chương trình nên người lập trình không phải quan tâm.

5.6. Lỗi do người dùng định nghĩa

Ngoài những ngoại lệ đã được định nghĩa trong Java, người dùng có thể tự đưa ra các trường hợp được coi là ngoại lệ của riêng mình. Một ngoại lệ mới phải là một lớp kế thừa từ lớp ngoại lệ để lớp ngoại lệ mới có thể sử dụng những có chế xử lý ngoại lệ sẵn có của Java.

Ví dụ:

```
package xulyloi;
import java.util.Scanner;

public class Xulyloi {
    public static void main(String[] args){
        Xulyloi x = new Xulyloi();
        try{
            x.inputNumber();
        }
        catch(InputNumLessThanFive e){
            e.printStackTrace();
        }
    }
    void inputNumber() throws InputNumLessThanFive{
        Scanner s = new Scanner(System.in);
        System.out.println("Nhap vao mot so nguyen nho hon
5:");
        int n = s.nextInt();
        if(n<5){
            throw new InputNumLessThanFive("So nhap vao " +
n + "<5");
        }
        else{
            System.out.println("So nhap vao la:" + n);
        }
    }
}
class InputNumLessThanFive extends Exception{
    InputNumLessThanFive(String str){
        System.out.println(str);
    }
}
```


BÀI TẬP CHƯƠNG 5

Bài 1: Nhập vào một mảng N phần tử. In ra một phần tử bất kỳ của mảng đó. Dùng try/catch để bắt lỗi khi người dùng nhập vào sai chỉ số phần tử của mảng.

Bài 2: Tạo một lớp SoAmException để bắt lỗi khi người dùng nhập vào số âm.

Bài 3: Tạo một lớp SoNhoHon100Exception để bắt lỗi khi người dùng nhập vào số nhỏ hơn 100.

Bài 4 : Chương trình cho người dùng nhập vào một số nguyên. Nếu giá trị nhỏ hơn 5 thì sẽ coi như là một ngoại lệ.

PDF

CHƯƠNG 6. LẬP TRÌNH ĐA LUỒNG

6.1. Đa luồng (Multithreading)

Thread (luồng) về cơ bản là một tiến trình con (sub-process) và là một đơn vị xử lý nhỏ nhất của máy tính có thể thực hiện một công việc riêng biệt. Trong Java, các luồng được quản lý bởi máy ảo Java (JVM).

Multi-thread (đa luồng) là một tiến trình thực hiện nhiều luồng đồng thời. Một ứng dụng Java ngoài luồng chính có thể có các luồng khác thực thi đồng thời làm ứng dụng chạy nhanh và hiệu quả hơn. Ví dụ khi play nhạc, chúng ta vẫn có thể tương tác được với nút điều khiển như: Play, pause, next, back ... vì luồng phát nhạc là luồng riêng biệt với luồng tiếp nhận tương tác của người dùng.

Khi một chương trình Java chạy, một luồng chính sẽ được xử lý. Luồng chính có 2 đặc điểm sau:

- Các luồng con có thể được tạo ra từ luồng chính;
- Luồng chính là luồng kết thúc cuối cùng. Thời điểm luồng chính dừng (stop) thì chương trình sẽ kết thúc.

Tất cả các chương trình được đề cập trong các chương trước chỉ có một luồng thực hiện duy nhất. Mỗi chương trình được tiến hành tuần tự, hết lệnh này đến lệnh khác, cho đến khi hoàn thành quá trình xử lý và kết thúc. Các chương trình đa luồng tương tự như các chương trình đơn luồng đã được đề cập. Chúng chỉ khác nhau ở chỗ chúng hỗ trợ nhiều hơn một luồng được thực thi đồng thời - nghĩa là chúng có thể thực hiện đồng thời nhiều chuỗi lệnh. Mỗi chuỗi lệnh có luồng điều khiển riêng độc lập với tất cả các luồng khác. Các chuỗi lệnh được thực hiện độc lập này được gọi là các luồng.

Nếu máy tính của chúng ta chỉ có một CPU duy nhất (single processor) thì chỉ một luồng thực thi duy nhất xảy ra tại một thời điểm nhất định. CPU nhanh chóng chuyển đổi qua lại giữa một số luồng để tạo cảm giác rằng các luồng đang thực thi cùng một lúc. Các hệ thống đơn CPU hỗ trợ xử lý song song về mặt logic (logical concurrency) chứ không phải về mặt vật lý (physical concurrency). Song song logic là đặc tính được thể hiện khi nhiều luồng thực thi đồng thời và mỗi luồng được điều khiển độc lập, riêng biệt. Trên các hệ thống đa bộ xử lý (multi processor), trên thực tế, một số luồng thực hiện cùng lúc và thực hiện đồng thời về mặt vật lý. Tính năng quan trọng của các chương trình đa luồng là chúng hỗ trợ đồng thời về logic, chứ không phải đồng thời về vật lý.

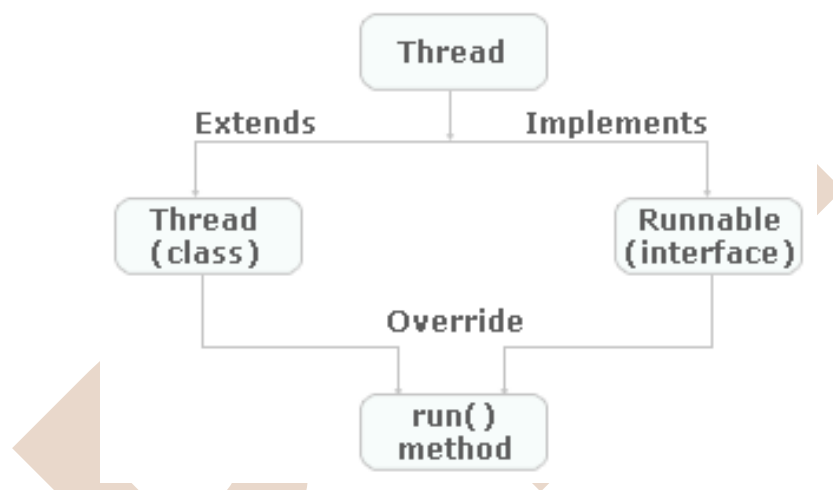
6.2. Các cách tạo luồng

Lập trình đa luồng trong Java được tập trung xung quanh lớp `java.lang.Thread`. Lớp này cung cấp khả năng tạo các đối tượng của lớp `Thread`, mỗi đối tượng có luồng điều khiển riêng.

Java cung cấp hai cách tiếp cận để tạo đối tượng Thread. Trong cách thứ nhất, chúng ta có khai báo một lớp con của lớp Thread và ghi đè phương thức *run()* để thực hiện các công việc của luồng. Sau khi khởi tạo một đối tượng của lớp được tạo ra, phương thức *start()* được gọi để bắt đầu thực thi các chuỗi lệnh của lớp con đó. Phương thức *start()* là phương thức được kế thừa từ lớp Thread.

Với cách tạo Thread như trên có ưu điểm là đơn giản và dễ hiểu. Tuy nhiên nó có nhược điểm là yêu cầu các đối tượng Thread được tạo ra phải là lớp con của lớp Thread trong hệ thống phân cấp lớp. Như vậy chúng ta sẽ không sử dụng được cách này trong các ứng dụng Applet.

Một cách tiếp cận khác của Java để tạo các luồng nhằm giải quyết vấn đề trên là sử dụng lớp giao diện `java.lang.Runnable`. Giao diện Runnable bao gồm một phương thức duy nhất, phương thức *run()*, phải được ghi đè bởi lớp được tạo ra. Hình 6.1 mô tả cây gia phả của lớp Thread.



Hình 6. 1 Cây gia phả của lớp `java.lang.Thread`

Cách 1: Khai báo một lớp là lớp con của lớp trừu tượng Thread và cài đè phương thức *run()* của lớp Thread.

1. Khai báo 1 lớp mới kế thừa từ lớp Thread.
2. Override lại phương thức *run()* ở lớp này, những gì trong phương thức *run()* sẽ được thực thi khi luồng bắt đầu chạy. Sau khi luồng chạy xong tất cả các câu lệnh trong phương thức *run* thì luồng cũng tự hủy.
3. Tạo 1 thể hiện (hay 1 đối tượng) của lớp ta vừa khai báo.
4. Sau đó gọi phương thức *start()* của đối tượng này để bắt đầu thực thi luồng.

Ví dụ 1:

Khai báo lớp `SubThread` kế thừa lớp Thread. Lớp `SubThread` được sử dụng để tạo các luồng con từ luồng luồng chính.

```

class SubThread extends Thread{
    int index=1;
    public void run(){
        try{
            for(int i=0;i<7;i++){
                System.out.println("SubThread running:"+ index++);
                Thread.sleep(2000);
            }
        }
        catch(InterruptedException e){
            System.err.println("SubThread is error");
        }
        System.out.println("SubThread stopped");
    }
}

```

Tạo lớp ThreadDemo được kế thừa lớp Thread trong đó luồng ThreadDemo là luồng chính và luồng SubThread là luồng con :

```

package threaddemo;
public class ThreadDemo extends Thread{
    public static void main(String[] args) {
        int index=1;
        try{
            for(int i=0;i<2;i++){
                System.out.println("MainThread running 1:"+ index++);
            }
            Thread.sleep(2000);
        }
        catch(InterruptedException e){
            System.out.println("MainThread is error");
        }
        SubThread hwt = new SubThread();
        hwt.start();

        SubThread hwt = new SubThread();
        hwt.start();
        try{
            for(int i=0;i<10;i++){
                System.out.println("MainThread running 2:"+ index++);
                Thread.sleep(1000);
            }
        }
        catch(InterruptedException e){
            System.out.println("MainThread is error");
        }
        System.out.println("MainThread stopped");
    }
}

```

Ví dụ 2 :

```
package threaddemo1;
public class ThreadDemo1 extends Thread{
    String name;
    ThreadDemo1(String n){
        this.name =n;
    }
    public void run(){
        for(int i=0;i<10;i++){
            System.out.println("Thread " + name + " running: " + i);
        }
    }
    public static void main(String[] args) {
        ThreadDemo1 t1 = new ThreadDemo1("Thread1");
        ThreadDemo1 t2 = new ThreadDemo1("Thread2");
        t1.start();
        t2.start();
    }
}
```

Cách 2: Khai báo một lớp là lớp implements của lớp Runnable.

1. Khai báo 1 lớp mới implements từ Interface Runnable
2. Override lại phương thức run() ở lớp này, những gì trong phương thức run() sẽ được thực thi khi luồng bắt đầu chạy. Sau khi luồng chạy xong tất cả các câu lệnh trong phương thức run thì luồng cũng tự hủy.
3. Tạo 1 thể hiện (hay 1 đối tượng) của lớp ta vừa khai báo. (VD : Tên đối tượng là r1)
4. Tạo 1 thể hiện của lớp Thread bằng phương thức khởi tạo: Thread(Runnable target) với Runnable target là đối tượng thuộc lớp được implements từ giao diện Runnable.
Ví dụ: Thread t1 = new Thread(r1);
5. Gọi phương thức start() của đối tượng t1.

```

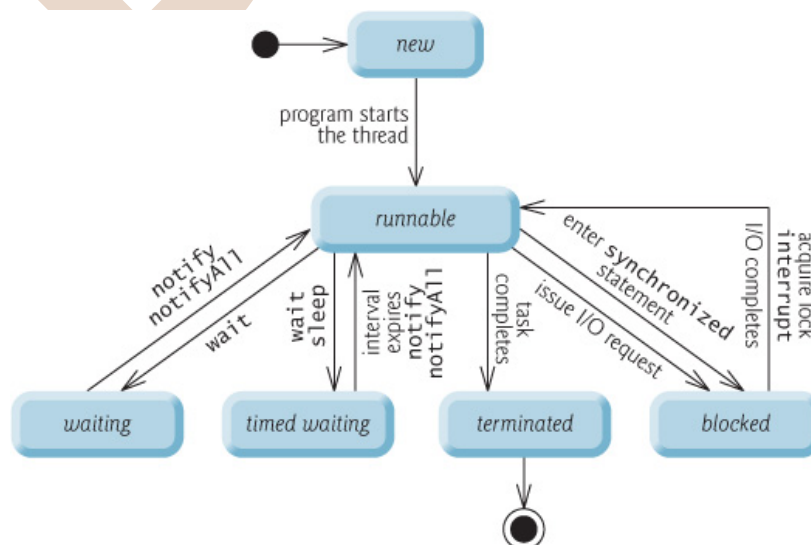
package runnableDemo;
public class RunnableDemo implements Runnable{
    String name;
    RunnableDemo(String n){
        this.name = n;
    }
    public static void main(String[] args) {
        RunnableDemo r1 = new RunnableDemo("Thread 1");
        Thread t1 = new Thread(r1);
        RunnableDemo r2 = new RunnableDemo("Thread 2");
        Thread t2 = new Thread(r2);
        t1.start();
        t2.start();
    }
    public void run() {
        for(int i=0;i<10;i++){
            System.out.println("Thread " + name + " running: " + i);
        }
    }
}

```

Cách khai báo này được sử dụng trong trường hợp lớp của chúng ta đã kế thừa một lớp nào đó (ví dụ Applet, Frame,...). Khi đó lớp của chúng ta không thể kế thừa thêm lớp Thread mà sẽ là một lớp thực hiện của lớp Runnable.

6.3. Vòng đời của một luồng

Trạng thái của một luồng được minh họa trong biểu đồ UML tại hình 6.2. Tại một thời điểm bất kỳ, luồng có thể rơi vào một trong các trạng thái này.



Hình 6. 2 Biểu đồ UML mô tả các trạng thái của một luồng

New: Đây là trạng thái khi luồng vừa được khởi tạo bằng phương thức khởi tạo của lớp Thread nhưng chưa được start(). Ở trạng thái này, luồng được tạo ra nhưng chưa được cấp phát tài nguyên và cũng chưa chạy.

Runnable (có thể chạy): Sau khi gọi phương thức start() thì luồng đã được cấp phát tài nguyên và các lịch điều phối CPU cho luồng cũng bắt đầu có hiệu lực. Ở đây, chúng ta dùng trạng thái là Runnable chứ không phải Running, vì luồng không thực sự đang chạy mà nó muốn chạy nhưng phải đợi vì một luồng khác đang chạy.

Waiting (đợi): Đôi khi một luồng đang ở trạng thái runnable có thể chuyển sang trạng thái waiting để chờ một luồng khác chạy. Từ trạng thái waiting, luồng được trở về trạng thái runnable khi có một luồng khác đánh thức nó để tiếp tục chạy.

Timed waiting (đợi trong khoảng thời gian): Một luồng đang ở trạng thái runnable có thể chuyển sang trạng thái timed waiting để chờ trong một khoảng thời gian. Luồng trở về trạng thái runnable khi hết thời gian chờ hoặc khi sự kiện nào đó mà luồng đang chờ xảy ra.

Blocked (tạm dừng): Một luồng chuyển sang trạng thái blocked khi nó cố gắng thực hiện một nhiệm vụ nhưng chưa thể hoàn thành ngay mà phải tạm thời đợi cho đến khi nhiệm vụ đó kết thúc. Ví dụ, khi một luồng đưa ra yêu cầu liên quan đến I/O, hệ điều hành sẽ block luồng cho đến khi các yêu cầu I/O được thực hiện xong. Sau đó luồng sẽ trở về trạng thái runnable. Khi luồng ở trạng thái blocked thì nó không thể sử dụng tài nguyên của bộ vi xử lý cho dù bộ vi xử lý rỗi.

Terminated (kết thúc): Luồng chuyển sang trạng thái terminated khi đã hoàn thành các công việc hoặc khi có lỗi xảy ra.

6.4. Tạm ngừng, chạy lại và ngừng luồng

Java cung cấp một số phương thức cho phép tạm ngừng, chạy lại hoặc ngừng hẳn luồng như sau :

suspend(): Đây là phương thức làm tạm dừng hoạt động của 1 luồng nào đó bằng cách ngưng cung cấp CPU cho luồng này. Để cung cấp lại CPU cho luồng ta sử dụng phương thức resume(). Cần lưu ý 1 điều là ta không thể dừng ngay hoạt động của luồng bằng phương thức này. Phương thức suspend() không dừng ngay tức thì hoạt động của luồng mà sau khi luồng này trả CPU về cho hệ điều hành thì không cấp CPU cho luồng nữa.

resume(): Đây là phương thức làm cho luồng chạy lại khi luồng bị dừng do phương thức suspend() bên trên. Phương thức này sẽ đưa luồng vào lại lịch điều phối CPU để luồng được cấp CPU chạy lại bình thường.

stop(): Luồng này sẽ kết thúc phương thức run() bằng cách ném ra 1 ngoại lệ ThreadDeath, điều này cũng sẽ làm luồng kết thúc 1 cách ép buộc. Nếu giả sử, trước khi

gọi stop() mà luồng đang nắm giữ 1 đối tượng nào đó hoặc 1 tài nguyên nào đó mà luồng khác đang chờ thì có thể dẫn tới việc xảy ra deadlock.

destroy(): dừng hẳn luồng.

sleep(long) : tạm dừng luồng trong một khoảng thời gian millisecond.

Ví dụ với phương thức sleep():

```
package threaddemo1;
public class ThreadDemo1 extends Thread{
    String name;
    ThreadDemo1(String n){
        this.name =n;
    }
    public void run(){
        for(int i=0;i<10;i++){
            System.out.println("Thread " + name + " running: " + i);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException ex) {
                ex.printStackTrace();
            }
        }
    }
    public static void main(String[] args) {
        ThreadDemo1 t1 = new ThreadDemo1("Thread1");
        ThreadDemo1 t2 = new ThreadDemo1("Thread2");
        t1.start();
        t2.start();
    }
}
```

6.5. Mức độ ưu tiên và lập lịch cho các luồng

Khác với hệ thống có nhiều bộ vi xử lý có thể thực thi các luồng đồng thời về mặt vật lý, đối với hệ thống đơn xử lý chỉ có thể xử lý các luồng đồng thời về mặt logic. Điều này có nghĩa là các luồng chia sẻ thời gian thực hiện với nhau dựa trên tính khả dụng của bộ vi xử lý trong hệ thống. Để giải quyết vấn đề này một cách đồng bộ, Java phải thực hiện lập lịch để thực thi các luồng (scheduling) và thiết lập mức độ ưu tiên (priority) cho các luồng. Mức độ ưu tiên của luồng có thể được gán khi tạo luồng. Luồng ưu tiên cao nhất tiếp tục chạy cho đến khi nó chuyển sang trạng thái không thể chạy được (non – runnable) hoặc giảm mức độ ưu tiên hoặc khi một luồng có mức độ ưu tiên cao hơn được thực thi.

Mức độ ưu tiên của luồng là một giá trị số nguyên giữa MIN_PRIORITY và MAX_PRIORITY. Các hằng số này được định nghĩa trong lớp Thread. Trong Java 1.0, MIN_PRIORITY là 1 và MAX_PRIORITY là 10. Mức ưu tiên của luồng được đặt khi

nó được tạo. Nó được đặt ở mức ưu tiên giống như luồng đã tạo ra nó. Mức ưu tiên mặc định của một luồng là NORM_PRIORITY và bằng 5. Mức độ ưu tiên của một luồng có thể được thay đổi bằng phương thức setPriority().

Cách tiếp cận lập lịch của Java được gọi là lập lịch ưu tiên. Khi một luồng có mức độ ưu tiên cao hơn có thể chạy được, nó sẽ xử lý các luồng có mức độ ưu tiên thấp hơn và ngay lập tức được thực hiện tại vị trí của chúng. Nếu hai hoặc nhiều luồng ưu tiên cao hơn trở nên có thể chạy được, bộ lập lịch Java sẽ thực thi các luồng đan xen.

Ví dụ :

```
package prioritydemo;
import java.util.logging.Level;
import java.util.logging.Logger;
public class PriorityDemo {
    public static void main(String[] args) {
        Priority t1 = new Priority();
        t1.start();
        Priority t2 = new Priority();
        t2.start();
        Priority t3 = new Priority();
        t3.start();
    }
}
class Priority extends Thread{
    int sleep;
    static int prio = 3;
    public Priority(){
        sleep += 500;
        prio++;
        setPriority(prio);
    }
    public void run(){
        try {
            Thread.sleep(sleep);
            System.out.println("Name: " + getName() + "
Priority: " + getPriority());
        } catch (InterruptedException ex) {
        }
    }
}
```

Kết quả:

```
Name: Thread-2 Priority: 6
Name: Thread-1 Priority: 5
Name: Thread-0 Priority: 4
```

6.6. Đồng bộ (Synchronization)

Khi làm việc với đa luồng, chúng ta có thể gặp tình huống nhiều luồng cùng truy cập tới tài nguyên chung, ví dụ như cùng biến hoặc cùng dữ liệu trong cơ sở dữ liệu. Trong trường hợp này có thể xảy ra trường hợp xung đột về dữ liệu. Ví dụ một luồng đang đọc dữ liệu trong khi một luồng khác đang thay đổi dữ liệu. Để giải quyết vấn đề này, chúng ta phải đặt ra quy định chỉ cho một luồng truy cập dữ liệu tại một thời điểm. Khi một luồng hoàn thành công việc với dữ liệu thì luồng khác mới có thể truy cập vào dữ liệu. Trong Java, để thực hiện việc này, chúng ta dùng từ khóa “synchronized” để đồng bộ việc truy cập của các luồng.

Ví dụ: Giả sử chúng ta có một biến đếm count và chúng ta cho 2 luồng cùng truy cập vào biến này để tăng giá trị của biến. Sau mỗi lần tăng, chúng ta in giá trị của biến ra. Kết quả mong muốn như sau:

Applet

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,

Giả sử chương trình được viết như sau:

Lớp chính:

```
import java.applet.Applet;
import java.awt.Graphics;
public class NewApplet extends Applet {
    public void init() {
        this.setSize(600,200);
        Graphics g = getGraphics();
        SharedNumber count = new SharedNumber(g);
        Counter c1 = new Counter(count,"Thread 1");
        Counter c2 = new Counter(count,"Thread 2");
        c1.start();
        c2.start();
    }
}
```

Lớp Counter:

```
public class Counter extends Thread {
    private SharedNumber count;
    public Counter(SharedNumber count,String name){
        super(name);
        this.count = count;
    }
    public void run(){
        for(int i=1;i<=10;i++){
            count.increment();
        }
    }
}
```

```

        try {
            Thread.sleep(1000);
        } catch (InterruptedException ex) {

Logger.getLogger(Counter.class.getName()).log(Level.SEVERE,
null, ex);

        }
        System.out.println(this.getName());
    }
}
}

```

Lớp SharedNumber:

```

import java.awt.Graphics;
public class SharedNumber {
    private int n=0;
    private Graphics g;
    private int x=1;
    public SharedNumber(Graphics g){
        this.g =g;
    }
    public void increment(){
        n = n+1;
        g.drawString(n+ " ", x*20,30);
        x++;
    }
}

```

Với chương trình như trên sẽ cho kết quả như sau:

Applet

1, 3, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,

Trong kết quả trên ta thấy một số vị trí bị lỗi khi các giá trị được in ra trên cửa sổ bị trùng nhau về tọa độ. Sở dĩ như vậy vì lý do sau:

- Khi luồng thứ nhất truy cập phương thức *increment()* và tăng giá trị của biến *n* lên 1 đơn vị (*n=1*) và in giá trị của biến *n* (*n=1*) tại tọa độ (20,30).
- Luồng thứ hai chiếm quyền sử dụng phương thức *increment()*. Khi đó giá trị của biến *n* được tăng lên một đơn vị (*n=2*), giá trị của biến *n* được in ra cũng tại tọa độ (20,30) và tăng giá trị *x* lên một đơn vị (*x=2*). Chính vì vậy, giá trị 1 và 2 của *n* bị trùng nhau.
- Luồng thứ nhất tiếp tục sử dụng hàm *increment()* và tăng giá trị của *x* tiếp lên một đơn vị (*x=3*). Sau đó luồng này tiếp tục sử dụng hàm *increment()* để tăng giá trị *n* lên một đơn vị và in giá trị *n* = 3 lên màn hình tại tọa độ (60,30).

Chính vì vậy chúng ta thấy hai giá trị $n=1$ và $n=3$ được in ra với khoảng cách xa.

- Luồng thứ hai tiếp tục sử dụng hàm `increment()` và tăng giá trị của n lên 4 và in ra màn hình tại tọa độ (60,30). Chính vì vậy giá trị 3 và 4 được in ra cùng vị trí.

Để tránh việc hai luồng cùng truy cập vào phương thức `increment()` trong khi luồng kia chưa thực hiện xong phương thức đó, Java cung cấp cơ chế đồng bộ đối với phương thức này. Cụ thể, hàm `increment()` được khai báo như sau:

```
public synchronized void increment() {
    n = n+1;
    g.drawString(n+ " ", x*20, 30);
    x++;
}
```

Khi đó chúng ta sẽ được kết quả như sau:

Applet

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,

Như vậy chúng ta có thể hiểu là từ khóa **synchronized** trong phần khai báo phương thức giúp cho một luồng được quyền sử dụng toàn bộ các lệnh của phương thức đó xong rồi mới đến lượt các luồng khác được quyền sử dụng phương thức đó.

6.7. Tương tác giữa các luồng

Giả sử ta có 2 luồng cùng hiển thị lần lượt thông tin trên màn hình tại cùng vị trí. Chương trình có dạng như sau:

```
import java.applet.Applet;
import java.awt.Graphics;
public class Display extends Applet {
    public void init() {
        Graphics g = getGraphics();

        Thread1 t1 = new Thread1(g);
        Thread2 t2 = new Thread2(g);

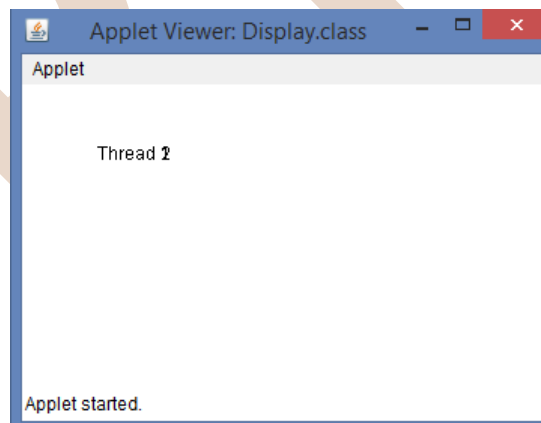
        t1.start();
        t2.start();
    }
}
Luồng 1:
public class Thread1 extends Thread {
    private Graphics g;
    public Thread1(Graphics g) {
        this.g = g;
```

```

    }
    public void run(){
        while(true){
            g.drawString("Thread 1",50,50);
            try {
                Thread.sleep(5000);
            } catch (InterruptedException ex) {
            }
        }
    }
}
}
Luồng 2:
public class Thread2 extends Thread {
    private Graphics g;
    public Thread2(Graphics g){
        this.g = g;
    }
    public void run(){
        while(true){
            g.drawString("Thread 2",50,50);
            try {
                Thread.sleep(5000);
            } catch (InterruptedException ex) {
            }
        }
    }
}
}

```

Kết quả hiển thị như sau:



Trong chương trình trên chúng ta thấy hai luồng cùng sử dụng màn hình tại một thời điểm mà không có sự trao đổi qua lại để định khoảng thời gian sử dụng cho mỗi luồng. Vì vậy, thông báo trên màn hình được hiển thị chồng lấn lên nhau. Để giải quyết vấn đề này, chúng ta phải định ra một khoảng thời gian cho mỗi luồng. Khi luồng nào hết thời gian thì sẽ thông báo cho các luồng khác để biết. Cụ thể chương trình như sau:

Luồng 1:

```
public class Thread1 extends Thread {
    private Graphics g;
    private SharedScreen s;
    public Thread1(Graphics g, SharedScreen s){
        this.g = g;
        this.s = s;
    }
    public void run(){
        while(true){
            s.request();
            g.drawString("Thread 1",50,50);
            s.relinquish();
            try {
                Thread.sleep(5000);
            } catch (InterruptedException ex) {
            }
        }
    }
}
```

Luồng 2:

```
public class Thread2 extends Thread {
    private Graphics g;
    private SharedScreen s;
    public Thread2(Graphics g, SharedScreen s){
        this.g = g;
        this.s = s;
    }
    public void run(){
        while(true){
            s.request();
            g.drawString("Thread 2",50,50);
            s.relinquish();
            try {
                Thread.sleep(5000);
            } catch (InterruptedException ex) {
            }
        }
    }
}
```

Lớp định thời và thông báo việc sử dụng màn hình:

```
public class SharedScreen {
    private boolean inUse = false;
    private Graphics g;
    SharedScreen(){
    }
}
```

```
public synchronized void request(){
    while(inUse)
        try {
            wait();
        } catch (InterruptedException ex) {

        }
    inUse = true;
}
public synchronized void relinquish(){
    inUse = false;
    notifyAll();
}
}
```



BÀI TẬP CHƯƠNG 6

Bài 1: Viết chương trình hiển thị hai luồng cùng truy cập tới một biến nguyên. Luồng thứ nhất tăng giá trị của biến lên 1 đơn vị. Luồng thứ hai tăng giá trị của biến lên 2 đơn vị. Mỗi luồng thực hiện công việc 10 lần.

Bài 2: Viết chương trình hiển thị hai luồng. Luồng thứ nhất hiển thị một số ngẫu nhiên trong khoảng từ 0 đến 10. Luồng thứ hai hiển thị giá trị bình phương của số ngẫu nhiên của luồng 1.

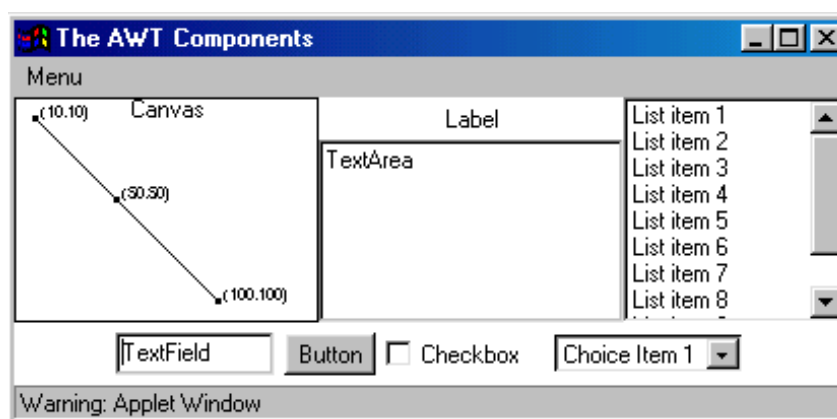
PTIT

CHƯƠNG 7. LẬP TRÌNH GIAO DIỆN TRONG JAVA

7.1. Lập trình giao diện với các đối tượng cơ bản

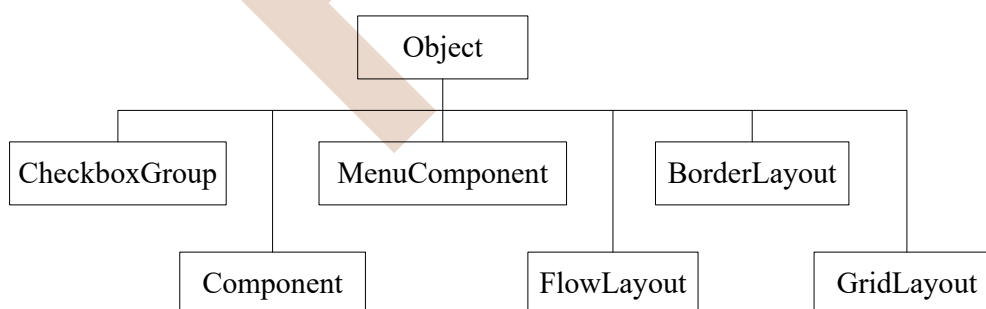
Giao diện đồ họa (Graphical User Interface) đóng một vai trò quan trọng trong các ứng dụng phần mềm. GUI giúp cho người sử dụng dễ dàng tương tác với chương trình thay vì việc phải làm theo từng bước các hướng dẫn dạng text nhàm chán và khó hiểu.

Trong Java, một GUI có thể bao gồm các thành phần đồ họa như button, textbox, label, list box,... Mỗi thành phần đồ họa là một đối tượng đồ họa của Java. Người dùng tương tác với các đối tượng này thông qua chuột và bàn phím. Các thành phần đồ họa kể trên là đối tượng được tạo ra từ lớp *Component* thuộc gói *java.awt* của Java.



Hình 7. 1 Các thành phần giao diện của AWT

AWT (Abstract Windowing Toolkit) là tập hợp các lớp Java cho phép người lập trình tạo ra giao diện đồ họa, nhận dữ liệu người dùng nhập từ chuột và bàn phím. AWT cung cấp các lớp dùng để quản lý các thành phần đồ họa, quản lý font chữ, quản lý các sự kiện (các tương tác của người dùng với các thành phần đồ họa). Gói AWT bao gồm các lớp, giao diện và các gói đồ họa khác. Các lớp của AWT như sau:



Hình 7. 2 Các gói giao diện đồ họa của AWT

Ngoài thư viện đồ họa là gói AWT được phát hành cho phiên bản JDK 1.0, Java hiện nay cung cấp thêm các gói thư viện nâng cấp lên từ AWT là các gói SWING cho phiên bản JDK 1.1 và JavaFX cho phiên bản JDK 8.

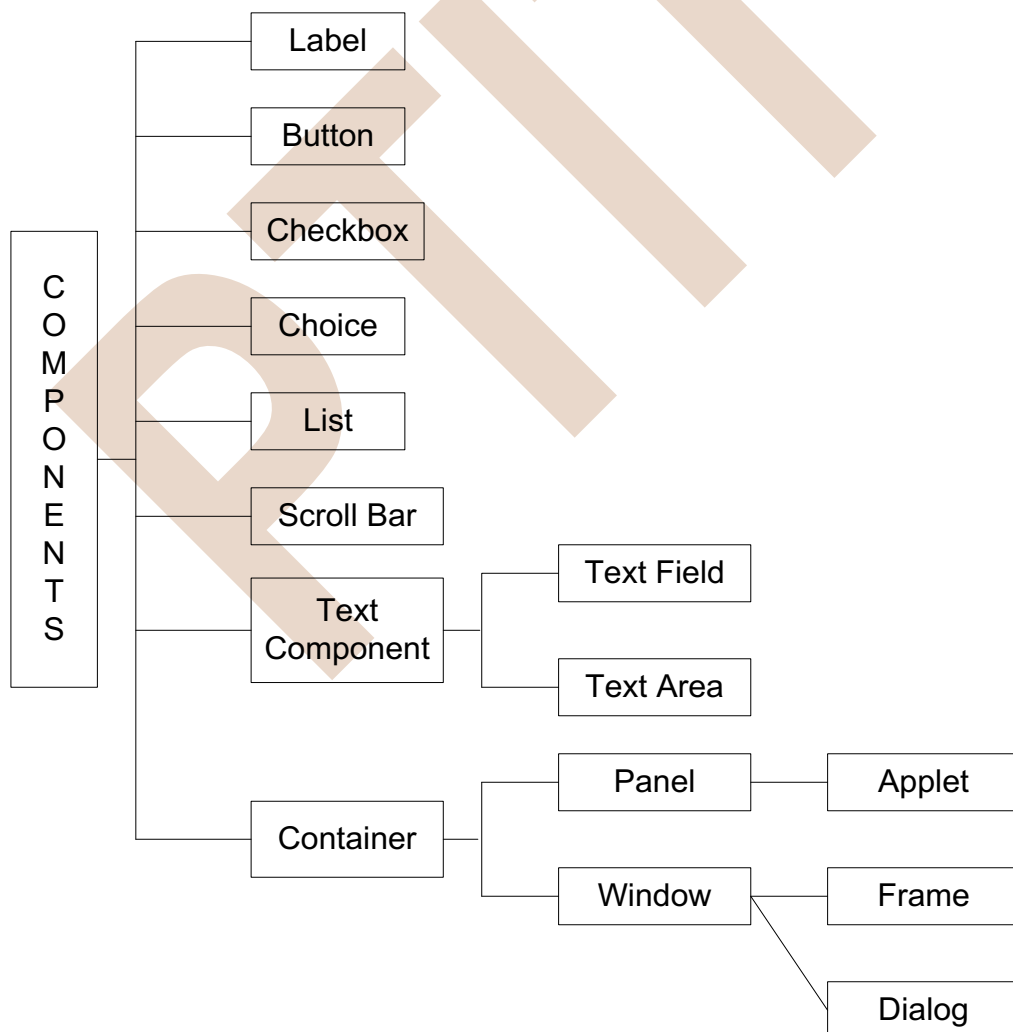
Bên cạnh các API đồ họa AWT / Swing / JavaFX được cung cấp trong JDK, một số tổ chức / nhà cung cấp khác cũng đã cung cấp API đồ họa làm việc với Java, như Bộ

công cụ tiện ích tiêu chuẩn của Eclipse (SWT) (được sử dụng trong Eclipse), Bộ công cụ web của Google (GWT) trong Android, API đồ họa 3D trong OpenGL(JOGL) và Java3D.

7.2. Container

Container trong Java được hiểu là một cửa sổ và là nơi đặt vào đó các thành phần đồ họa để tạo nên một giao diện người dùng. Ngoài ra, Container cũng cho phép vẽ các thành phần đồ họa (đường thẳng, hình tròn, hình tam giác,...) lên đó.

Gói AWT của Java chứa một lớp tên là Container. Các lớp con của lớp Container là lớp Panel và lớp Frame. Các lớp con này cũng được coi là các Container. Frame là một lớp con của Window. Mỗi Frame là một cửa sổ giao diện riêng biệt có thể nhìn thấy được trong khi Panel cũng là container nhưng Panel nằm trong cửa sổ và không nhìn thấy được. Panel không có các khung viền xung quanh và thường được hiển thị bởi trình duyệt hoặc ứng dụng Appletviewer. Appletviewer là công cụ mà JDK cung cấp để hiển thị Applet.



Hình 7. 3 Cây gia phả của AWT

7.2.1. Frame

Frame là một container dùng để chứa các thành phần đồ họa trong Java. Phương thức khởi tạo của Frame như sau :

Frame(): Tạo một Frame ẩn.

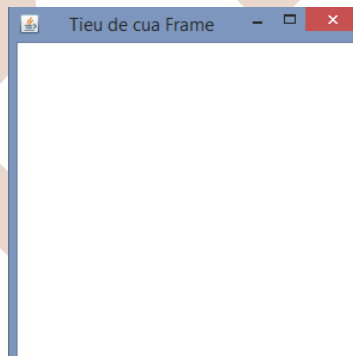
Frame(String title): Tạo một frame ẩn với một tiêu đề trên cửa sổ.

Trong ví dụ dưới đây, một lớp FrameExample được kế thừa lớp Frame. Vì vậy, lớp FrameExample cũng là một Frame và được sử dụng để thiết kế giao diện cho giao diện người sử dụng. Trong phương thức constructor của FrameExample, phương thức super() được sử dụng để gọi hàm khởi tạo của lớp Frame.

Ví dụ :

```
package frameexample;
import java.awt.Frame;
public class FrameExample extends Frame{
    FrameExample(String title){
        super(title);
    }
    public static void main(String[] args) {
        FrameExample fe = new FrameExample("Tieu de cua Frame");
        fe.setSize(300,300);
        fe.setVisible(true);
    }
}
```

Kết quả :



7.2.2. Panel

Panel cũng là một cửa sổ nhưng nó không hiển thị được giống như Frame mà nó ở dạng ẩn và được sử dụng chỉ để quản lý một nhóm các component nào đó. Phương thức khởi tạo của Panel là Panel(). Vì Panel ở dạng ẩn nên muốn hiển thị nó thì ta phải đặt nó vào trong một Frame.

Ví dụ :

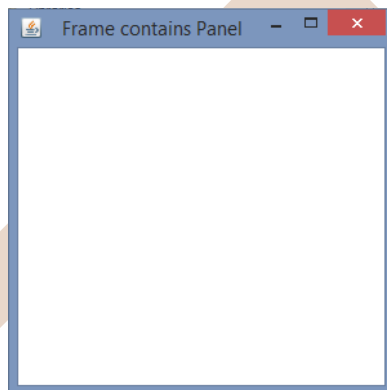
```

package panelexample;
import java.awt.*;
public class PanelExample extends Panel{
    PanelExample() {

    }
    public static void main(String[] args) {
        PanelExample pe = new PanelExample();
        Frame f = new Frame("Frame contains Panel");
        f.add(pe);
        f.setSize(300,300);
        f.setVisible(true);
    }
}

```

Kết quả :



7.2.3. Dialog

Hộp thoại là một cửa sổ con được bật lên để hiển thị thông báo cho người dùng. Hộp thoại (dialog) giống như một frame, là một lớp con của lớp Window. Hộp thoại được hiển thị dựa trên một Frame. Khi Frame bị hủy thì hộp thoại cũng bị hủy theo.

Mỗi hộp thoại có một tham số 'modal' dùng để khóa các tương tác của người dùng với các cửa sổ đang hiển thị trên màn hình cho đến khi yêu cầu trên hộp thoại được người dùng đáp ứng.

Hàm khởi tạo :

```
Frame frametest = new Frame() ;
```

```
String title = "Title" ;
```

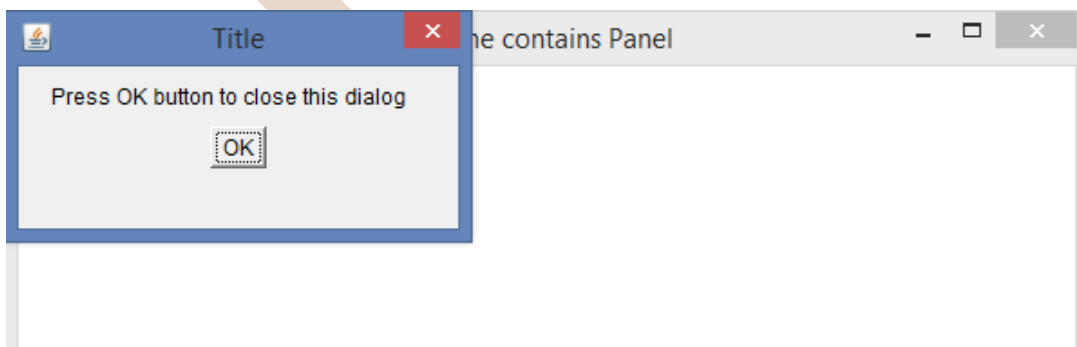
```
boolean modal = true ;
```

```
Dialog dlg = new Dialog(frametest,title,modal) ;
```

Ví dụ :

```
public class FrameTest extends Panel{
    FrameTest() {
    }
    public static void main(String[] args) {
        FrameTest ft = new FrameTest();
        Frame f = new Frame("Frame contains Panel");
        f.add(ft);
        f.setSize(600,600);
        f.setVisible(true);
        String title = "Title";
        boolean modal = true;
        Dialog dlg = new Dialog(f,title,modal);
        dlg.setLayout(new FlowLayout());
        Label lb = new Label("Press OK button to close this dialog");
        dlg.add(lb);
        Button bt = new Button("OK");
        bt.addActionListener(new ActionListener()
        {
            public void actionPerformed(ActionEvent e) {
                dlg.setVisible(false);
            }
        });
        dlg.add(bt);
        dlg.setVisible(true);
    }
}
```

Kết quả :



7.3. Component

7.3.1. Label

Lớp nhãn dùng để hiển thị một chuỗi ký tự trên một frame. Hàm khởi tạo của nhãn như sau :

Tạo một nhãn trắng :

Label() ;

Tạo một nhãn với một dòng text :

Lable(String labelstring) ;

Tạo một nhãn với dòng text được căn chỉnh theo lề trái, phải hoặc giữa :

Label(String labelstring, int alignment) ;

Alignment có thể là Label.LEFT, Label.RIGHT, Label.CENTER.

Một số phương thức của lớp Label :

Phương thức	Mô tả
setFont(Font f)	Thay đổi font chữ của nhãn
setText(String s)	Thiết lập chuỗi ký tự của nhãn
getText()	Lấy ra chuỗi ký tự của nhãn.

Bảng 7. 1 Một số phương thức của Label

7.3.2. *TextField*

TextField là ô cho người dùng nhập vào một dòng ký tự. Hàm khởi tạo của TextField như sau :

Tạo một TextField trắng :

TextField() ;

Tạo một TextField với số lượng ký tự cho trước :

TextField(int column) ;

Tạo một TextField với một chuỗi ký tự cho trước :

TextField(String s) ;

Tạo một TextField với một chuỗi ký tự và độ dài cho trước :

TextField(String s, int column) ;

Một số phương thức thông dụng của TextField :

Phương thức	Mô tả
setEchoChar(char)	Hiện thị các ký tự nhập vào dưới dạng ký tự cho trước. Ví dụ : setEchoChar('*') ;
setText(String s)	Thiết lập chuỗi ký tự hiển thị trong TextField
getText()	Lấy ra chuỗi ký tự trong TextField

setEditable(boolean)	Cho phép TextField được phép nhập ký tự hay không.
isEditable()	Kiểm tra xem TextField được phép nhập ký tự hay không.

Bảng 7. 2 Một số phương thức của TextField

7.3.3. *TextArea*

TextArea được sử dụng cho người dùng nhập một hoặc nhiều dòng ký tự. Một số hàm khởi tạo của TextArea :

Tạo một TextArea trắng : *TextArea()* ;

Tạo một TextArea với số hàng và cột cho trước :

TextArea (int rows, int cols) ;

Tạo một TextArea với một chuỗi ký tự cho trước :

TextArea (String s) ;

Tạo một TextArea với một chuỗi ký tự và kích thước cho trước :

TextArea (String s, int rows, int cols) ;

Một số phương thức thường sử dụng của TextArea :

Phương thức	Mô tả
setText(String s)	Thiết lập chuỗi ký tự hiển thị trong TextArea
getText()	Lấy ra chuỗi ký tự trong TextArea
setEditable(boolean)	Cho phép TextArea được phép nhập ký tự hay không.
isEditable()	Kiểm tra xem TextField được phép nhập ký tự hay không.
insertText(String, int)	Chèn một chuỗi ký tự vào một vị trí trong TextArea
replaceText(String, int, int)	Thay thế chuỗi ký có sẵn bằng một chuỗi ký tự mới tại vị trí giữa 2 giá trị nhập vào.

Bảng 7. 3 Một số phương thức của TextArea

7.3.4. *Button*

Nút bấm được tạo ra bằng một trong các phương thức sau :

Tạo một nút bấm trắng : *Button()* ;

Tạo một nút bấm với nhãn cho trước : *Button(String s) ;*

Thiết lập và truy xuất nhãn của nút bấm : *setLabel()*, *getLabel()*.

Ví dụ :

```
package frameexample;
import java.awt.*;
public class FrameExample extends Frame{
    Label lb1 = new Label("Email:");
    TextField tf1 = new TextField(20);
    Label lb2 = new Label("Password:");
    TextField tf2 = new TextField(20);

    TextArea ta = new TextArea("Comments",10,10);

    Button bt = new Button("Submit");

    FrameExample(String title){
        super(title);
        this.setLayout(new FlowLayout());
        this.add(lb1);
        this.add(tf1);
        this.add(lb2);
        this.add(tf2);
        tf2.setEchoChar('*');
        this.add(ta);
        this.add(bt);
    }
    public static void main(String[] args) {
        FrameExample fe = new FrameExample("Tieu de cua Frame");
        fe.setSize(300,300);
        fe.setVisible(true);
    }
}
```

Kết quả :



7.3.5. *Checkbox và RadioButton*

Checkbox được sử dụng trong trường hợp cho người dùng có thể chọn nhiều trường hợp. Người dùng chọn hoặc hủy chọn một hoặc nhiều checkbox. Radiobutton tương tự như checkbox. Tuy nhiên, người dùng chỉ có thể chọn một trong các radiobutton trong một nhóm.

Các hàm khởi tạo dùng để tạo ra các checkbox :

Checkbox() ;

Checkbox(String text) ;

Để tạo ra radiobutton, trước hết ta phải tạo đối tượng CheckboxGroup :

CheckboxGroup fruitGroup = new CheckboxGroup() ;

Sau đó ta tạo ra các checkbox lẻ trong nhóm checkboxgroup trên :

Checkbox apple = new Checkbox("Apple",fruitGroup,true) ;

Checkbox orange = new Checkbox("Orange",fruitGroup,false) ;

Checkbox peer = new Checkbox("Peer",fruitGroup,false) ;

Để thiết lập và lấy ra trạng thái của checkbox, ta dùng phương thức *setState()* và *getState()*.

7.3.6. *Choice*

Thành phần đồ họa Choice hiển thị một danh sách các lựa chọn và cho phép người dùng lựa chọn một mục trong danh sách đó. Để tạo ra danh sách lựa chọn, ta thực hiện các bước sau :

Tạo một đối tượng Choice :

Choice color = new Choice() ;

Thêm các mục vào cho danh sách :

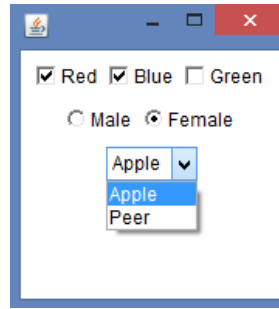
```
color.addItem('Red');
```

```
color.addItem( "Green" );
```

Ví dụ :

```
package choicelist;
import java.awt.*;
public class ChoiceList extends Frame{
    //Tạo Checkbox
    Checkbox cb1 = new Checkbox("Red");
    Checkbox cb2 = new Checkbox("Blue");
    Checkbox cb3 = new Checkbox("Green");
    //Tạo RadioButton
    CheckboxGroup cbg = new CheckboxGroup();
    Checkbox m = new Checkbox("Male", cbg, true);
    Checkbox fm = new Checkbox("Female", cbg, false);
    //Tạo ChoiceList
    Choice c = new Choice();
    ChoiceList() {
        this.setLayout(new FlowLayout());
        this.add(cb1);
        this.add(cb2);
        this.add(cb3);
        this.add(m);
        this.add(fm);
        this.add(c);
        c.addItem("Apple");
        c.addItem("Peer");
    }
    public static void main(String[] args) {
        ChoiceList cl = new ChoiceList();
        cl.setSize(300,300);
        cl.setVisible(true);
    }
}
```

Kết quả :



7.4. Layout

Mỗi Container có một lớp có chức năng quản lý cách bố trí sắp xếp các thành phần giao diện trong Container đó gọi là Layout Manager. AWT hỗ trợ các lớp Layout Manager bao gồm: FlowLayout, GridLayout, BorderLayout, GridBagLayout, BoxLayout, CardLayout.

Container có một phương thức dùng để xác định loại Layout Manager mà Container sử dụng. Phương thức đó là `setLayout()`.

7.4.1. FlowLayout Manager

FlowLayout là cách sắp xếp mặc định cho các ứng dụng Applet và container Panel. Trong cách sắp xếp này, các thành phần đồ họa được sắp đặt lần lượt từ trái sang phải, từ trên xuống dưới. Hàm khởi tạo cho FlowLayout như sau :

```
FlowLayout fl = new FlowLayout() ;
```

```
setLayout(fl) ;
```

Hai lệnh trên có thể thay bằng một lệnh sau:

```
setLayout(new FlowLayout()) ;
```

7.4.2. BorderLayout Manager

BorderLayout là cách sắp xếp mặc định của Window, Frame và Dialog. Cách sắp đặt này cho phép sắp xếp tối đa 5 thành phần giao diện trong một container. Các thành phần này được đặt tại 5 vị trí theo hướng Đông, Tây, Nam, Bắc và trung tâm của container.

Ví dụ :

```
import java.awt.*;
import java.awt.event.*;

// An AWT GUI program inherits the top-level container
java.awt.Frame
public class AWTBorderLayoutDemo extends Frame {
    private Button btnNorth, btnSouth, btnCenter, btnEast,
    btnWest;
```

```

// Constructor to setup GUI components and event handlers
public AWTBorderLayoutDemo () {
    setLayout(new BorderLayout(3, 3));
    // "super" Frame sets layout to BorderLayout,
    // horizontal and vertical gaps of 3 pixels

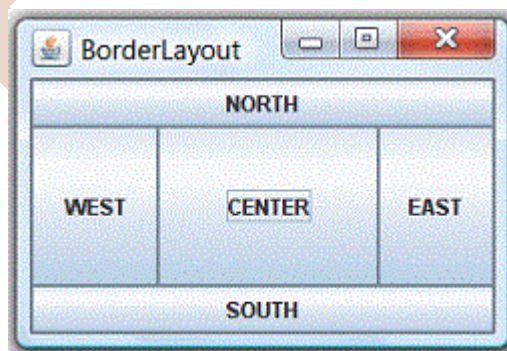
    // The components are added to the specified zone
    btnNorth = new Button("NORTH");
    add(btnNorth, BorderLayout.NORTH);
    btnSouth = new Button("SOUTH");
    add(btnSouth, BorderLayout.SOUTH);
    btnCenter = new Button("CENTER");
    add(btnCenter, BorderLayout.CENTER);
    btnEast = new Button("EAST");
    add(btnEast, BorderLayout.EAST);
    btnWest = new Button("WEST");
    add(btnWest, BorderLayout.WEST);

    setTitle("BorderLayout Demo"); // "super" Frame sets
title
setSize(280, 150); // "super" Frame sets initial
size
setVisible(true); // "super" Frame shows
}

// The entry main() method
public static void main(String[] args) {
    new AWTBorderLayoutDemo(); // Let the constructor do
the job
}
}

```

Kết quả:



7.4.3. GridLayout Manager

Cách sắp đặt này chia cửa sổ container thành các ô lưới. Các thành phần đồ họa được đặt vào vào một ô của lưới theo tọa độ được chỉ ra. Hàm khởi tạo của GridLayout :

```
GridLayout gl = GridLayout(4,4) ; // Tạo lưới GridLayout gồm 16 ô.
```

Ví dụ :

```
import java.awt.*;
import java.awt.event.*;

// An AWT GUI program inherits the top-level container
java.awt.Frame
public class AWTPanelDemo extends Frame {
    private Button[] btnNumbers; // Array of 10 numeric
Buttons
    private Button btnHash, btnStar;
    private TextField tfDisplay;

    // Constructor to setup GUI components and event handlers
    public AWTPanelDemo () {
        // Set up display panel
        Panel panelDisplay = new Panel(new FlowLayout());
        tfDisplay = new TextField("0", 20);
        panelDisplay.add(tfDisplay);

        // Set up button panel
        Panel panelButtons = new Panel(new GridLayout(4, 3));
        btnNumbers = new Button[10]; // Construct an array of
10 numeric Buttons
        btnNumbers[1] = new Button("1"); // Construct Button
"1"
        panelButtons.add(btnNumbers[1]); // The Panel adds this
Button
        btnNumbers[2] = new Button("2");
        panelButtons.add(btnNumbers[2]);
        btnNumbers[3] = new Button("3");
        panelButtons.add(btnNumbers[3]);
        btnNumbers[4] = new Button("4");
        panelButtons.add(btnNumbers[4]);
        btnNumbers[5] = new Button("5");
        panelButtons.add(btnNumbers[5]);
        btnNumbers[6] = new Button("6");
        panelButtons.add(btnNumbers[6]);
        btnNumbers[7] = new Button("7");
        panelButtons.add(btnNumbers[7]);
        btnNumbers[8] = new Button("8");
        panelButtons.add(btnNumbers[8]);
        btnNumbers[9] = new Button("9");
        panelButtons.add(btnNumbers[9]);
        // You should use a loop for the above
statements!!!
        btnStar = new Button("*");
        panelButtons.add(btnStar);
        btnNumbers[0] = new Button("0");
        panelButtons.add(btnNumbers[0]);
        btnHash = new Button("#");
        panelButtons.add(btnHash);
    }
}
```

```

        setLayout(new BorderLayout()); // "super" Frame sets to
BorderLayout
        add(panelDisplay, BorderLayout.NORTH);
        add(panelButtons, BorderLayout.CENTER);

        setTitle("BorderLayout Demo"); // "super" Frame sets
title
        setSize(200, 200); // "super" Frame sets
initial size
        setVisible(true); // "super" Frame shows
    }

    // The entry main() method
    public static void main(String[] args) {
        new AWTPanelDemo(); // Let the constructor do the job
    }
}

```

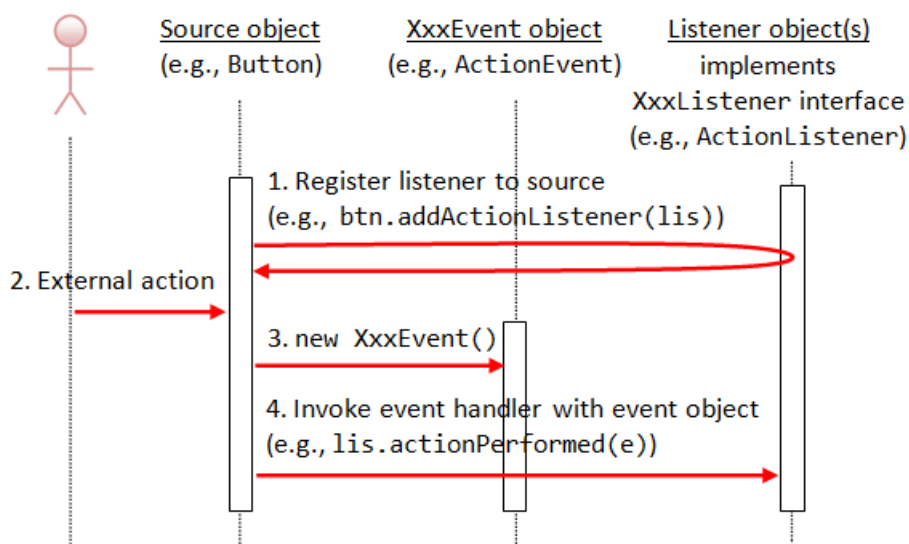
7.5. Xử lý sự kiện

7.5.1. Khái niệm xử lý sự kiện

Trong lập trình giao diện đồ họa, sự kiện (event) được định nghĩa là một hành động được thực hiện bởi người sử dụng. Ví dụ người dùng ấn vào một phím, kích chuột, nhấn lên một nút, một thanh cuộn,...

Xử lý sự kiện (event handling) là quá trình phát hiện sự kiện và thực hiện nhiệm vụ mà nó yêu cầu. Ví dụ khi người dùng nhấn vào nút Exit thì chương trình phải có nhiệm vụ phát hiện việc này và thoát ra khỏi chương trình đang chạy.

Vậy làm thế nào để phát hiện sự kiện và thực hiện nhiệm vụ ? Câu trả lời là phần mềm phải luôn nằm trong một vòng lặp để chờ sự kiện xảy ra để thực hiện. Tuy nhiên, người lập trình sẽ không phải viết vòng lặp này mà Java đã hỗ trợ việc này cho người lập trình. Cụ thể, Java sẽ luôn ở trạng thái lắng nghe sự kiện (event listening), khi sự kiện xảy ra, Java sẽ tự động gọi đến phương thức xử lý do người lập trình viết. Đây được gọi là mô hình điều khiển theo sự kiện (event – driven). Mô hình này được mô tả như sau :



Hình 7. 4 Mô hình tuần tự xử lý sự kiện

Mô hình trên gồm 2 thành phần chính :

Nguồn tạo ra sự kiện (Source): đây là một thành phần giao diện mà trên đó sự kiện xuất hiện. Ví dụ như button, textfield,... Source chịu trách nhiệm cung cấp thông tin về sự kiện đã xảy ra tới bộ xử lý Handler của nó.

Khối xử lý sự kiện (Listener): Còn được biết như là Event Handler. Listener chịu trách nhiệm tạo phản hồi tới một sự kiện. Theo quan điểm của Java, Listener cũng là một đối tượng. Khi một sự kiện đã được nhận, Listener xử lý sự kiện đó và sau đó trả về kết quả.

Các bước hoạt động của mô hình :

1. Thành phần giao diện đăng ký khối xử lý sự kiện tương ứng để khối xử lý sự kiện có thể xử lý sự kiện phù hợp với thành phần giao diện đó. Về bản chất, khối xử lý sự kiện là các lớp giao diện được cung cấp bởi Java. Trong các lớp giao diện này có các phương thức để xử lý các sự kiện được tạo ra bởi thành phần giao diện.

Ví dụ : Lớp xử lý sự kiện `MouseListener` có các phương thức để xử lý các sự kiện liên quan đến chuột :

```

public interface MouseListener {
    // Xử lý sự kiện khi chuột được ấn và giữ
    public void mousePressed(MouseEvent evt);
    // Xử lý sự kiện khi chuột được thả
    public void mouseReleased(MouseEvent evt);
    // Xử lý sự kiện khi chuột được ấn và thả
    public void mouseClicked(MouseEvent evt);
    // Xử lý sự kiện khi con trỏ chuột được đặt vào phạm vi của
    // thành phần giao diện

```

```

        public void mouseEntered(MouseEvent evt);
// Xử lý sự kiện khi con trỏ chuột được kéo ra khỏi phạm vi
của thành phần giao diện
        public void mouseExited(MouseEvent evt);
}

```

2. Thành phần sự kiện thay đổi trạng thái khi người dùng tác động.
3. Thành phần giao diện tạo ra đối tượng tương ứng với sự kiện xảy ra.
4. Thành phần giao diện gọi phương thức tương ứng của lớp xử lý sự kiện để xử lý và đối tượng vừa được tạo ra sẽ được truyền vào phương thức đó để xử lý.

7.5.2. Các bước lập trình xử lý sự kiện

Bước 1: Khai báo lớp xử lý sự kiện. Lớp này có nhiệm vụ implement lớp giao diện Listener

```

public class DemoClass extends Frame implements ActionListener
{
}

```

Bước 2: Gán một đối tượng xử lý sự kiện của lớp xử lý sự kiện được khai báo ở trên cho các components liên quan.

```

Component.addActionListener(instanceOf DemoClass);

```

Bước 3: Triển khai các phương thức xử lý sự kiện của lớp giao diện Listener

```

public void actionPerformed(ActionEvent e) {
    ...//code that reacts to the action...
}

```

7.5.3. Các lớp xử lý sự kiện

Sự kiện	Listener Interface	Mô tả
ActionEvent	ActionListener	Sự kiện xảy ra khi một nút được click, một danh sách các item được double click hoặc một menu được lựa chọn.
AdjustmentEvent	AdjustmentListener	Sự kiện xảy ra khi scrollbar được sử dụng.
ComponentEvent	ComponentListener	Sự kiện xảy ra khi component được thay đổi kích thước, di chuyển, ẩn hoặc hiện.
FocusEvent	FocusListener	Sự kiện xảy ra khi component được chuột hoặc bàn phím lựa chọn.

ItemEvent	ItemListener	Xảy ra khi menu item được chọn, giải phóng hoặc khi checkbox hoặc list item được click
WindowEvent	WindowListener	Xảy ra khi cửa sổ được activated, closed, opened hoặc quit.
TextEvent	TextListener	Xảy ra khi giá trị của text field hoặc text area thay đổi
MouseEvent	MouseListener MouseMotionListener	Xảy ra khi chuột được di chuyển, click, giữ hoặc giải phóng
KeyEvent	KeyListener	Xảy ra khi input được nhận từ keyboard

Bảng 7. 4 Một số lớp xử lý sự kiện

Ví dụ : Sử dụng thành phần giao diện Choice

```
import java.awt.*;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;
public class ChoiceList extends Frame implements ItemListener{
    Label lb = new Label(" ");
    Choice c = new Choice();
    ChoiceList() {
        this.setLayout(new FlowLayout());
        this.add(lb);
        this.add(c);
        c.addItem("Red");
        c.addItem("Blue");
        c.addItemListener(this);
    }
    public static void main(String[] args) {
        ChoiceList cl = new ChoiceList();
        cl.setSize(300,300);

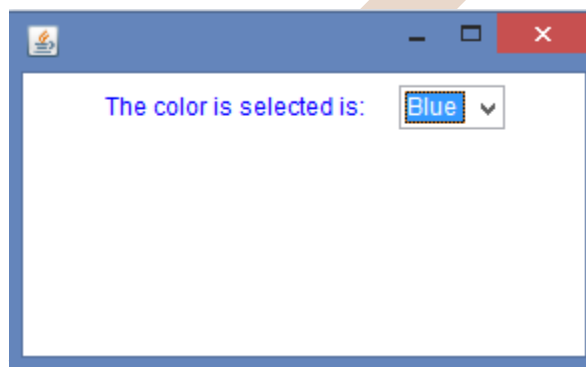
        cl.setVisible(true);
    }
}
```

```

@Override
public void itemStateChanged(ItemEvent e) {
    if(e.getItem()=="Red") {
        lb.setForeground(Color.RED);
        lb.setText("The color is selected is:");
    }
    if(e.getItem()=="Blue") {
        lb.setForeground(Color.BLUE);
        lb.setText("The color is selected is:");
    }
}
}

```

Kết quả :



Ví dụ : Sử dụng thành phần giao diện Button

```

import java.awt.*;
import java.awt.event.*;
public class SimpleFrame extends Frame implements ActionListener{
    Button bt1 = new Button("Exit");
    TextField tf1 = new TextField(20);
    Button bt2 = new Button("Double");
    TextField tf2 = new TextField(20);
    SimpleFrame() {
        setLayout(new FlowLayout());
        this.add(bt1);
        this.add(tf1);
        this.add(bt2);
        this.add(tf2);
        bt1.addActionListener(this);
        bt2.addActionListener(this);
    }
}

```

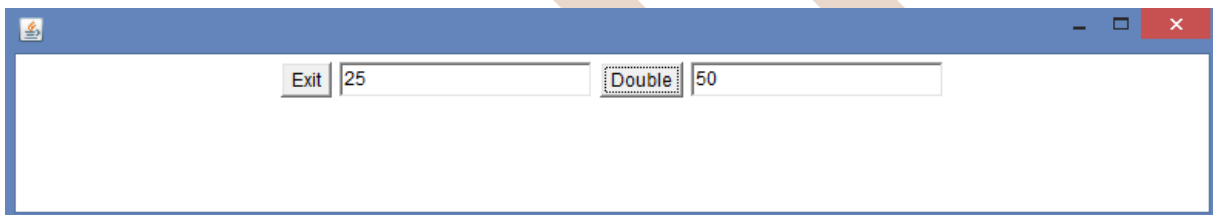
```

    public void actionPerformed(ActionEvent e){
        if(e.getSource()==bt1){
            System.exit(0);
        }
        if(e.getSource()==bt2){
            int temp = Integer.parseInt(tf1.getText())*2;
            tf2.setText(String.valueOf(temp));
        }
    }

    public static void main(String args[]){
        SimpleFrame sf= new SimpleFrame();
        sf.setSize(400,400);
        sf.setVisible(true);
    }
}

```

Kết quả :



7.6. Applet

7.6.1. Khái niệm Applet

Applet là một chương trình Java được chạy với sự hỗ trợ của web browser. Về bản chất, Applet cũng giống như một Frame nhưng Frame dùng cho các ứng dụng chạy độc lập trong khi Applet dùng cho các ứng dụng chạy trên web. Lớp Applet nằm trong gói java.applet của Java. Để tạo một Applet, ta cần import hai gói sau:

- Java.applet
- Java.awt

7.6.2. Cấu trúc của Applet

Mỗi Applet có 4 phương thức mặc định sẽ chạy tự động là init(), start(), stop() và destroy().

init – Hàm này được gọi trong quá trình khởi tạo applet. Trong quá trình khởi tạo, một đối tượng cụ thể của Applet được tạo ra. Ngoài ra, phương thức này cũng được sử dụng để nạp các đối tượng đồ họa, khởi tạo các biến và tạo các đối tượng.

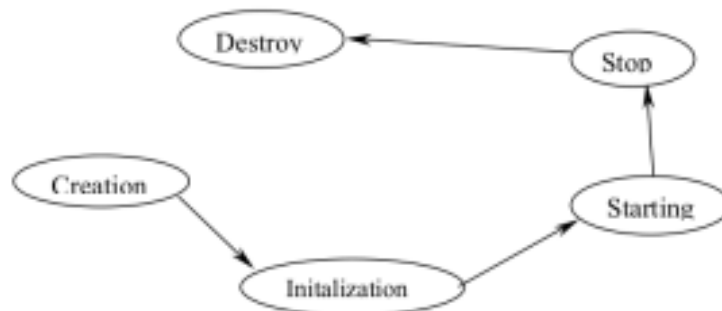
start- Khi hàm start được gọi nó sẽ thực hiện thực thi chương trình applet trên trình duyệt của bạn. Ngoài ra hàm này cũng được dùng để khởi động lại applet sau khi tạm dừng.

stop – Hàm này ngược lại với start, nó sẽ dừng hoạt động của chương trình applet khi đang được thực thi.

destroy – Hàm này thực hiện đóng hẳn chương trình applet.

Ngoài các phương thức chính ở trên thì chương trình còn một số phương thức khác như repaint(), paint(), paintComponents(),... Phương thức repaint() được dùng để cập nhật lại toàn bộ hiển thị trên cửa sổ Applet. Hai phương thức còn lại sẽ được dùng để vẽ các hình ảnh, màu sắc, chữ, hình học,... lên Applet thông qua đối tượng Graphics.

Hình ảnh dưới đây sẽ mô tả một chu trình hoạt động của Applet và cách Applet được thực hiện.



Hình 7. 5 Chu trình sống của Applet

- Bước đầu tiên khi chạy một applet là quá trình “khởi tạo”.
- Sau khi khởi tạo xong, applet “bắt đầu quá trình làm việc”. Bước này có thể được lặp lại nếu trước đó applet bị tạm dừng. Ví dụ nếu người dùng chuyển đến một trang web mới và sau đó quay trở lại trang chứa applet.
- Sự khác biệt giữa init() và start() là start() có thể được gọi nhiều lần nhưng init() chỉ được gọi một lần duy nhất.
- Phương thức stop() được gọi khi người dùng rời trang web hoặc thu nhỏ trang web.
- Phương thức destroy() được gọi khi người dùng tắt trang web. Phương thức này giúp giải phóng bộ nhớ và một số công việc nhằm “dọn dẹp” tài nguyên.

Sự khác nhau giữa Application và Applet:

- Để một Application được thực thi thì chúng ta phải dùng trình thông dịch java, trong khi đó applet có thể chạy được trên các trình duyệt có hỗ trợ java hoặc sử dụng AppletViewer.
- Quá trình thực thi của Application bắt đầu từ hàm main() còn Applet thì bắt đầu bằng hàm init().
- Application sử dụng lệnh system.out.println() để hiển thị kết quả ra màn hình console, trong khi đó Applet sử dụng phương thức drawString() để đưa một dòng chữ lên màn hình.

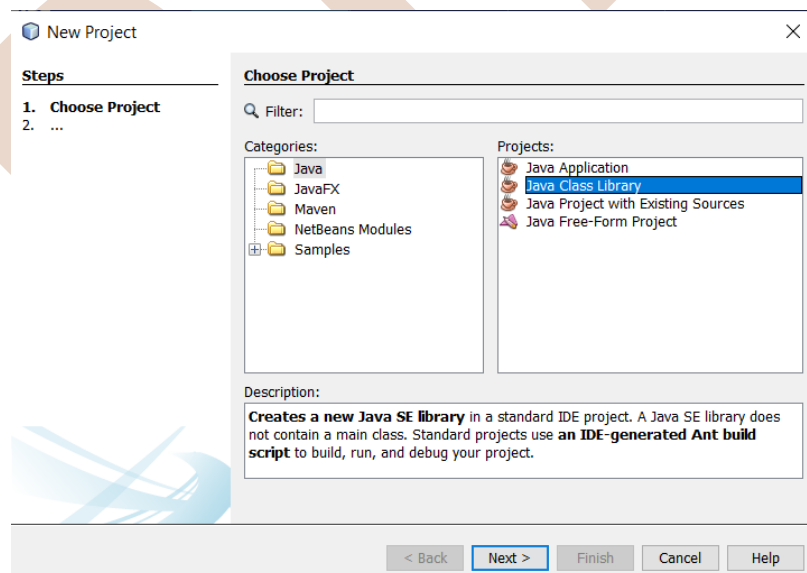
Vì Applet chạy trên web nên có một số giới hạn về hoạt động để chương trình Applet không làm ảnh hưởng đến hệ thống web của người sử dụng như không thể xóa file, lấy thông tin của hệ thống,... Cụ thể, Applet có một số hạn chế sau:

- Không thể đọc hoặc ghi file trên hệ thống file của người dùng.
- Chỉ có thể giao tiếp giữa hai website nếu hai website này đều có chương trình Applet.
- Không thể chạy bất kì chương trình gì trên hệ thống của người dùng.
- Không thể tải bất kì chương trình được lưu trữ trong hệ thống của người dùng.

Ví dụ về một chương trình Applet trên Netbeans:

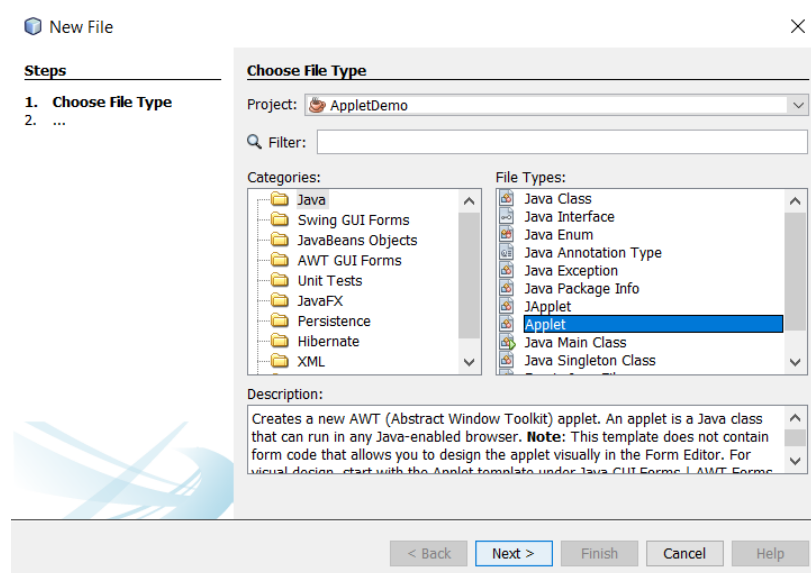
Giả sử ta tạo một lớp AppletExample kế thừa lớp Applet :

Bước 1 : Chọn New Project :



Bước 2 : Chọn Next -> nhập tên Project

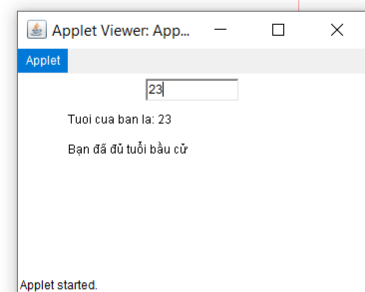
Bước 3 : Tại cửa sổ bên trái, click phải chuột vào Project Applet Example và chọn New -> Other -> New File



Bước 4 : Chọn Next -> Nhập tên file : AppletExample

Bước 5 : Viết chương trình cho người dùng nhập tuổi vào TextField. Nếu tuổi nhỏ hơn 18 thì báo người dùng là “Chưa đủ tuổi bầu cử”. Ngược lại, chương trình thông báo người dùng “Bạn đã đủ tuổi bầu cử”.

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
public class AppletExample extends Applet implements ActionListener {
    TextField tf = new TextField(10);
    int value = 0;
    public void init() {
        this.add(tf);
        tf.addActionListener(this);
    }
    public void paint(Graphics g) {
        g.drawString("Tuoi cua ban la: " + value, 50, 50);
        if(value >= 18)
            g.drawString("Bạn đã đủ tuổi bầu cử", 50, 80);
        else g.drawString("Bạn chưa đủ tuổi bầu cử", 50, 80);
    }
    public void textValueChanged(TextEvent e) {
        value = Integer.parseInt(tf.getText());
        repaint();
    }
}
```



Trong chương trình trên ta thấy cả hai phương thức `init()` và `paint()` đều được cài đặt. Một ứng dụng frame được chạy khi nó có hàm `main`, còn applet được chạy khi nó có hàm `init`, `paint`, `start`, `stop`,... và đặc biệt nó phải được kế thừa từ đối tượng `Applet`. Khi Applet được chạy nó sẽ tự động gọi tới hàm `init()` để khởi tạo những gì có trong nó trước, sau đó sẽ gọi tới hàm `paint()` và thực hiện nội dung ở trong hàm này. Phương thức `paint()` được sử dụng để hiển thị các văn bản và các hình đồ họa lên màn hình. Nó được gọi bởi trình duyệt Web hoặc trình applet viewer bất cứ khi nào cần phải vẽ lên màn hình.

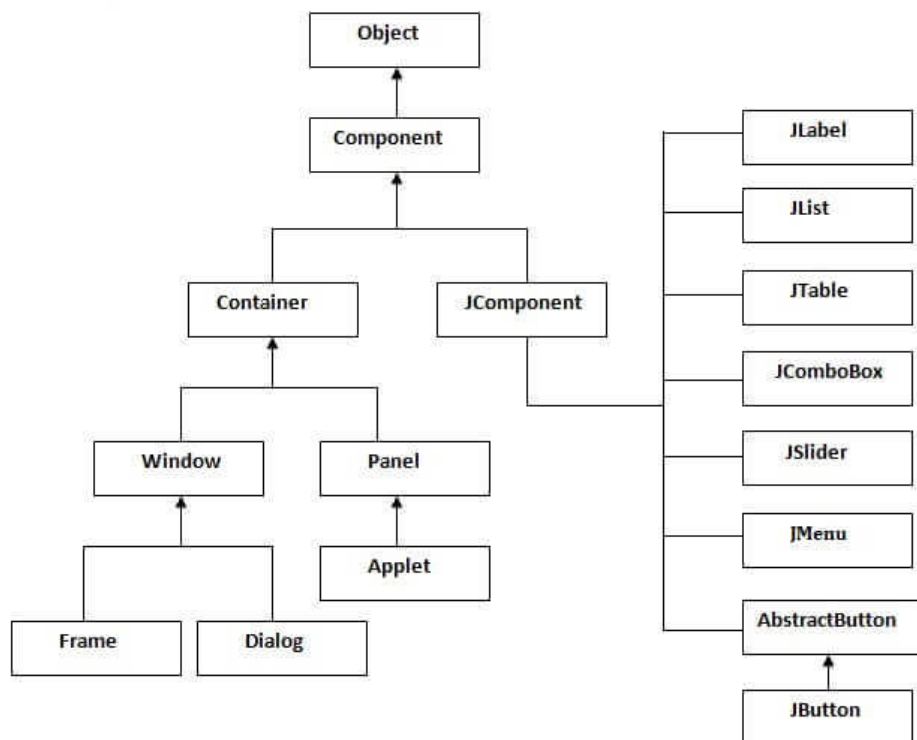
7.7. Lập trình giao diện với Swing

Java Swing là một trong các lớp nền tảng của Java (Java Foundation Classes - JFC) được sử dụng để tạo các ứng dụng dựa trên cửa sổ. Nó được phát triển dựa trên các API AWT và hoàn toàn được viết bằng java. Tuy nhiên, không giống như AWT, Java Swing cung cấp các thành phần đồ họa nhẹ và độc lập với nền tảng.

Gói javax.swing cung cấp các lớp cho API swing java như JButton, JTextField, JTextArea, JRadioButton, JCheckBox, JMothy, JColorChooser, v.v. Cấu trúc cây gia phả của Java Swing được thể hiện như hình 7.6

7.7.1. Tạo các thành phần giao diện với Java Swing

Tương tự như với AWT, chúng ta có thể tạo giao diện trên Java Swing bằng cách viết lệnh trực tiếp. Ngoài ra, ta có thể dùng cách kéo thả các thành phần giao diện vào trong cửa sổ. Với cách này, chúng ta có thể chủ động sắp xếp các thành phần trong giao diện vào các vị trí một cách trực quan.



Hình 7. 6 Cây gia phả của các lớp trong SWING

Ví dụ :

```
import javax.swing.JFrame;

public class Test extends JFrame {
    public Test() {
        setSize(300, 200);
        setTitle("Test");
    }
}
```

```

        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }
    public static void main(String[] args) {
        Test test = new Test();
        test.setVisible(true);
    }
}

```

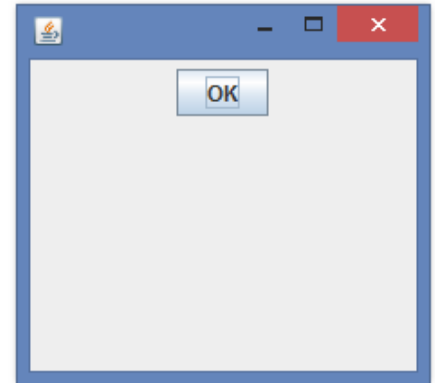
7.7.2. Một số ví dụ với Swing

Ví dụ 1 : Tạo nút bấm

```

package swingdemo;
import java.awt.FlowLayout;
import javax.swing.JButton;
import javax.swing.JFrame;
public class SwingDemo extends JFrame{
    JButton jb = new JButton("OK");
    SwingDemo() {
        setSize(300,300);
        setLayout(new FlowLayout());
        this.add(jb);
        jb.setBounds(20,20,30,30);
    }
    public static void main(String[] args) {
        // TODO code application logic here
        SwingDemo sd = new SwingDemo();
        sd.setVisible(true);
    }
}

```



Ví dụ 2 : Tạo Bảng với Swing

```

package swingdemo;
import javax.swing.*;
public class SwingDemo extends JFrame{
    String data[][]={ {"101","Amit","670000"},
                      {"102","Jai","780000"},
                      {"101","Sachin","700000"} };
    String column[]={"ID","NAME","SALARY"};
    JTable jt=new JTable(data,column);
    JScrollPane sp=new JScrollPane(jt);
    SwingDemo() {
        this.add(sp);
    }
    public static void main(String[] args) {
        SwingDemo sd = new SwingDemo();
        sd.setSize(300,300);
        sd.setVisible(true);
    }
}

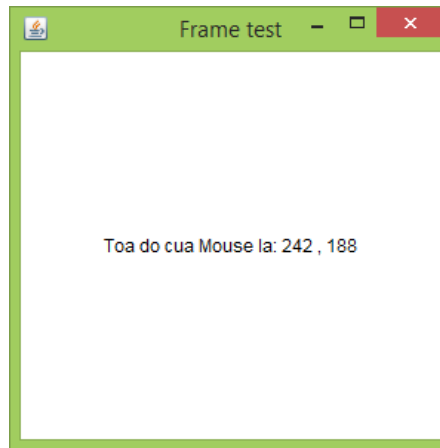
```

A screenshot of a Java Swing window titled "SwingDemo". The window contains a table with 3 columns: "ID", "NAME", and "SALARY". The table has 3 rows of data. Below the table is a large empty gray area.

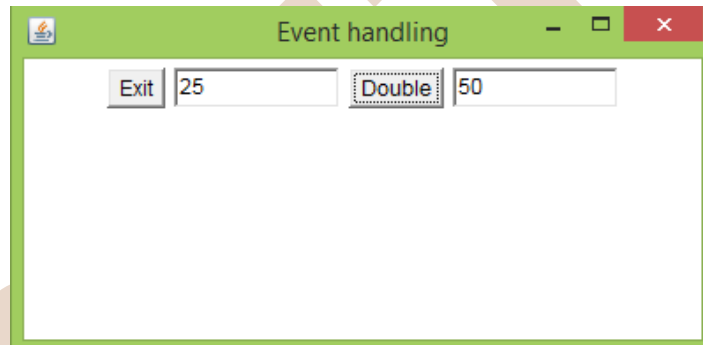
ID	NAME	SALARY
101	Amit	670000
102	Jai	780000
101	Sachin	700000

BÀI TẬP CHƯƠNG 7

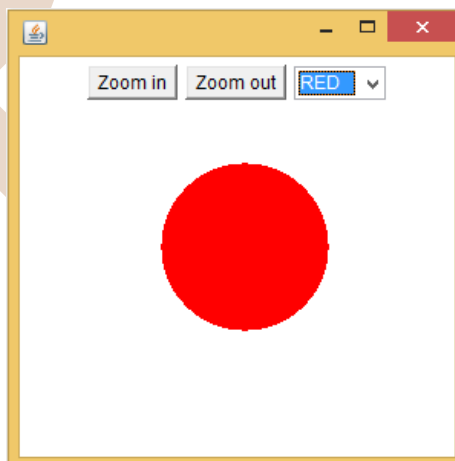
Bài 1 : Viết chương trình cho phép người dùng click chuột vào giao diện và hiển thị tọa độ của chuột khi người dùng click :



Bài 2 : Tạo giao diện như hình vẽ :



Bài 3 : Tạo giao diện với các chức năng như hình vẽ :



TÀI LIỆU THAM KHẢO

1. Kathy Sierra , Bert Bates. *Head First Java*. O'reilly Media, Inc, 2005.
2. Bruce Eckel. *Thinking in Java*. President, MindView, Inc, 2006.
3. Nguyễn Tiên, Ngô Quốc Việt, Phạm Nguyễn Tuấn Kỳ. *Giáo trình lý thuyết và thực hành Java 2*. Nhà xuất bản thống kê, 2000.
4. Trần Đình Quế. *Lập trình hướng đối tượng*. Học Viện Công Nghệ Bưu Chính Viễn Thông

PDF